

Avicenna: Fail-Slow Fault-Tolerant Replicated State Machines using Counterfactual Evaluations

Zijian Qin

Outlines

- Replicated State Machines
- Fail-slow Faults in Distributed Systems
- Overview of Avicenna
- Detailed design of Avicenna
- Evaluations
- Discussion and Conclusion

Outlines

- Replicated State Machines
- Fail-slow Faults in Distributed Systems
- Overview of Avicenna
- Detailed design of Avicenna
- Evaluations
- Discussion and Conclusion

Replicated State Machines

Replicated State Machines

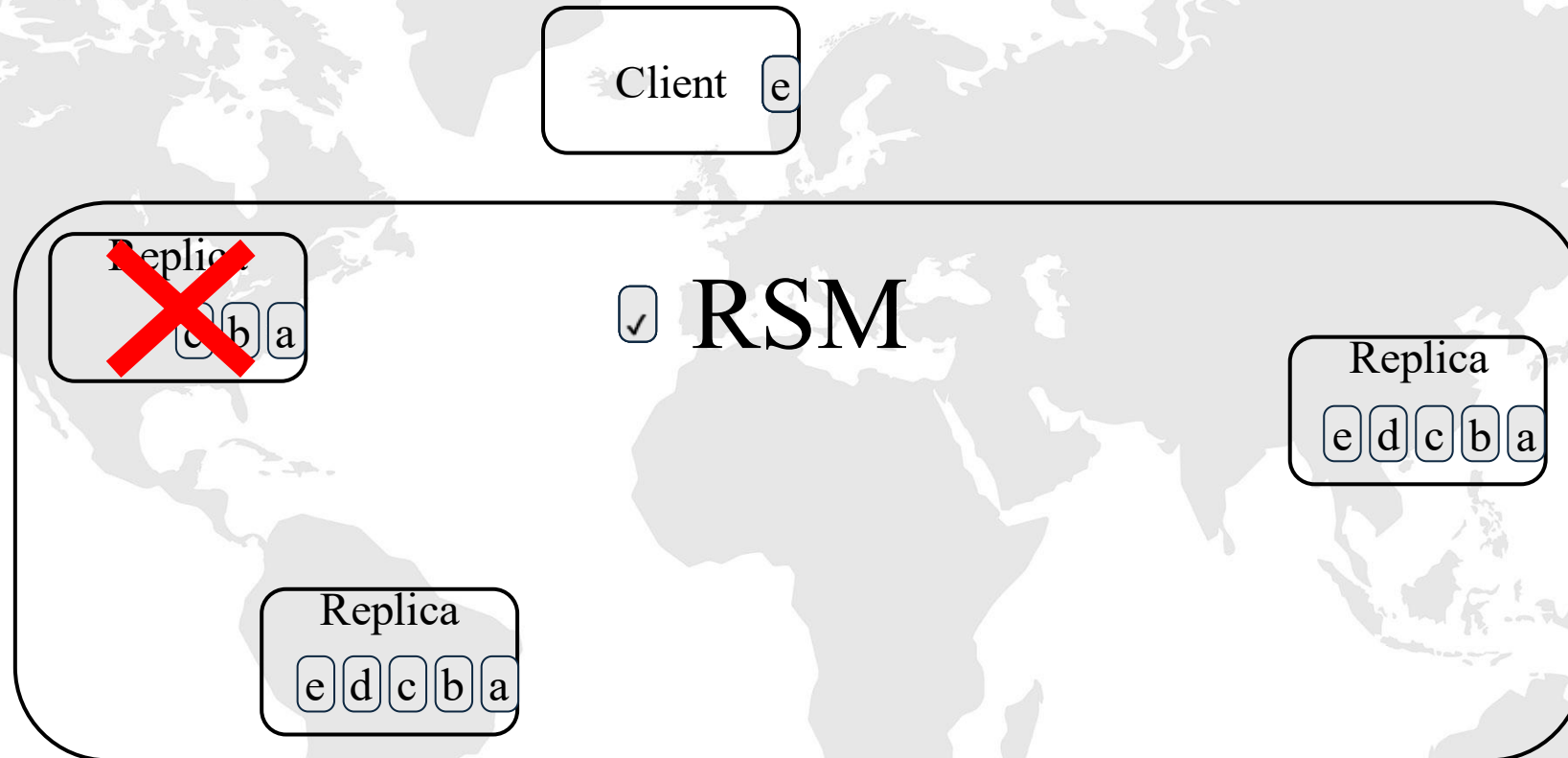
- Fault-tolerant groups of replicas that acts like a single machine that does not fail.

Replicated State Machines

- Fault-tolerant groups of replicas that acts like a single machine that does not fail.
- RSMs are everywhere!
distributed databases, coordination services, etc.



Fault Tolerance for High Availability



How Replicated State Machine Works?

How Replicated State Machine Works?

Same initial state,
same sequence of state transitions

How Replicated State Machine Works?

Replica 1	
key	value
x	0
y	0
z	0

Replica 2	
key	value
x	0
y	0
z	0

Replica 3	
key	value
x	0
y	0
z	0

Same initial state,
same sequence of state transitions

How Replicated State Machine Works?

Replica 1	
key	value
x	0
y	0
z	0

WRITE (x, 1)

Replica 1	
key	value
x	1
y	0
z	0

Replica 2	
key	value
x	0
y	0
z	0

WRITE (x, 1)

Replica 2	
key	value
x	1
y	0
z	0

Replica 3	
key	value
x	0
y	0
z	0

WRITE (x, 1)

Replica 3	
key	value
x	1
y	0
z	0

Same initial state,
same sequence of state transitions

How Replicated State Machine Works?

Replica 1	
key	value
x	0
y	0
z	0

WRITE (x, 1)

Replica 1	
key	value
x	1
y	0
z	0

WRITE (y, 5)

Replica 1	
key	value
x	1
y	5
z	0

WRITE (z, 8)

Replica 2	
key	value
x	0
y	0
z	0

WRITE (x, 1)

Replica 2	
key	value
x	1
y	0
z	0

WRITE (y, 5)

Replica 2	
key	value
x	1
y	5
z	0

WRITE (z, 8)

Replica 3	
key	value
x	0
y	0
z	0

WRITE (x, 1)

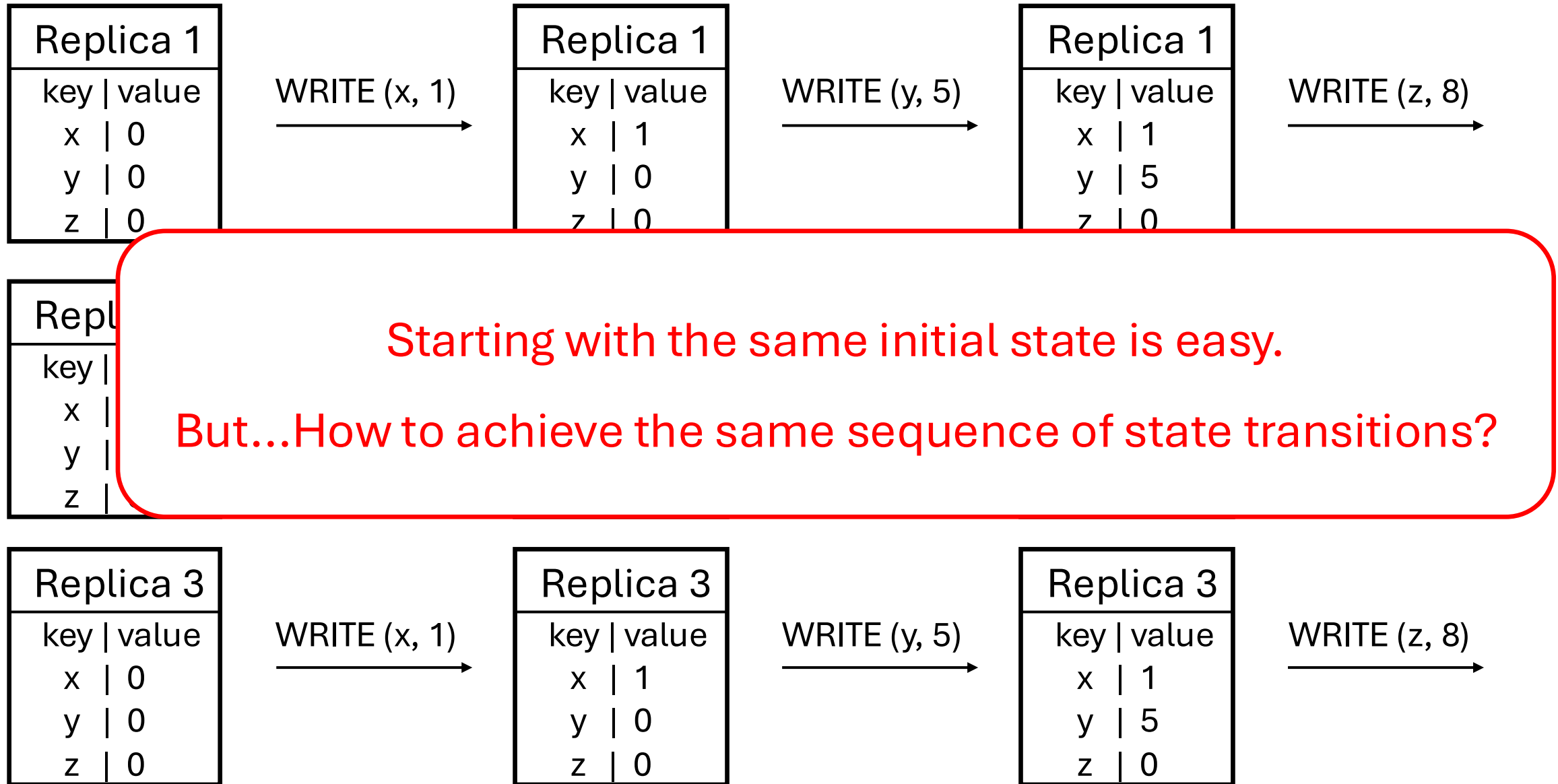
Replica 3	
key	value
x	1
y	0
z	0

WRITE (y, 5)

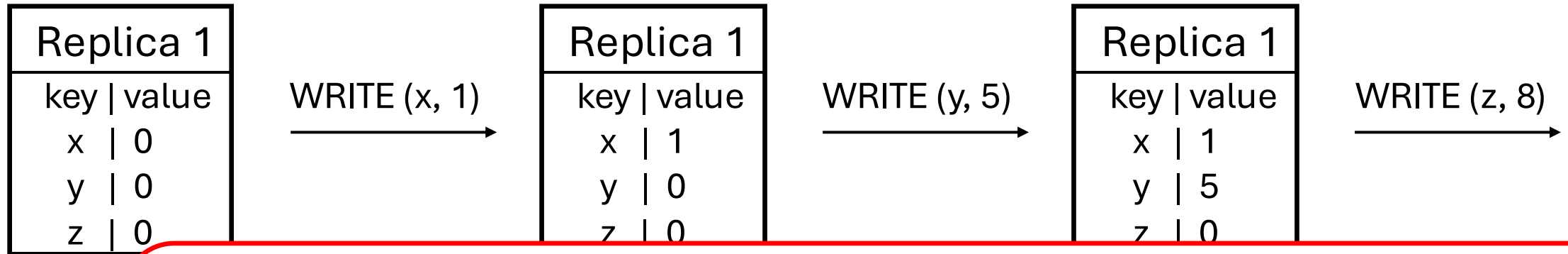
Replica 3	
key	value
x	1
y	5
z	0

WRITE (z, 8)

How Replicated State Machine Works?



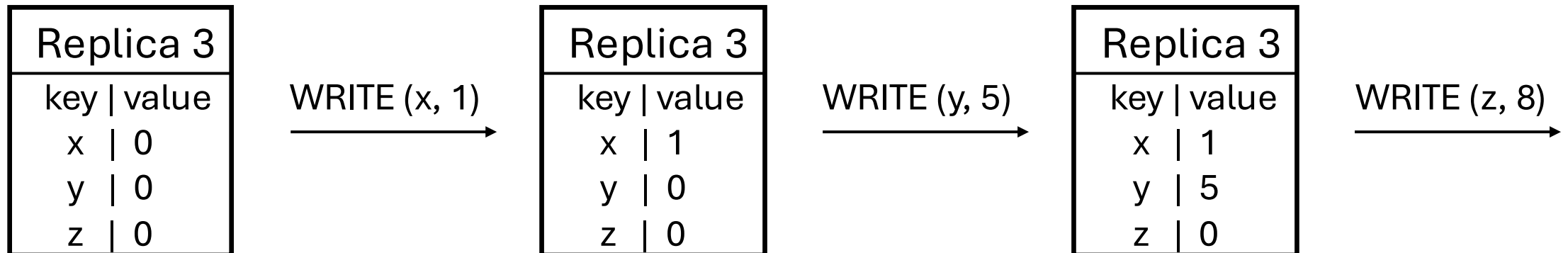
How Replicated State Machine Works?



Starting with the same initial state is easy.

But...How to achieve the same sequence of state transitions?

By Consensus algorithms.



Canonical Consensus Algorithm: Multi-Paxos

Canonical Consensus Algorithm: Multi-Paxos

Client



Leader

Replica



Follower

Replica



Follower

Replica



Follower

Replica

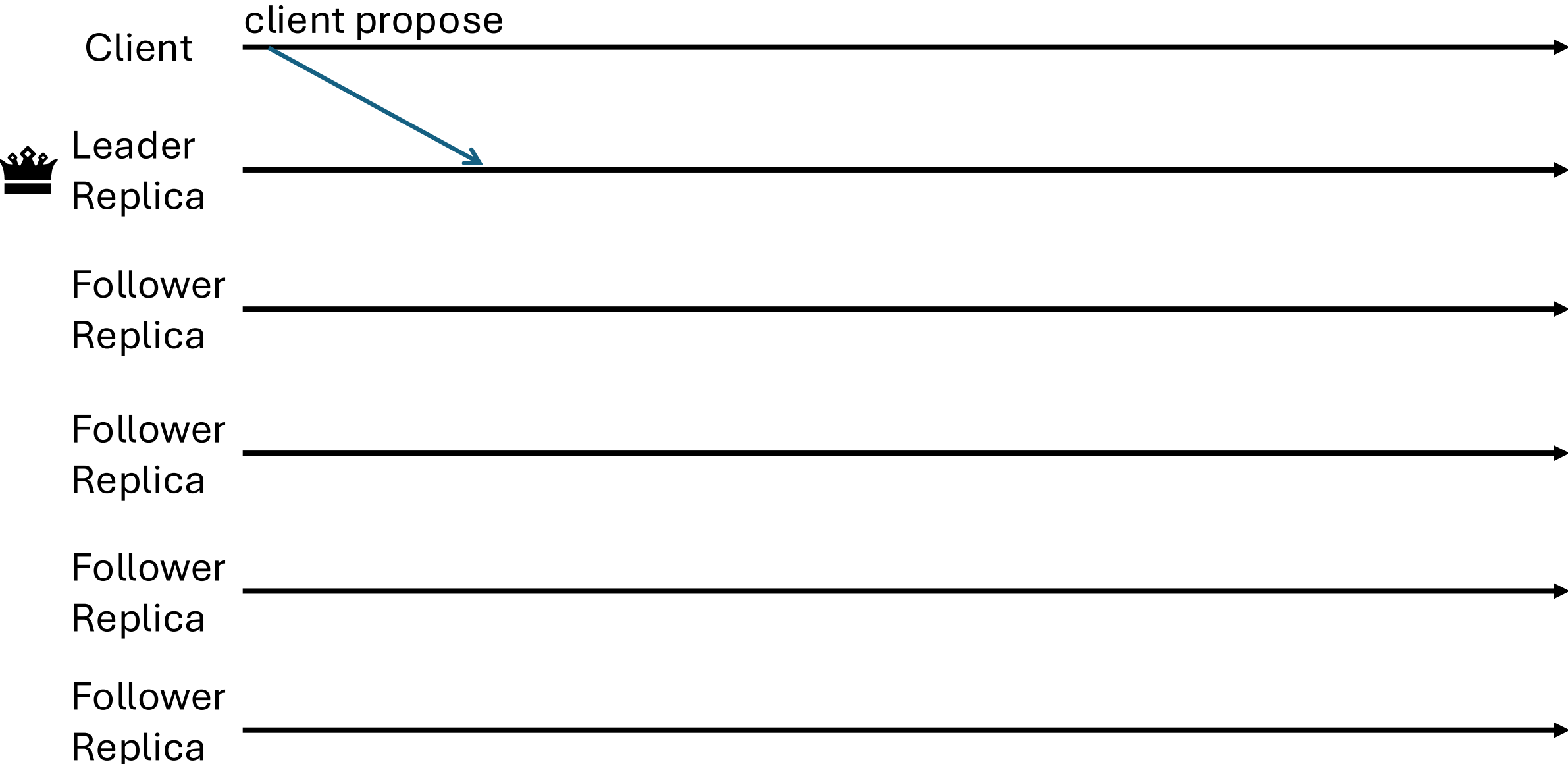


Follower

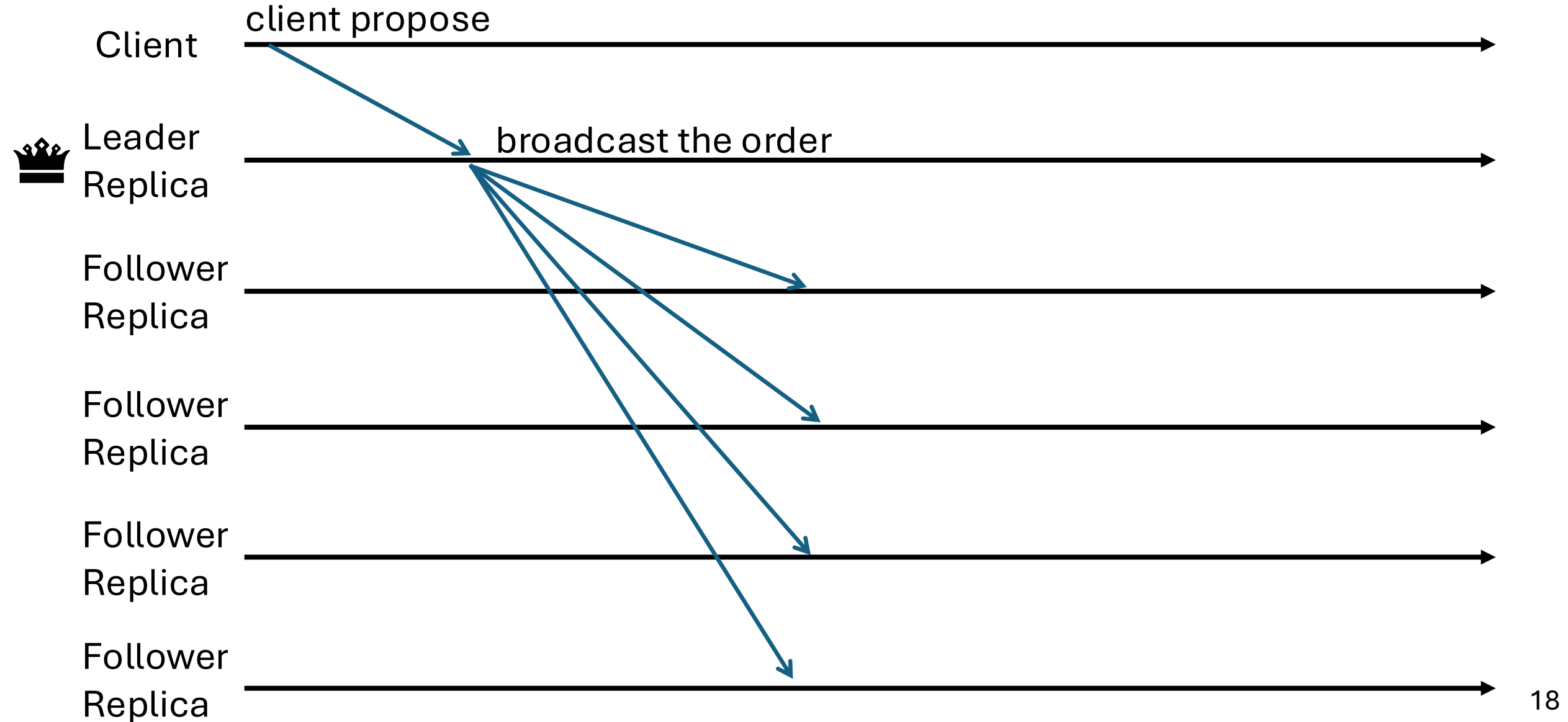
Replica



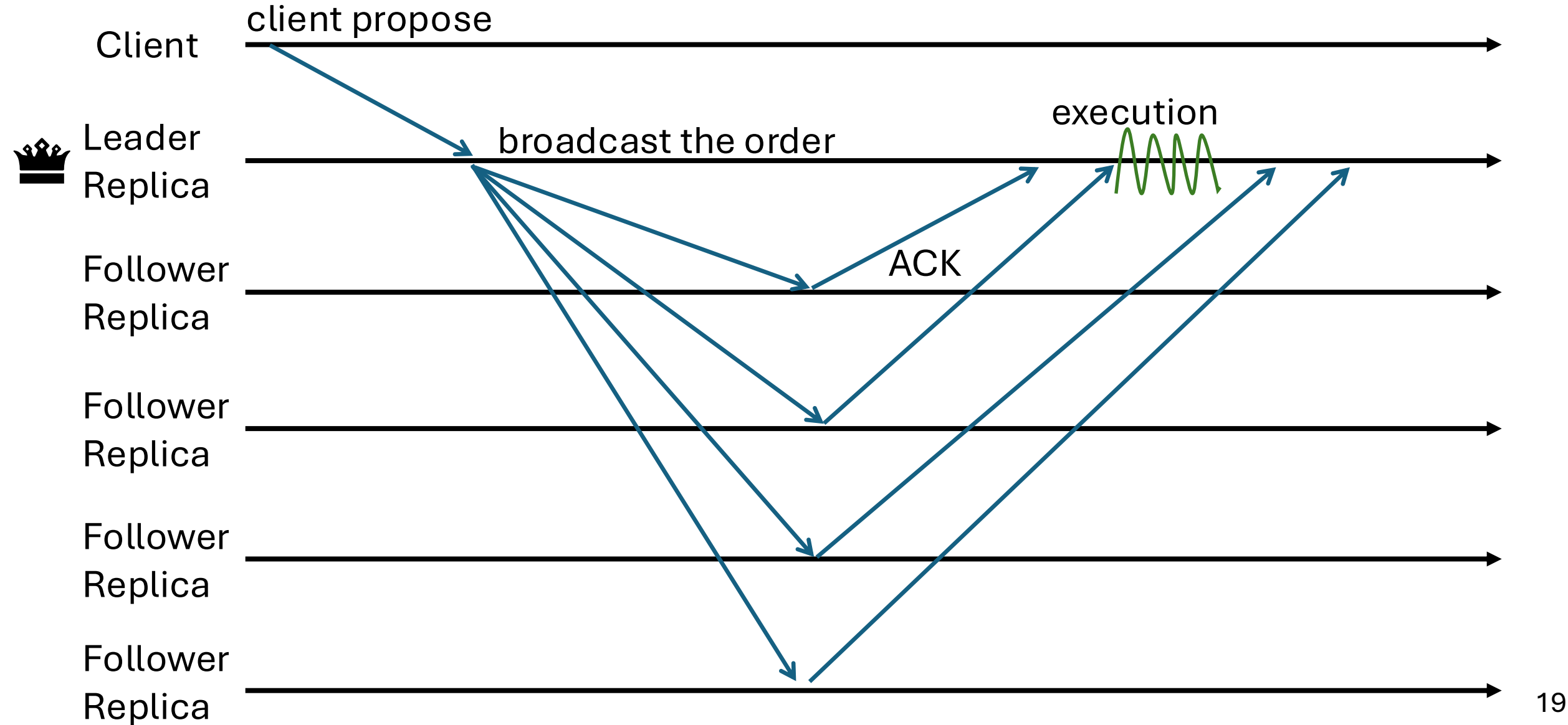
Canonical Consensus Algorithm: Multi-Paxos



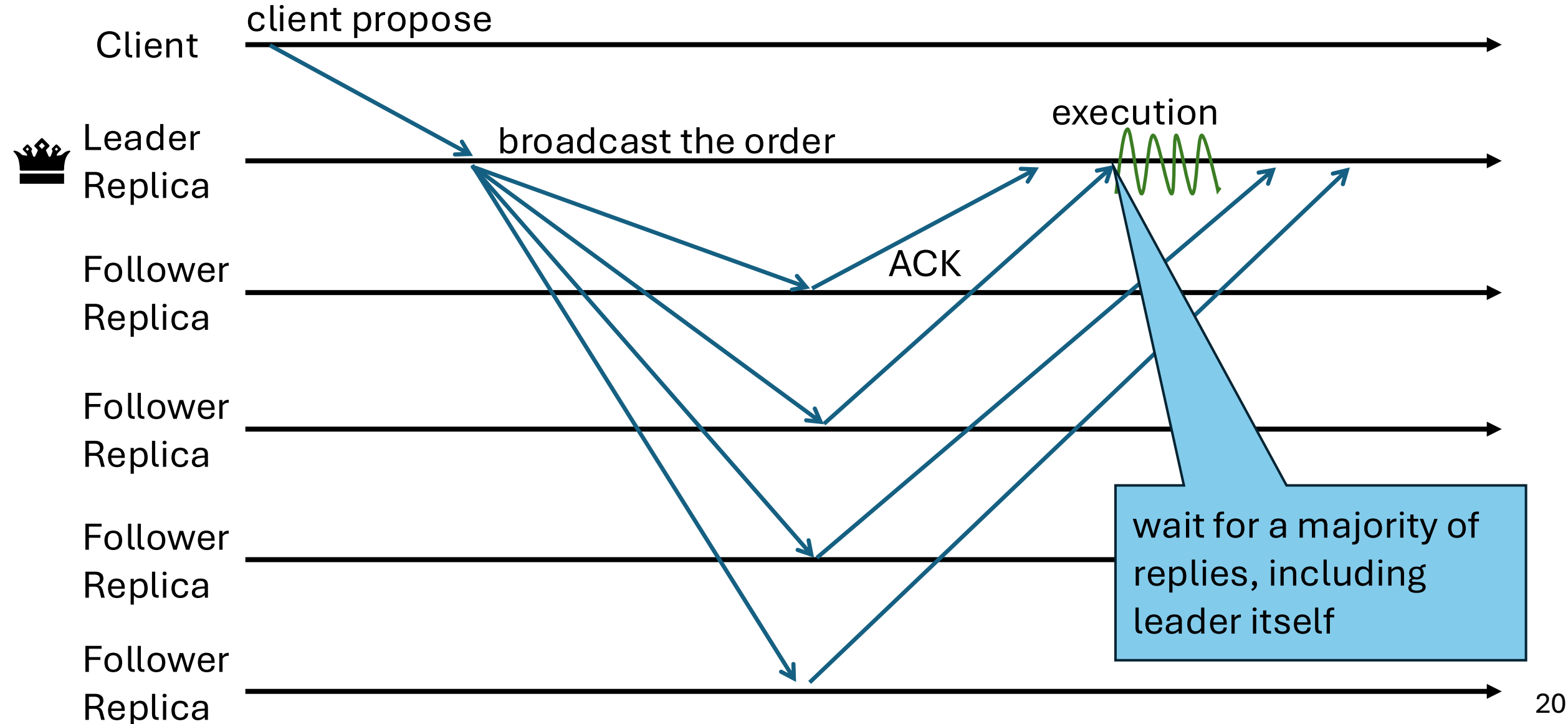
Canonical Consensus Algorithm: Multi-Paxos



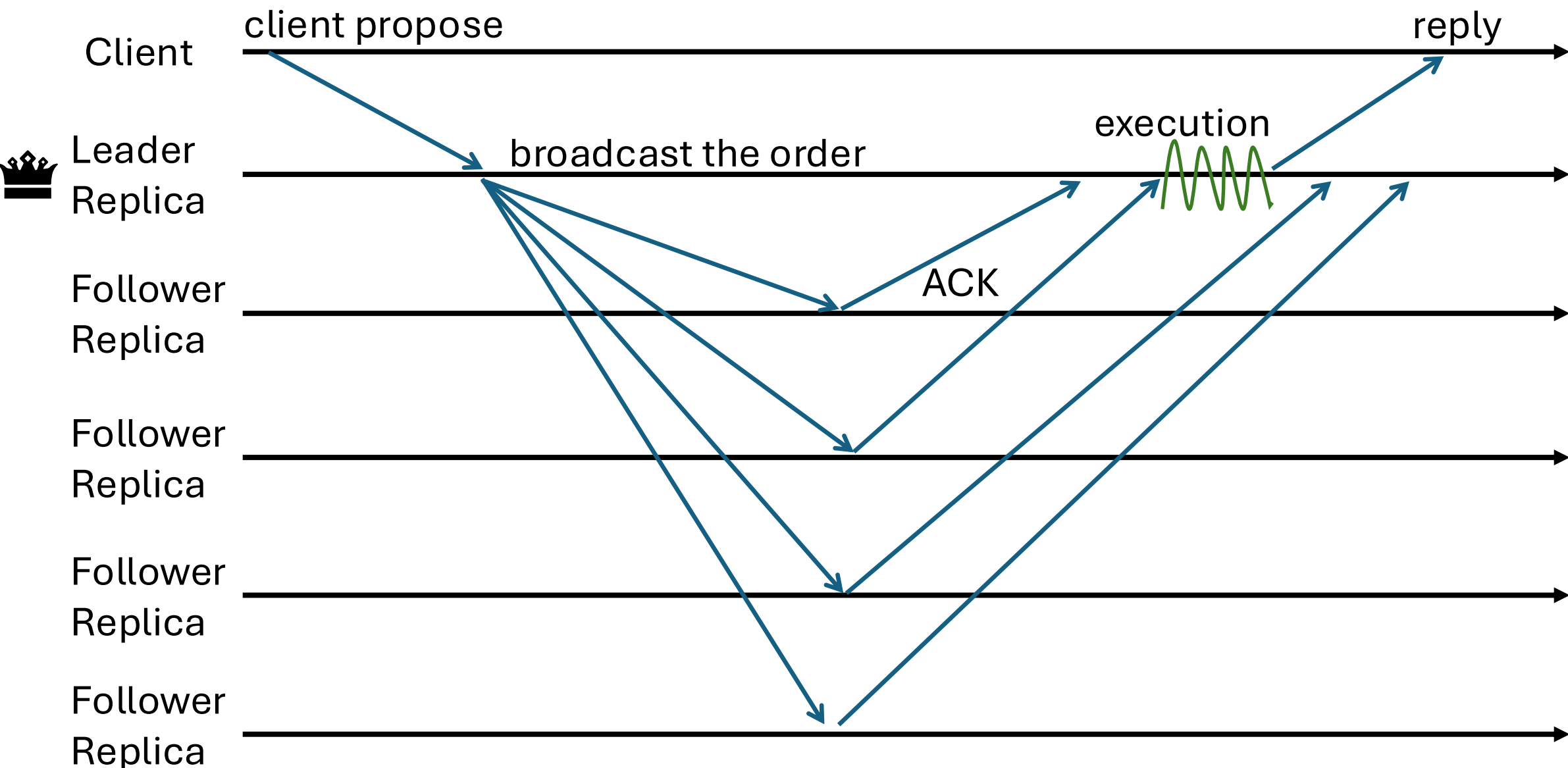
Canonical Consensus Algorithm: Multi-Paxos



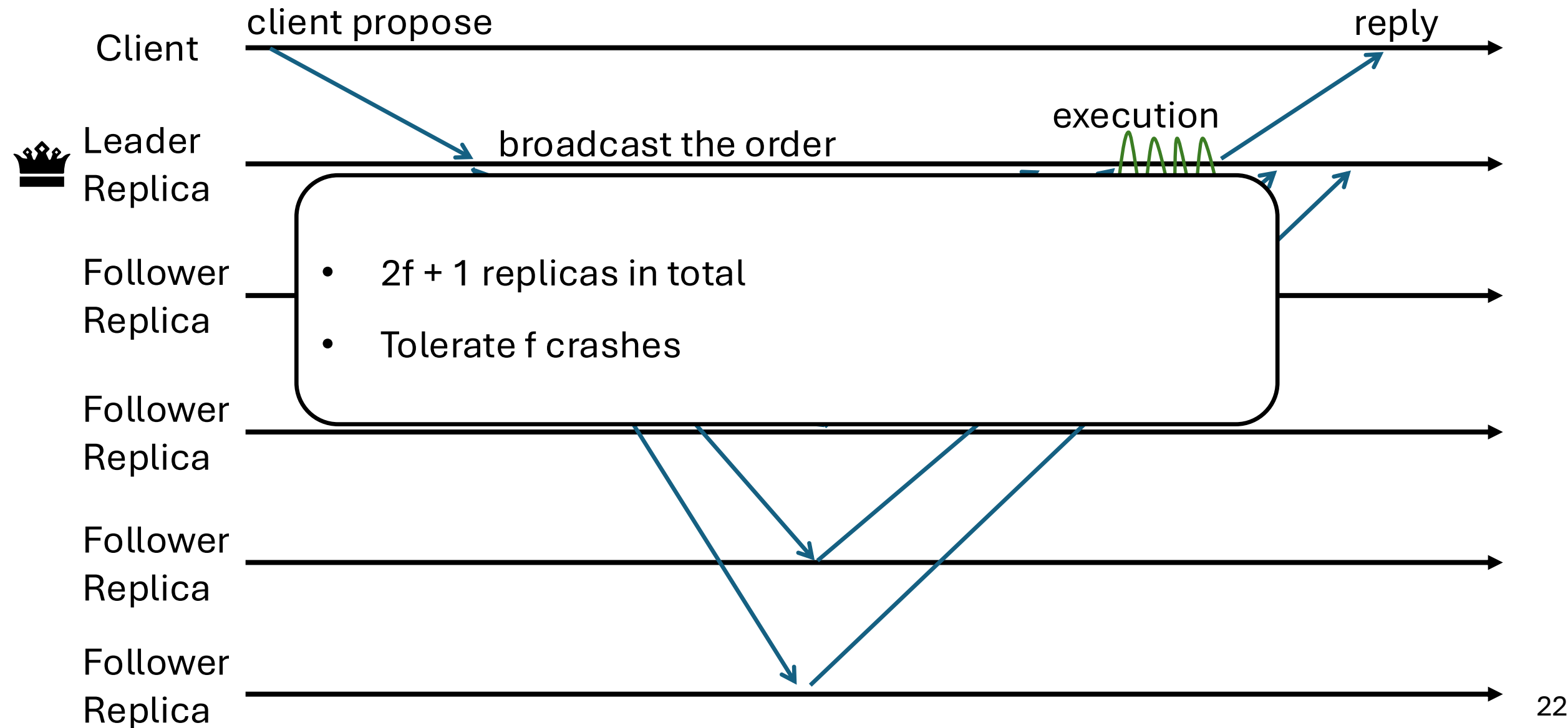
Canonical Consensus Algorithm: Multi-Paxos



Canonical Consensus Algorithm: Multi-Paxos



Canonical Consensus Algorithm: Multi-Paxos



Outlines

- Replicated State Machines
- **Fail-slow Faults in Distributed Systems**
- Overview of Avicenna
- Detailed design of Avicenna
- Evaluations
- Discussion and Conclusion

What are Fail-Slow Faults in Distributed Systems?

What are Fail-Slow Faults in Distributed Systems?

- Fail-slow Faults: a machine functions **correctly** but with **degrade performance**

What are Fail-Slow Faults in Distributed Systems?

- Fail-slow Faults: a machine functions **correctly** but with **degrade performance**



What are Fail-Slow Faults in Distributed Systems?

- Fail-slow Faults: a machine functions **correctly** but with **degrade performance**

stop completely



Crash

high-speed

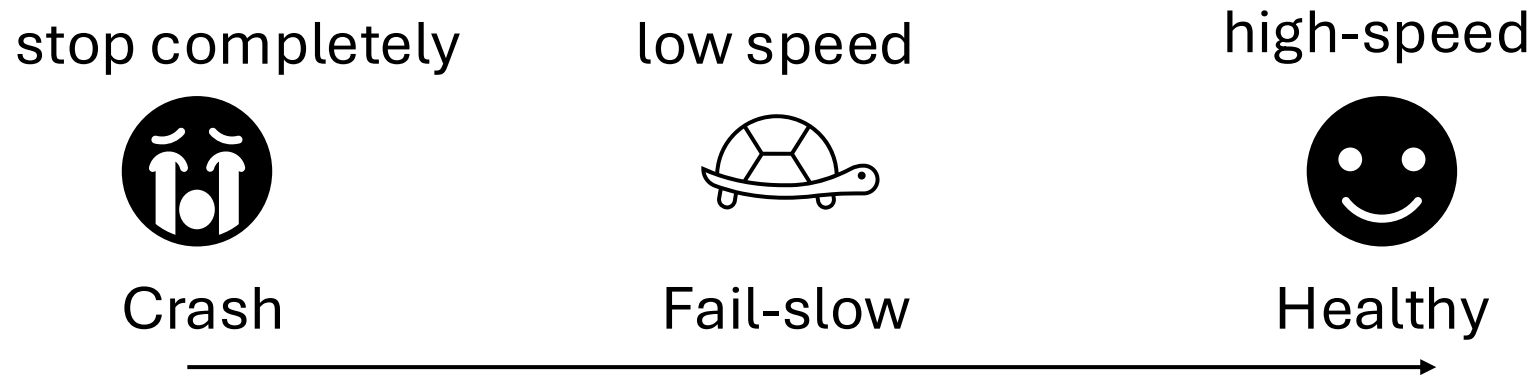


Healthy



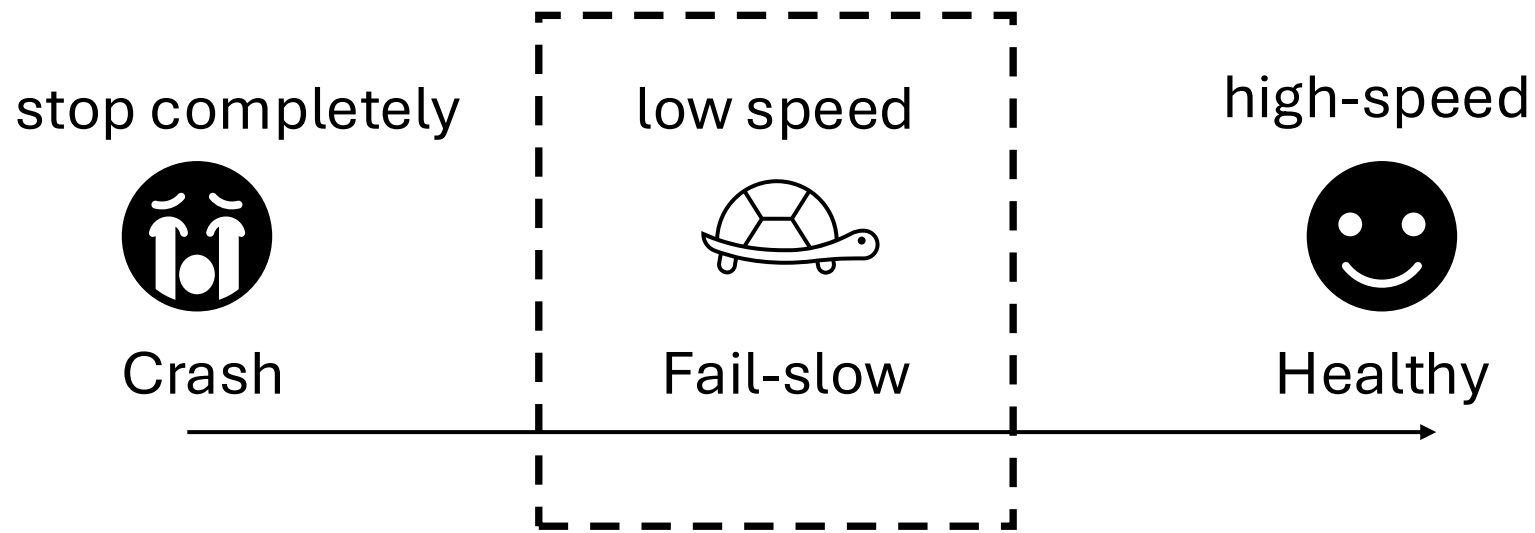
What are Fail-Slow Faults in Distributed Systems?

- Fail-slow Faults: a machine functions **correctly** but with **degrade performance**



What are Fail-Slow Faults in Distributed Systems?

- Fail-slow Faults: a machine functions **correctly** but with **degrade performance**



Avicenna: fail-slow tolerant replicated state machine

Examples of Fail-Slow Faults

NIC overheating / firmware bugs
Software / OS scheduling issues
Network congestion / packet delays
Partially-failed power supplies
etc.

Why Care about Fail-Slow Faults?

Why Care about Fail-Slow Faults?

- Fail-slow faults are common
can be as frequent as crash failures

Why Care about Fail-Slow Faults?

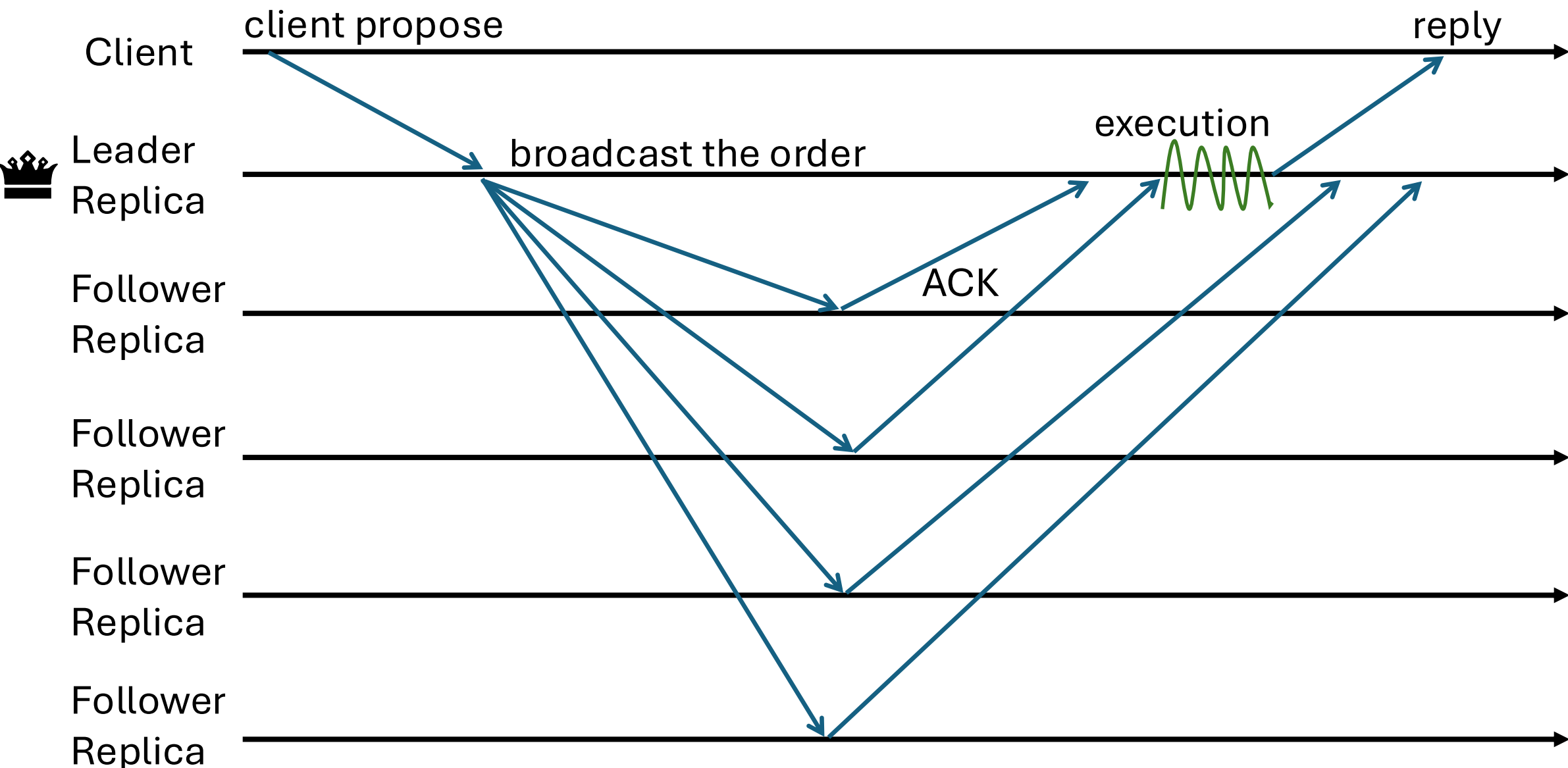
- Fail-slow faults are common
can be as frequent as crash failures
- Fail-slow fault is a severe problem
component - machine – cluster

Why Care about Fail-Slow Faults?

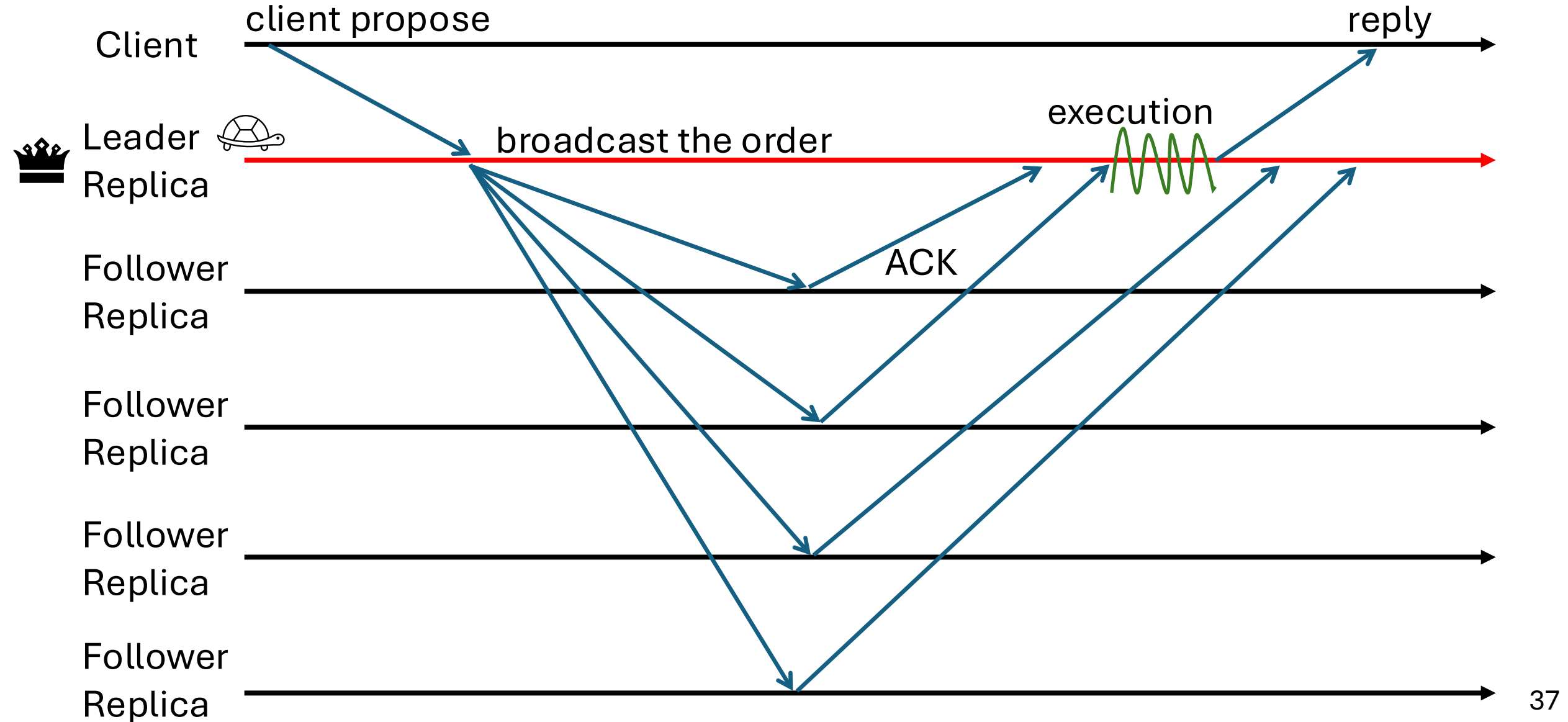
- Fail-slow faults are common
can be as frequent as crash failures
- Fail-slow fault is a severe problem
component - machine – cluster
- Fail-slow faults are hard to handle
system designers assume components to be either healthy or crash

Multi-Paxos is Not Fail-Flow Fault-Tolerant

Multi-Paxos is Not Fail-Flow Fault-Tolerant



Multi-Paxos is Not Fail-Flow Fault-Tolerant



Outlines

- Replicated State Machines
- Fail-slow Faults in Distributed Systems
- **Overview of Avicenna**
- Detailed design of Avicenna
- Evaluations
- Discussion and Conclusion

Fail-Slow Fault Tolerance vs Normal-Case Latency

Protocol	Fail-slow tolerance	Normal-Case Latency

Fail-Slow Fault Tolerance vs Normal-Case Latency

Protocol	Fail-slow tolerance	Normal-Case Latency
Multi-Paxos (a single leader)	No	Low (1 RTT to commit)

Fail-Slow Fault Tolerance vs Normal-Case Latency

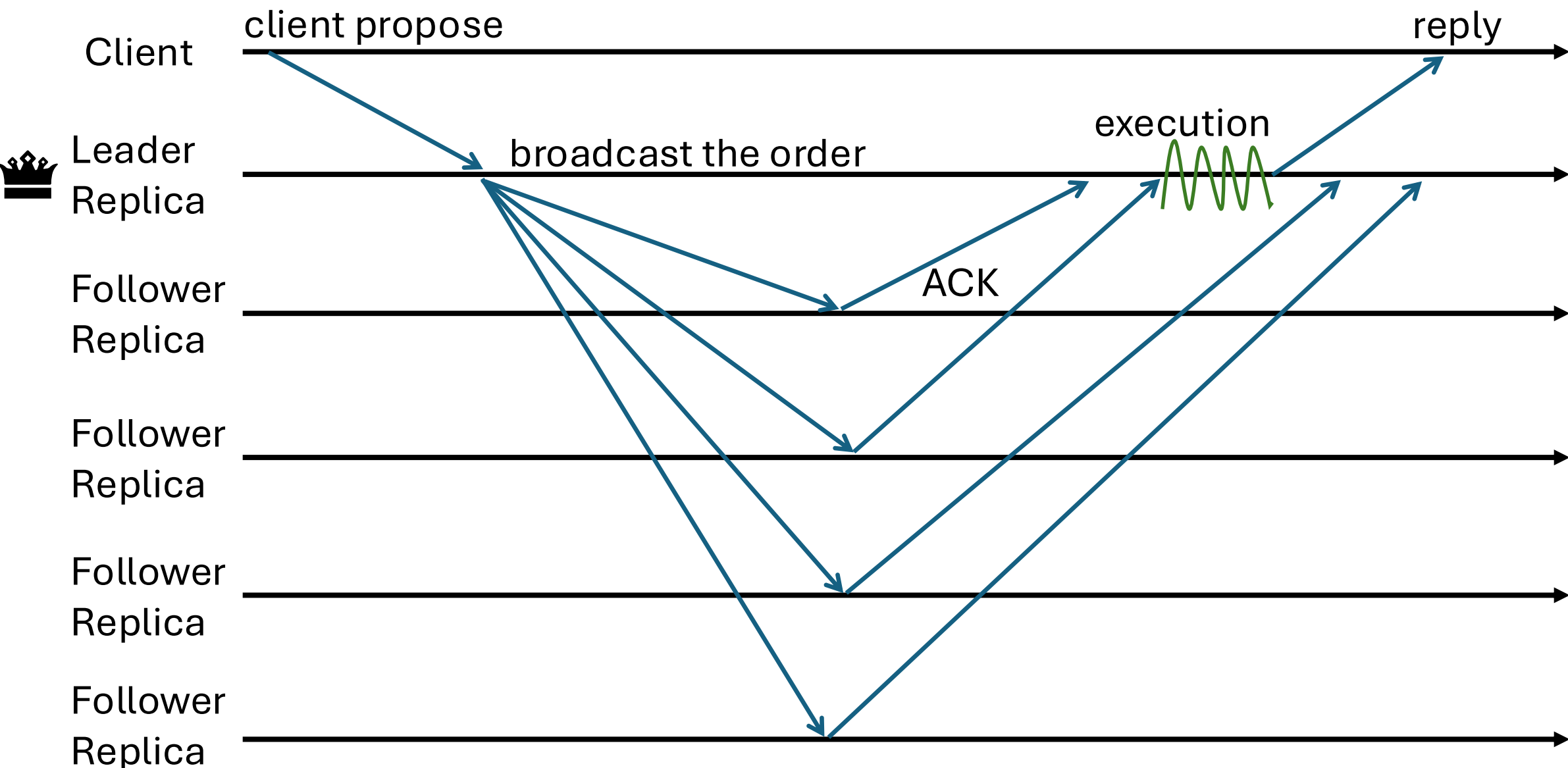
Protocol	Fail-slow tolerance	Normal-Case Latency
Multi-Paxos (a single leader)	No	Low (1 RTT to commit)
Copilot (OSDI 2020) (two leaders)	Yes	High (1 - 2 RTTs to commit)

Fail-Slow Fault Tolerance vs Normal-Case Latency

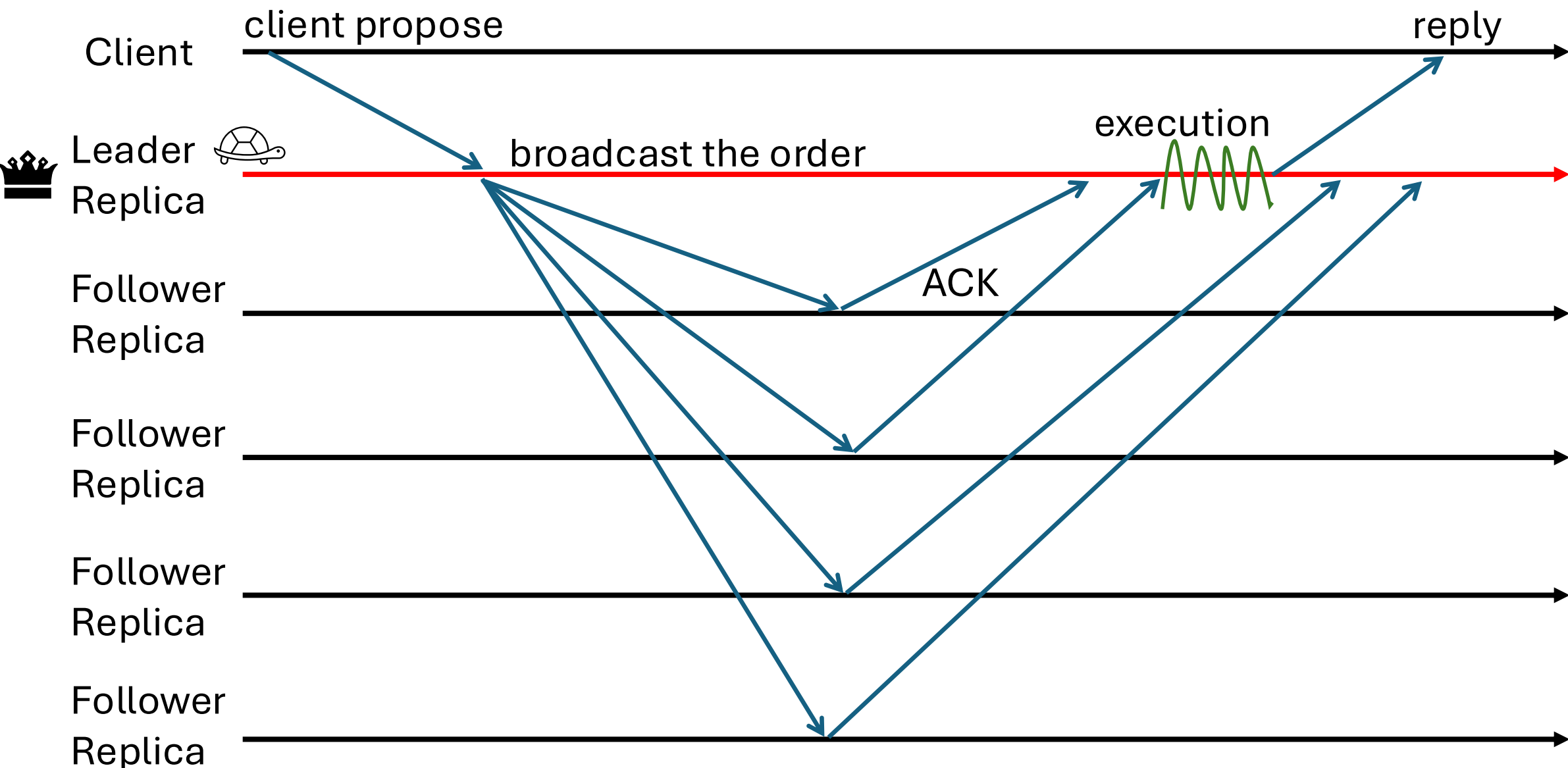
Protocol	Fail-slow tolerance	Normal-Case Latency
Multi-Paxos (a single leader)	No	Low (1 RTT to commit)
Copilot (OSDI 2020) (two leaders)	Yes	High (1 - 2 RTTs to commit)
Avicenna (a real leader, a shadow leader)	Yes	Low (1 RTT to commit)

Avicenna Intuition

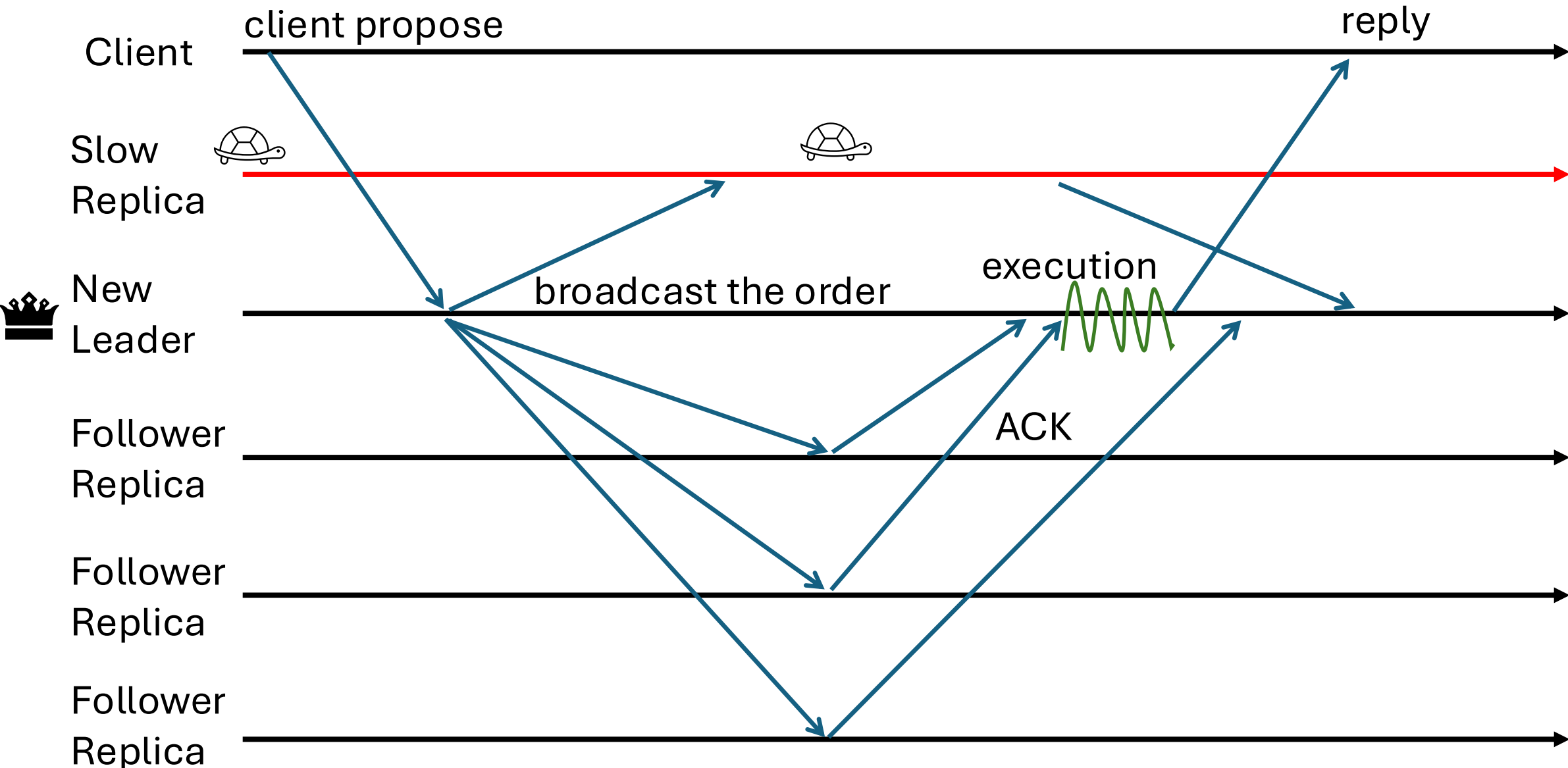
Avicenna Intuition



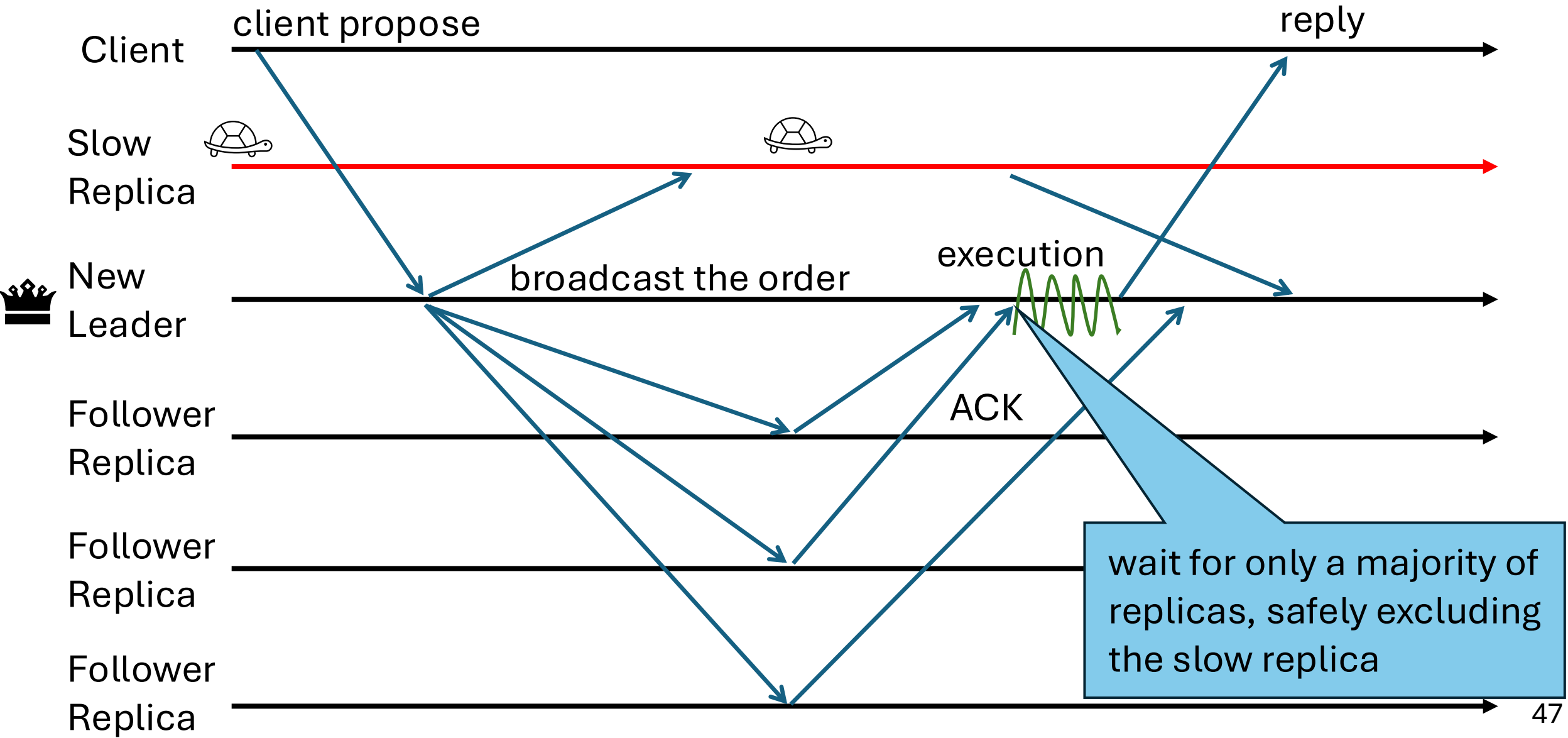
Avicenna Intuition



Avicenna Intuition: Rotate Leadership When Slow



Avicenna Intuition: Rotate Leadership When Slow



Avicenna Intuition

Normal-case: a single leader orders commands

Avicenna Intuition

Normal-case: a single leader orders commands



When the leader is suspected to be slow: rotate to another replica

Avicenna Intuition

Normal-case: a single leader orders commands



When the leader is suspected to be slow: rotate to another replica



Resume normal operation with the new leader

Avicenna Intuition

Normal-case: a single leader orders commands



When the leader is suspected to be slow: rotate to another replica



Resume normal operation with the new leader

Avicenna: Normal-case operation

Fail-slow detection

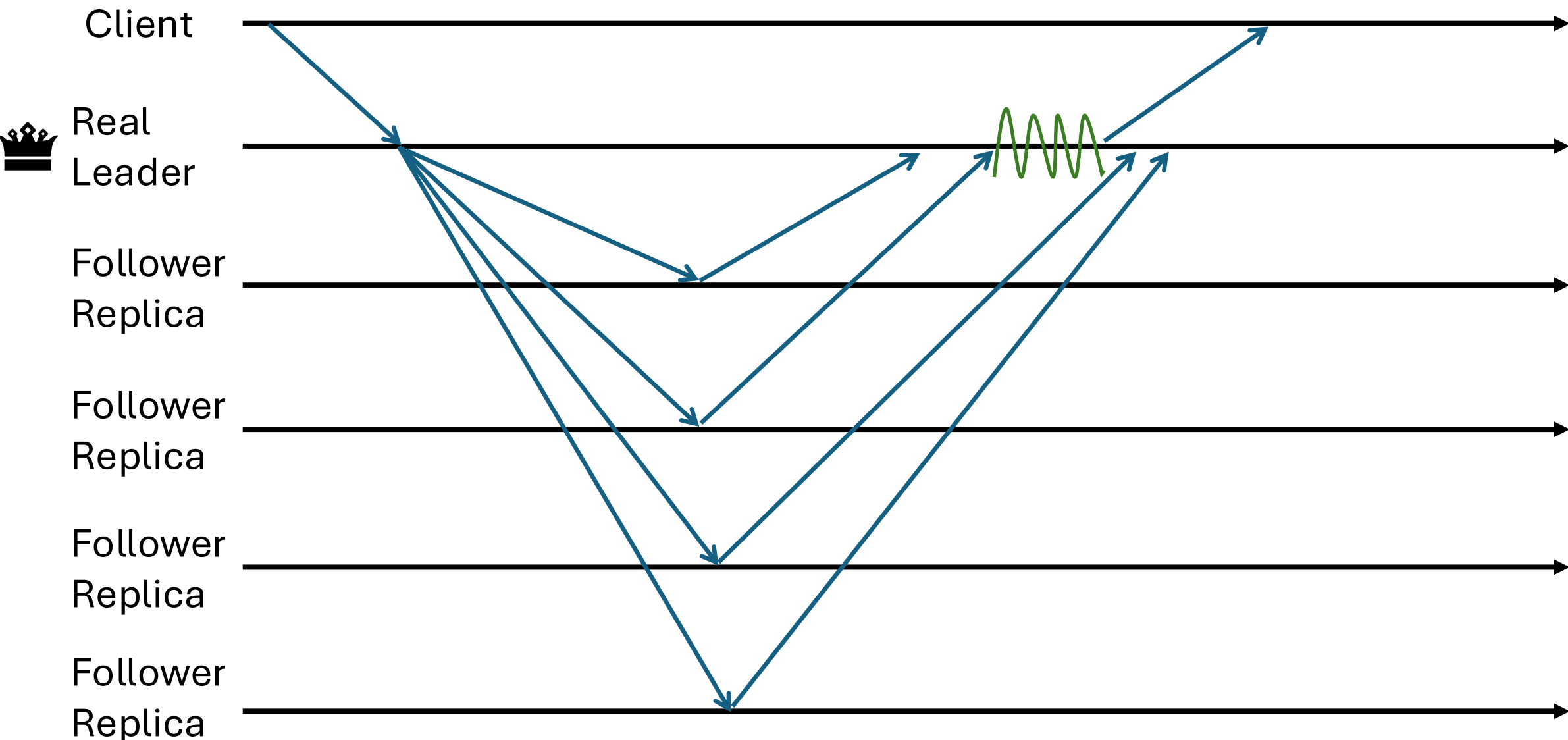
Fail-slow reaction

Outlines

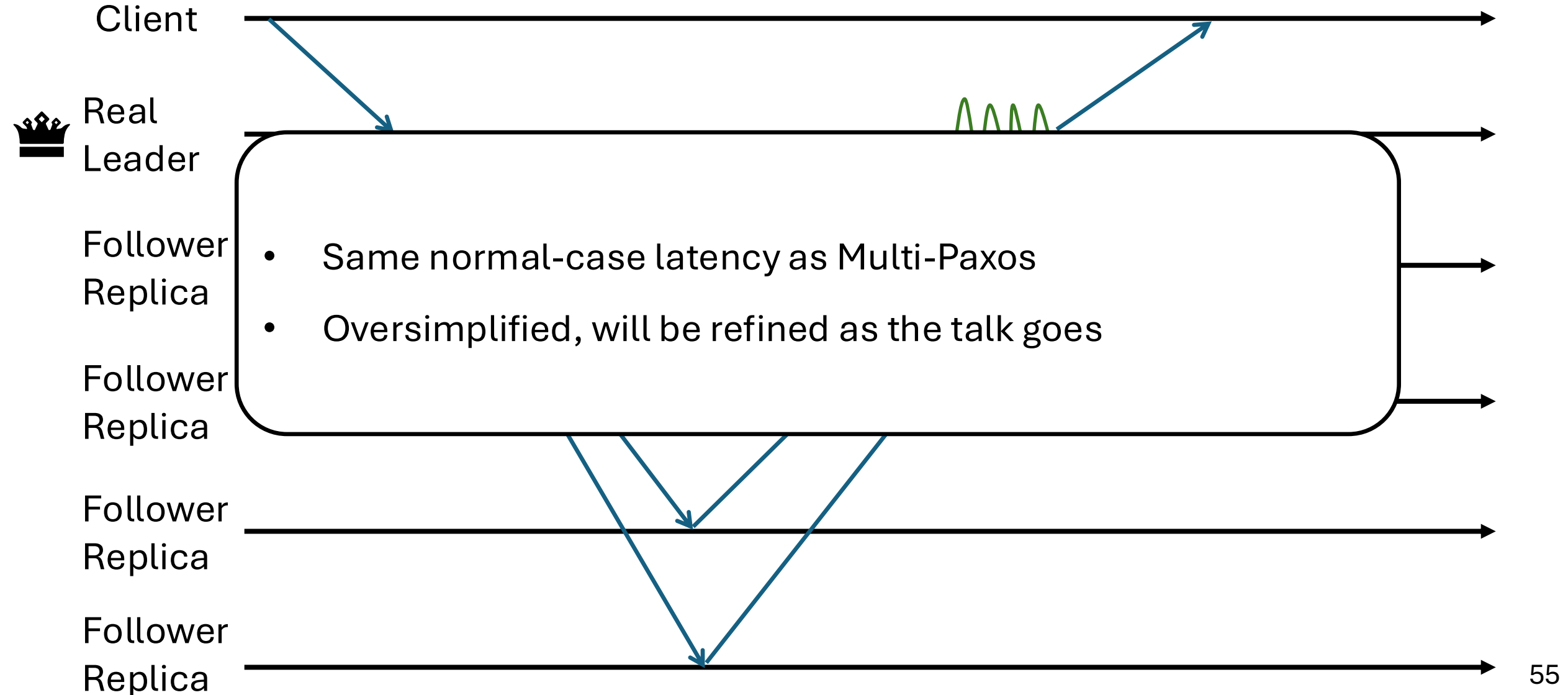
- Replicated State Machines
- Fail-slow Faults in Distributed Systems
- Overview of Avicenna
- Detailed design of Avicenna
 - Normal-Case Operation
 - Fail-Slow Detection
 - Fail-Slow Reaction
- Evaluations
- Discussion and Conclusion

Avicenna Normal-Case Operation: Simplified

Avicenna Normal-Case Operation: Simplified



Avicenna Normal-Case Operation: Simplified



Outlines

- Replicated State Machines
- Fail-slow Faults in Distributed Systems
- Overview of Avicenna
- Detailed design of Avicenna
 - Normal-Case Operation
 - Fail-Slow Detection**
 - Fail-Slow Reaction
- Evaluations
- Discussion and Conclusion

Fail-Slow Detection

When the real leader is slow, identify it!

Fail-Slow Detection Challenges

Fail-Slow Detection Challenges

Static threshold (e.g., timeout for heartbeat messages) in crash detection?

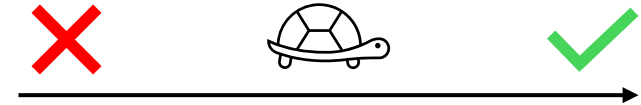
Not working in fail-slow scenarios

Fail-Slow Detection Challenges

Static threshold (e.g., timeout for heartbeat messages) in crash detection?

Not working in fail-slow scenarios

1. Fail-slow faults are non-binary



Fail-Slow Detection Challenges

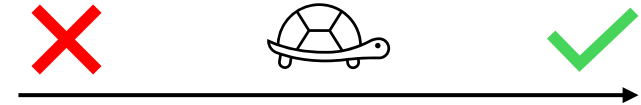
Static threshold (e.g., timeout for heartbeat messages) in crash detection?

Not working in fail-slow scenarios

1. Fail-slow faults are non-binary

Conservative threshold -> detect only the most severe faults

Aggressive threshold -> trigger frequent rotations



Fail-Slow Detection Challenges

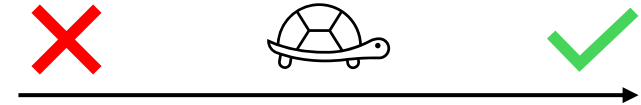
Static threshold (e.g., timeout for heartbeat messages) in crash detection?

Not working in fail-slow scenarios

1. Fail-slow faults are non-binary

Conservative threshold -> detect only the most sever faults

Aggressive threshold -> trigger frequent rotations



2. Some fail-slow faults behave as gray failure



Fail-Slow Detection Challenges

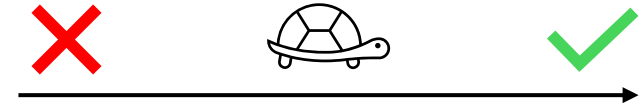
Static threshold (e.g., timeout for heartbeat messages) in crash detection?

Not working in fail-slow scenarios

1. Fail-slow faults are non-binary

Conservative threshold -> detect only the most sever faults

Aggressive threshold -> trigger frequent rotations



2. Some fail-slow faults behave as gray failure

Internal detection mechanism does not work

Example: disk slow: slowdown request processing, but reply to heartbeat messages normally



Fail-Slow Detection Challenges

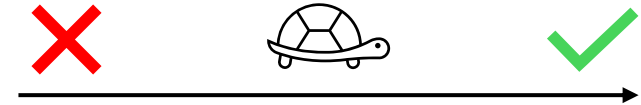
Static threshold (e.g., timeout for heartbeat messages) in crash detection?

Not working in fail-slow scenarios

1. Fail-slow faults are non-binary

Conservative threshold -> detect only the most sever faults

Aggressive threshold -> trigger frequent rotations



2. Some fail-slow faults behave as gray failure

Internal detection mechanism does not work

Example: disk slow: slowdown request processing, but reply to heartbeat messages normally

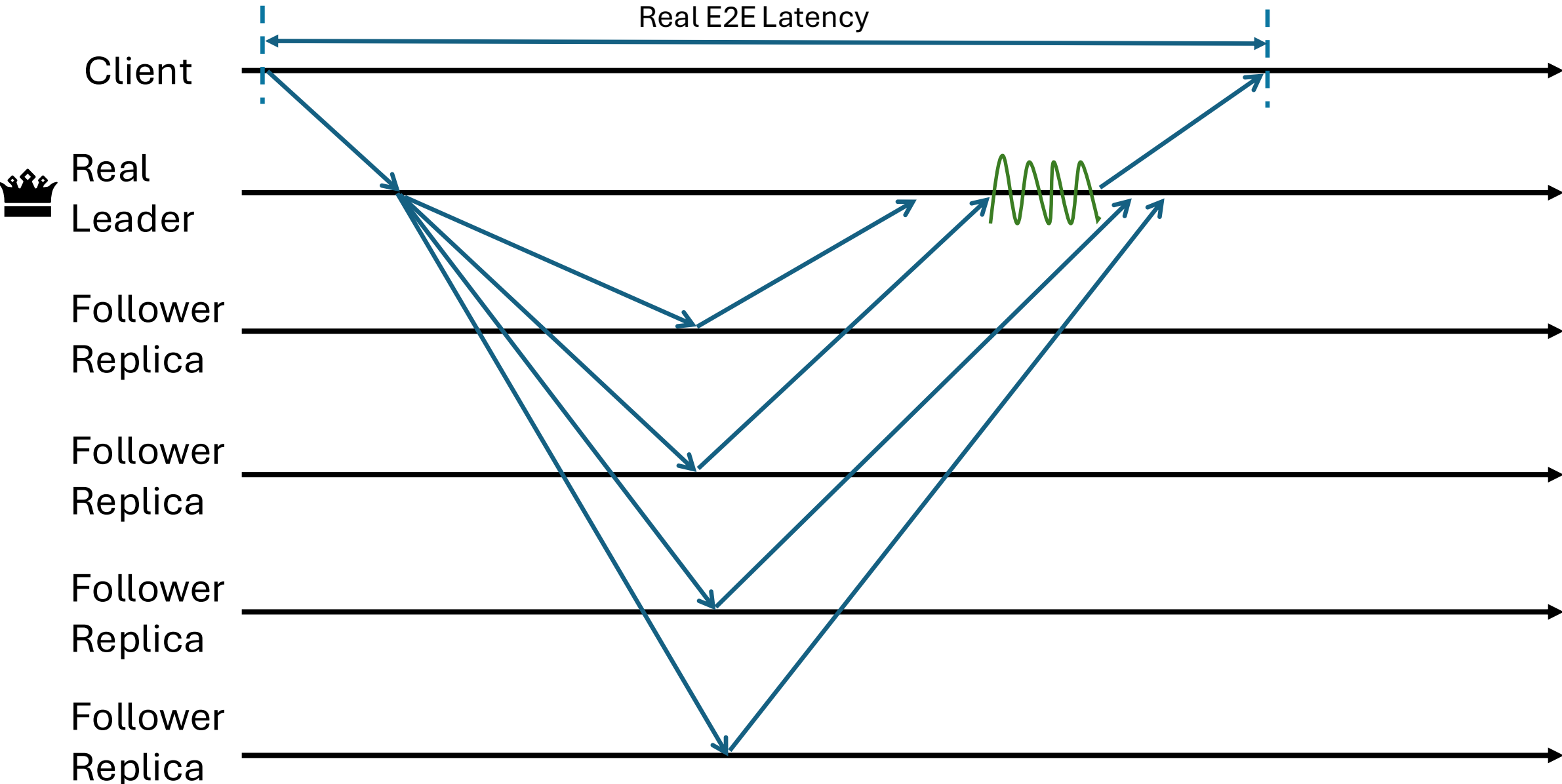
Solution: Shadow processing

Simulate the latency under an alternative leader

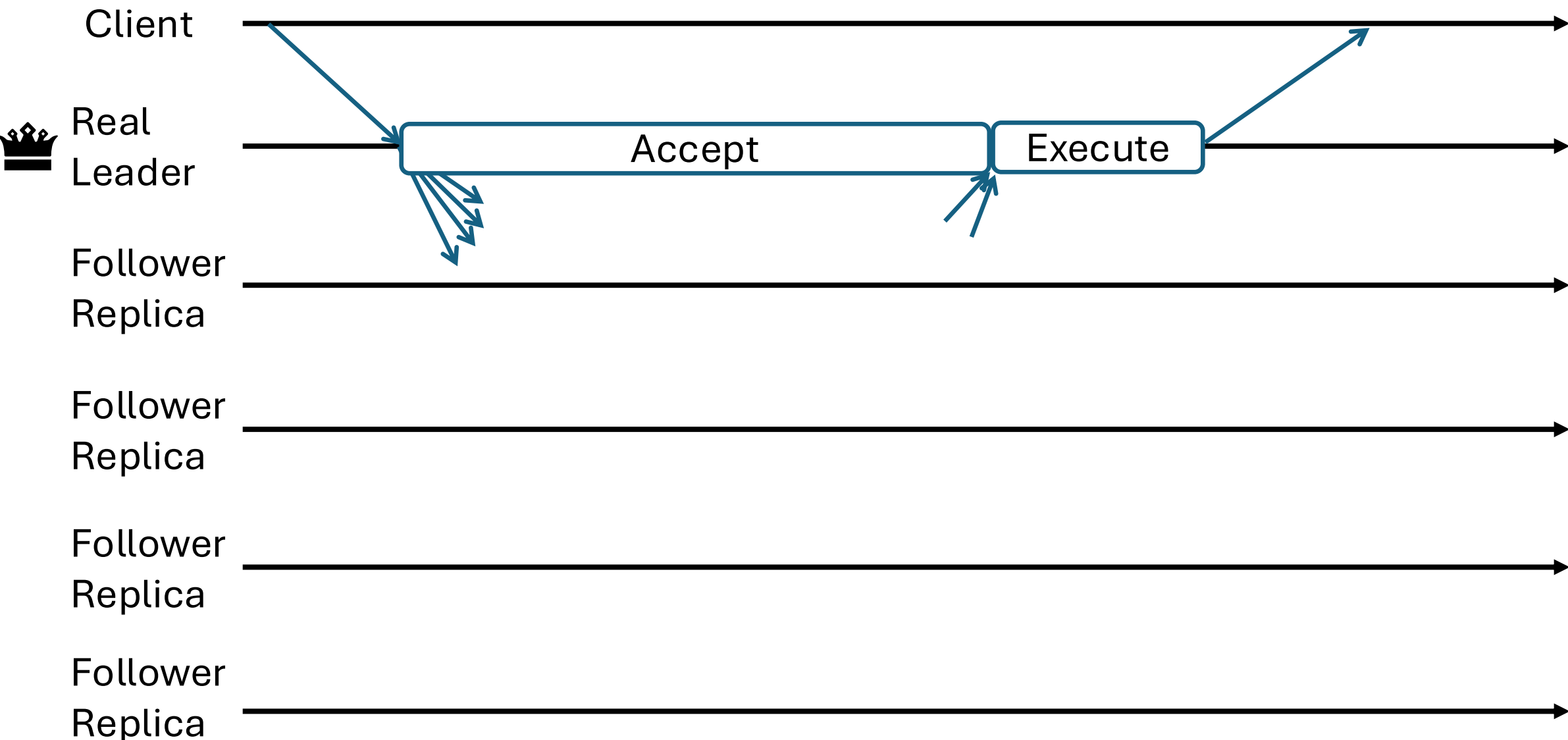


Measure Shadow Latency: Naive

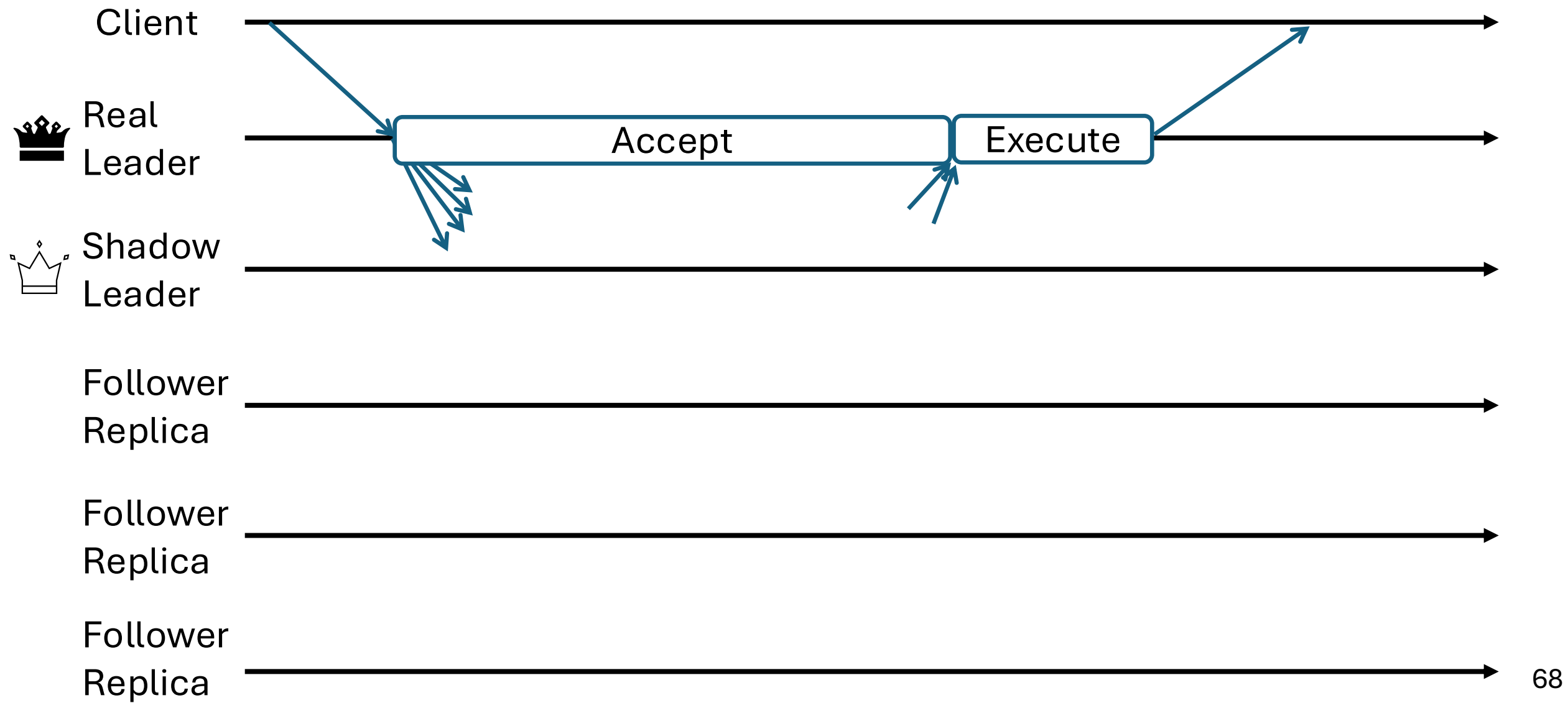
Measure Shadow Latency: Naive



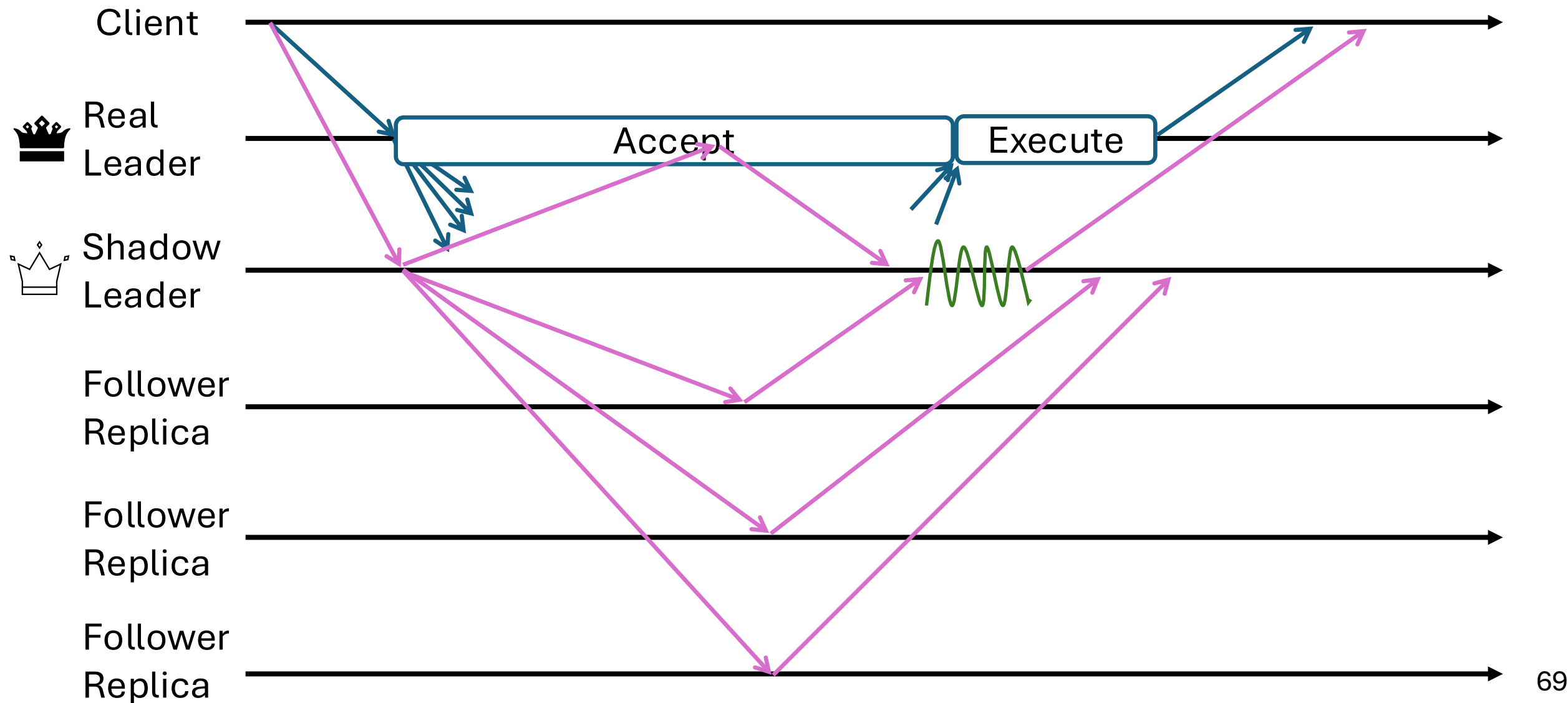
Measure Shadow Latency: Naive



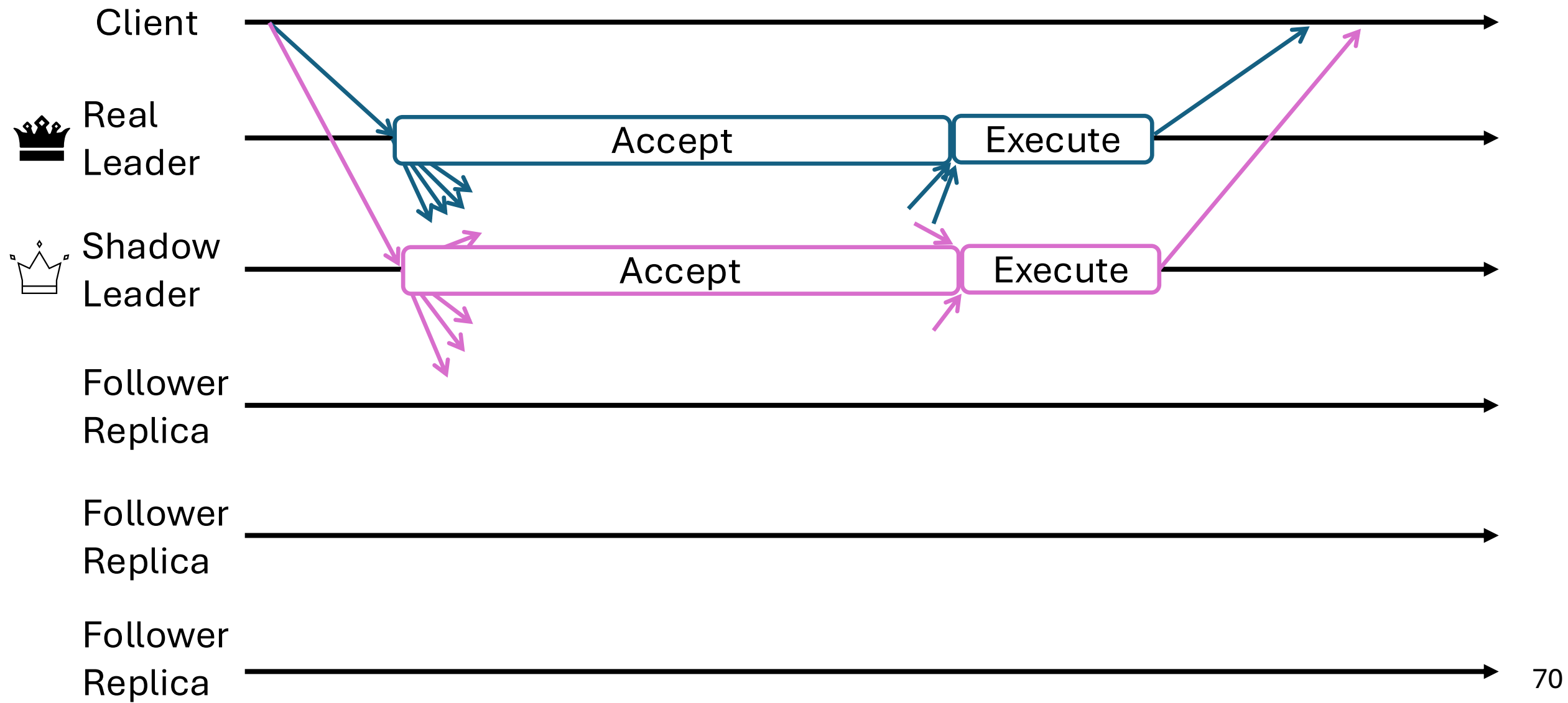
Measure Shadow Latency: Naive



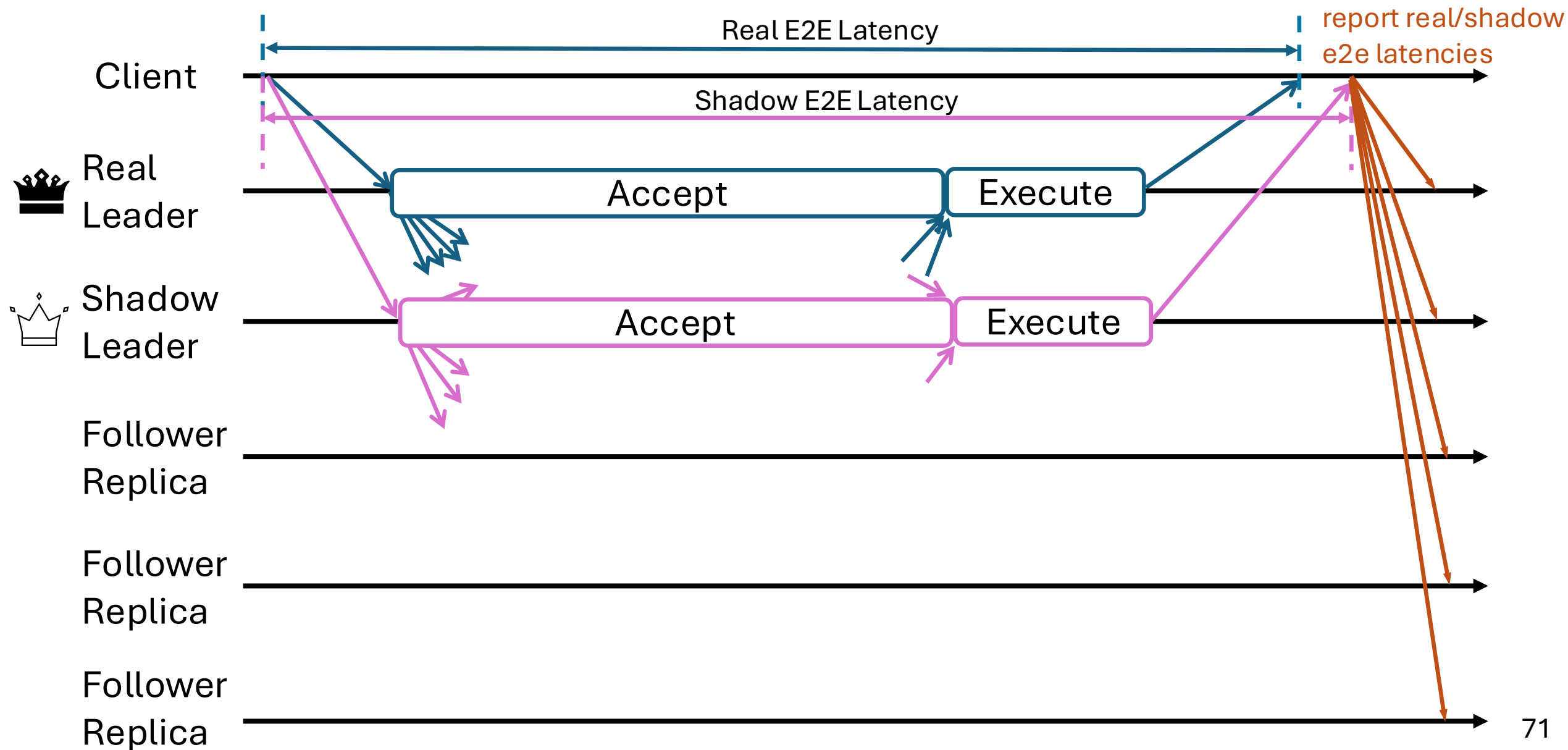
Measure Shadow Latency: Naive



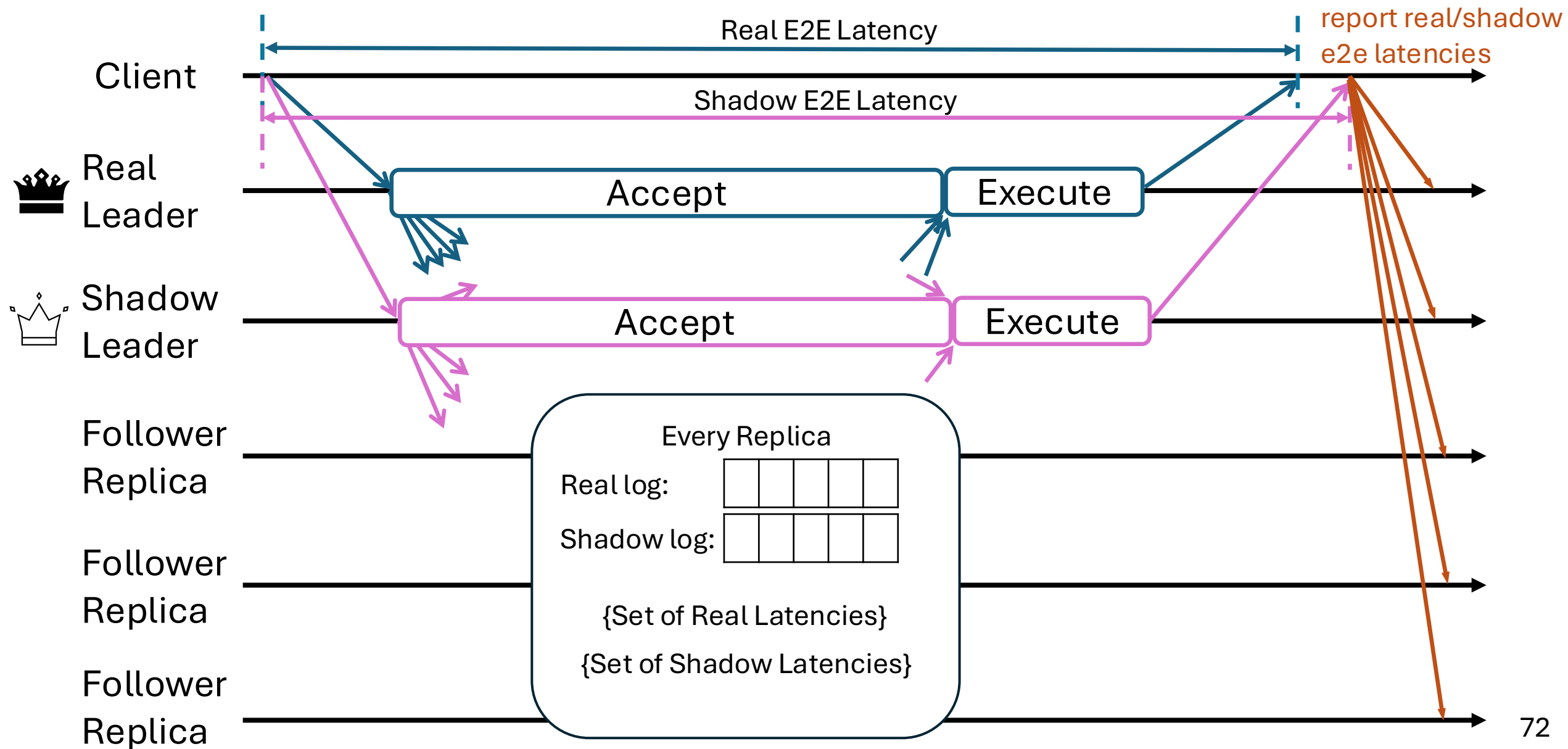
Measure Shadow Latency: Naive



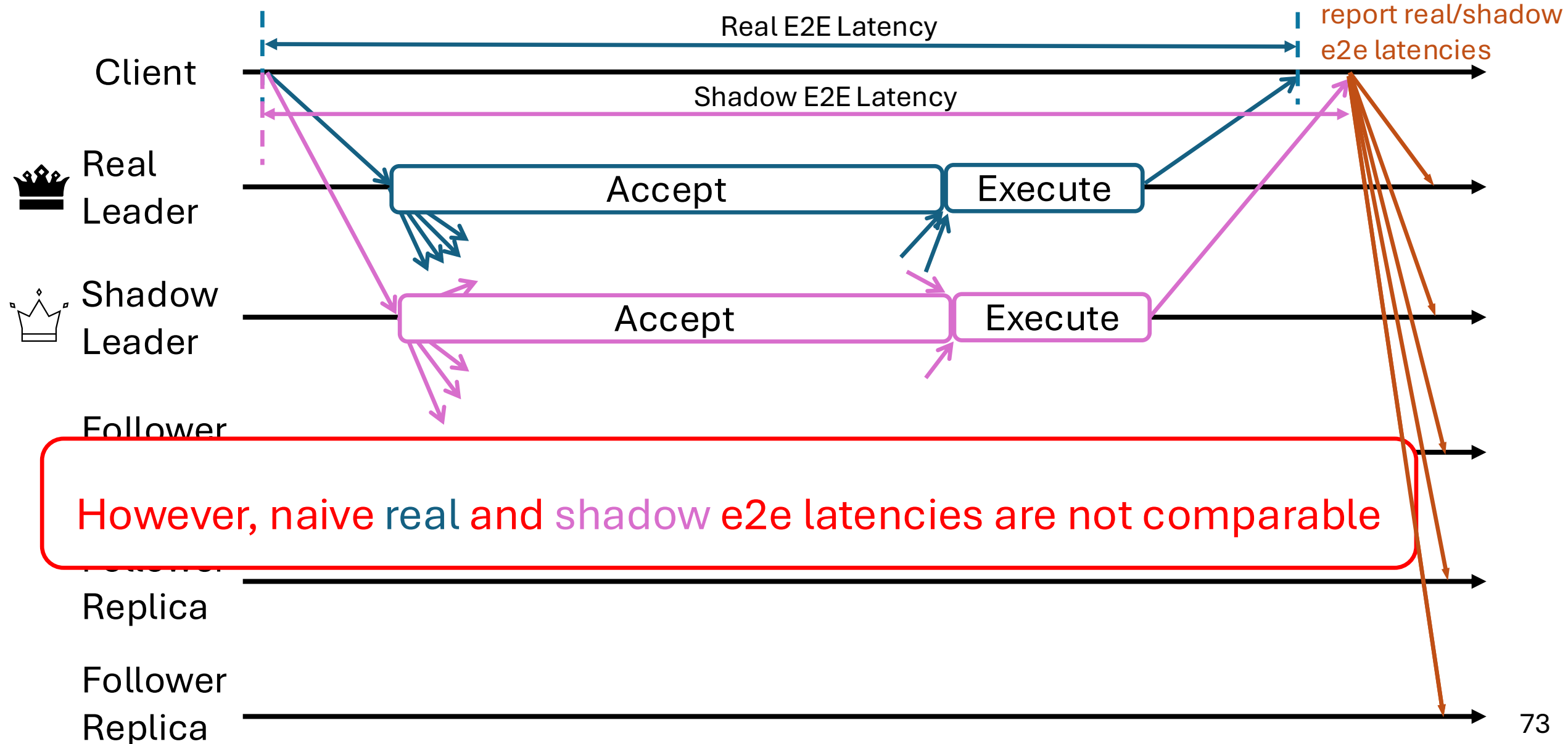
Measure Shadow Latency: Naive



Measure Shadow Latency: Naive

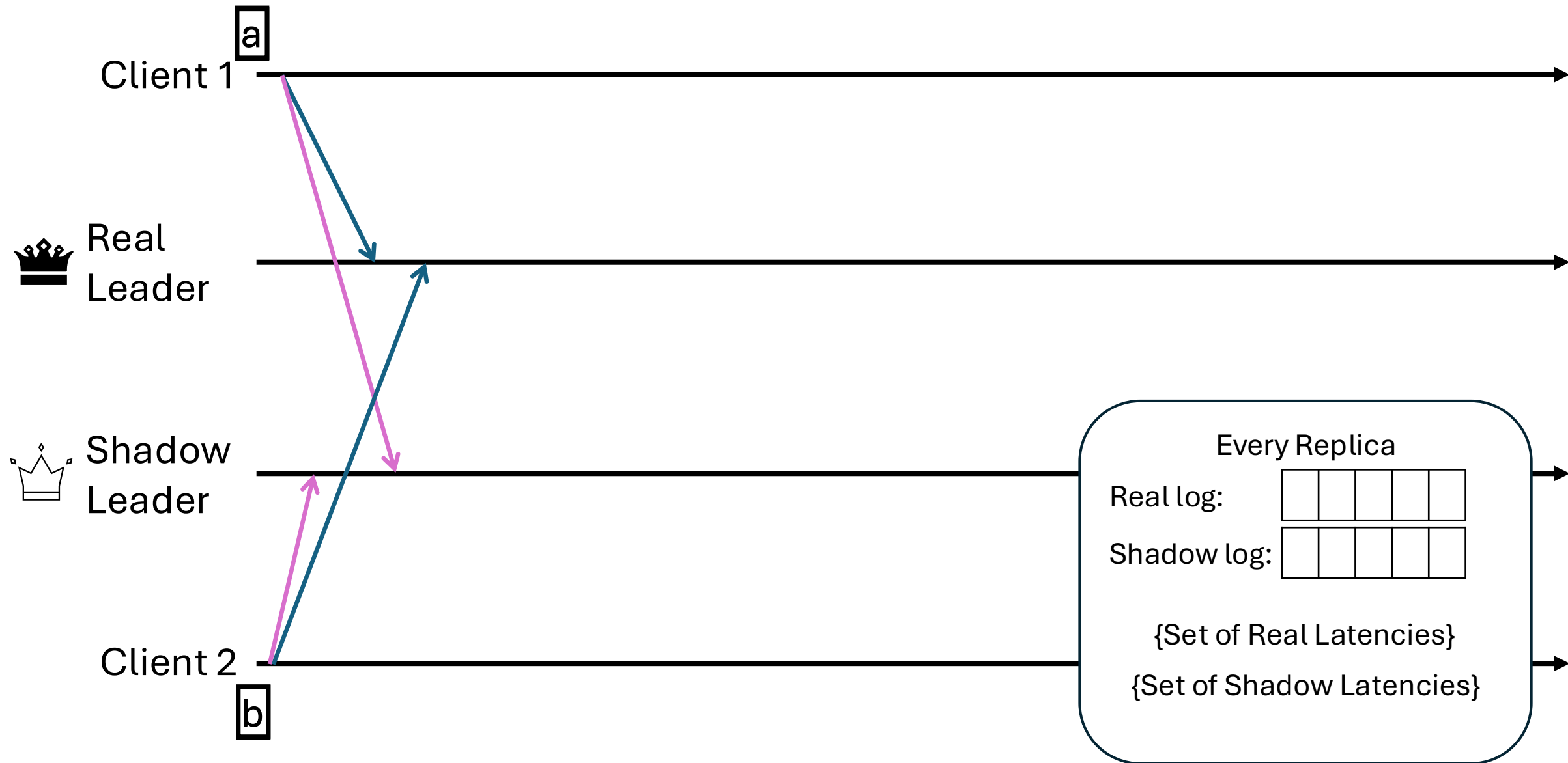


Measure Shadow Latency: Naive

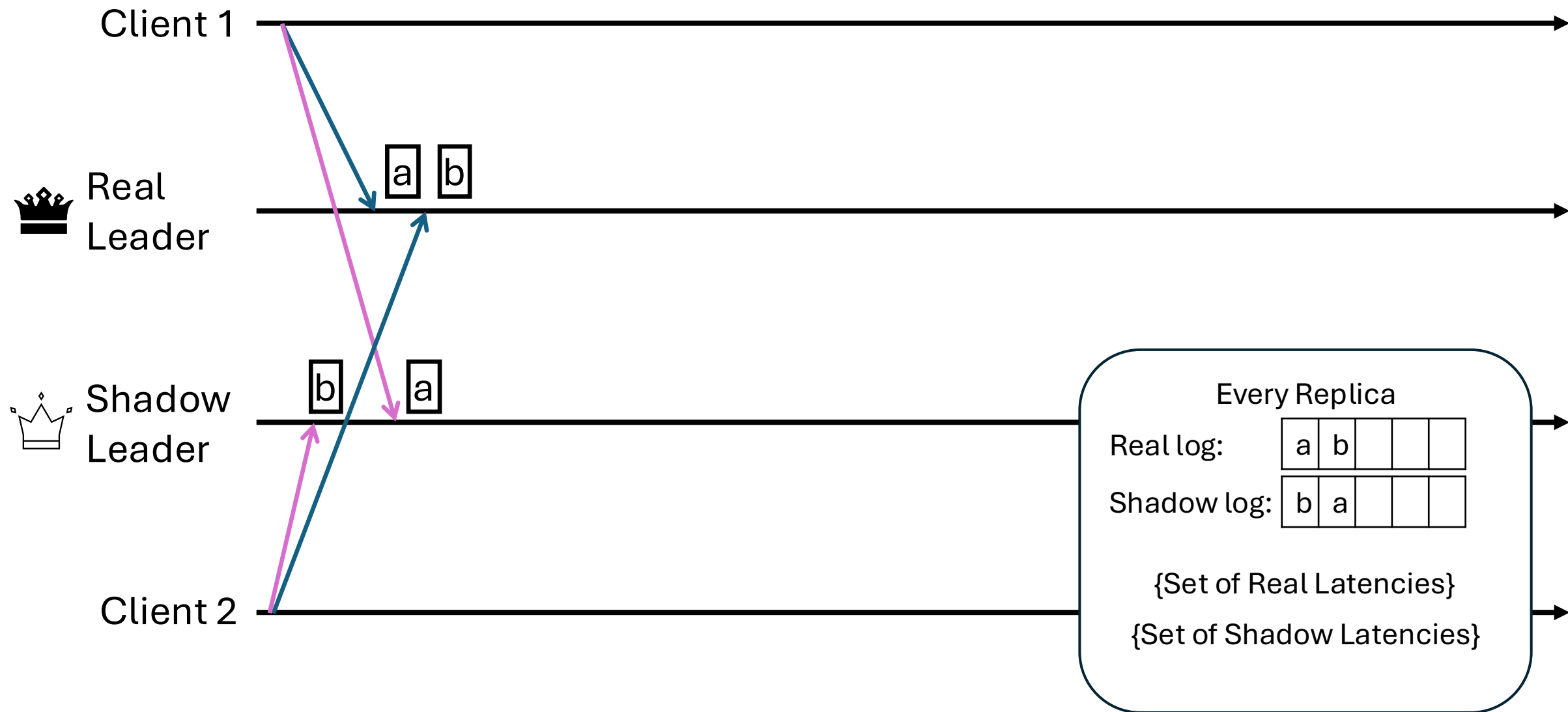


Incomparable Real and Shadow E2E Latency

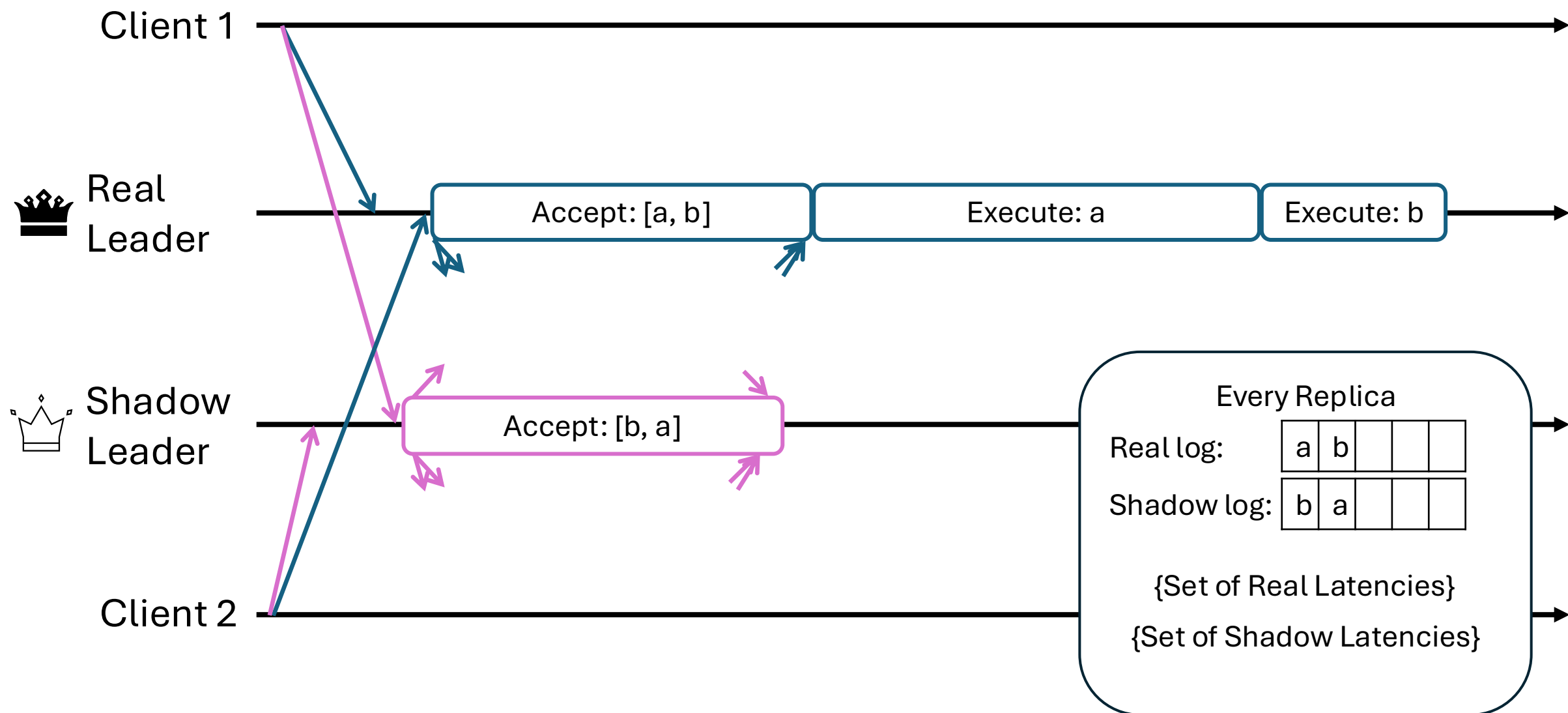
Incomparable Real and Shadow Execution Latency



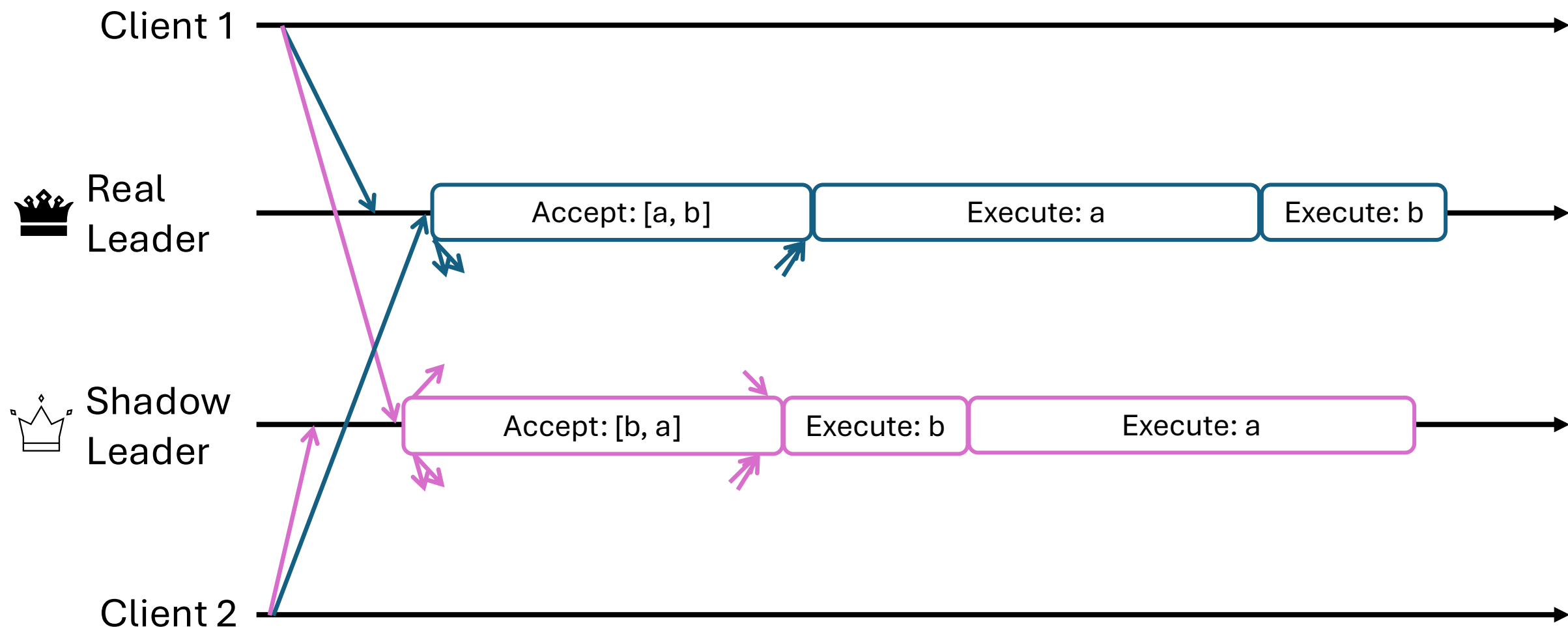
Incomparable Real and Shadow Execution Latency



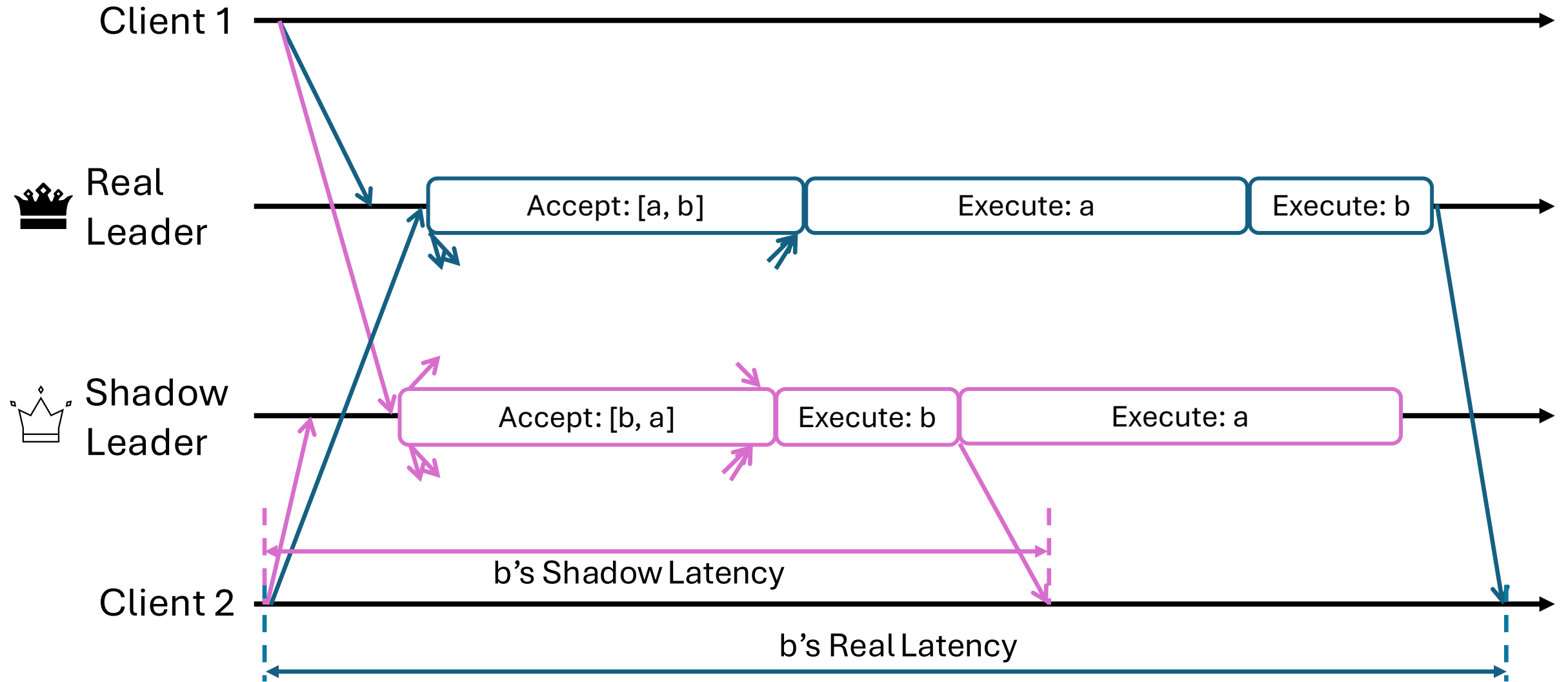
Incomparable Real and Shadow Execution Latency



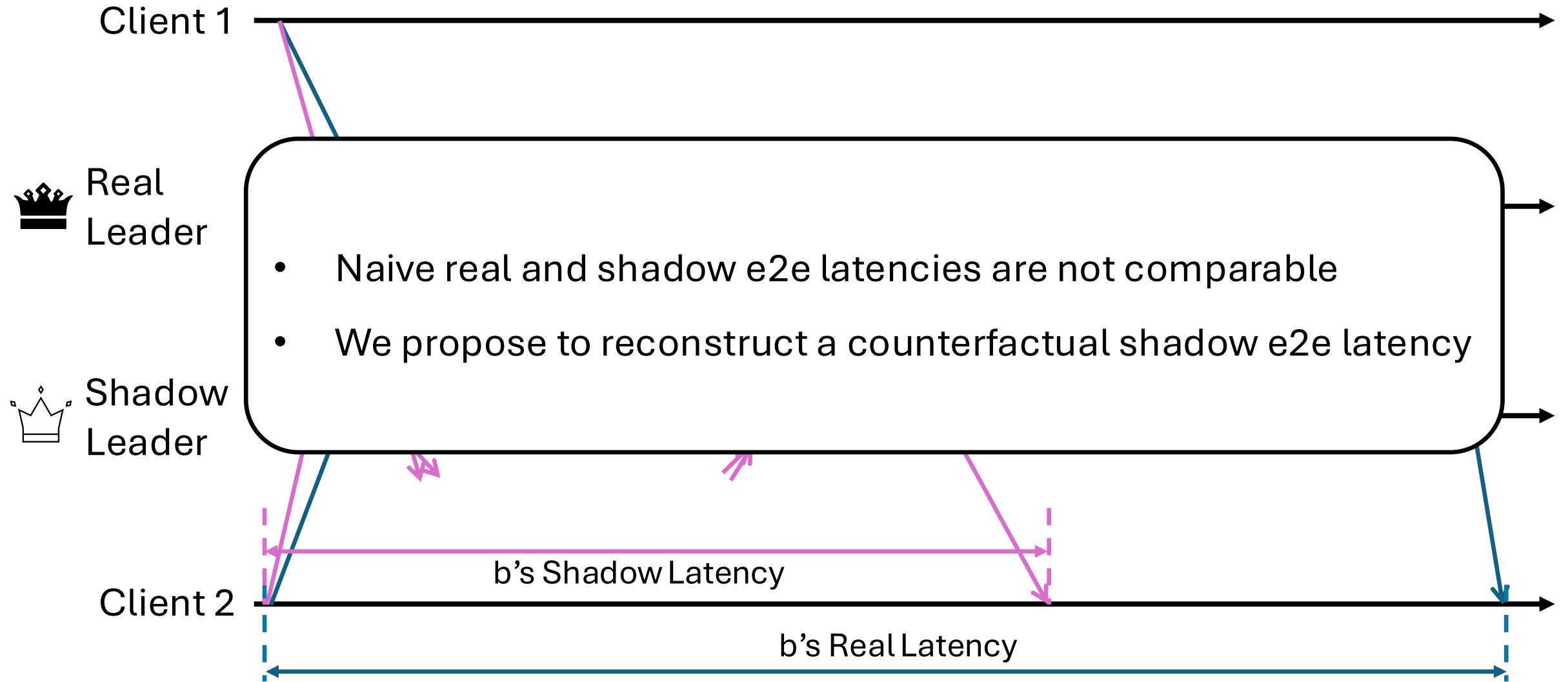
Incomparable Real and Shadow Execution Latency



Incomparable Real and Shadow Execution Latency

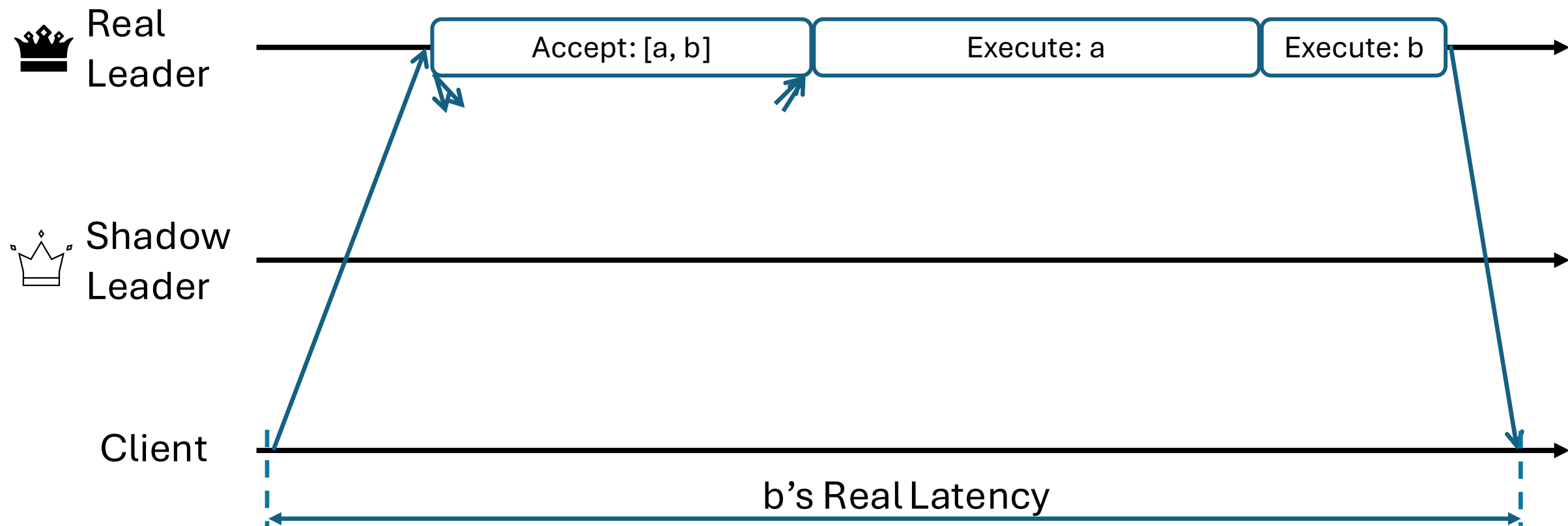


Incomparable Real and Shadow Execution Latency

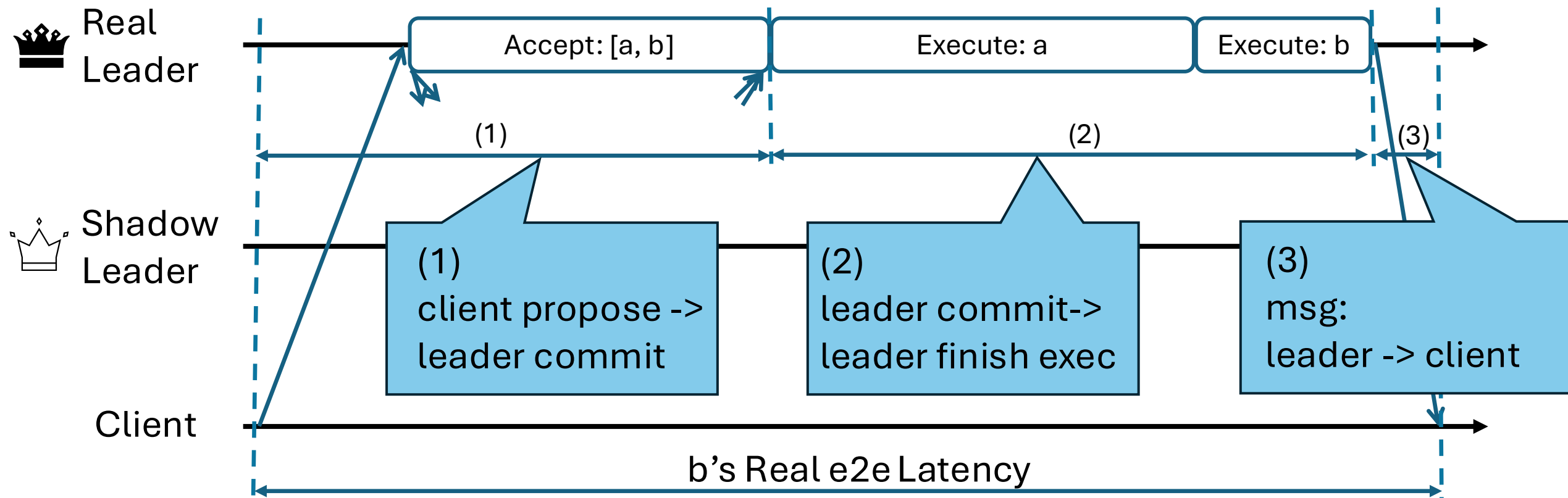


Components of Real E2E Latency

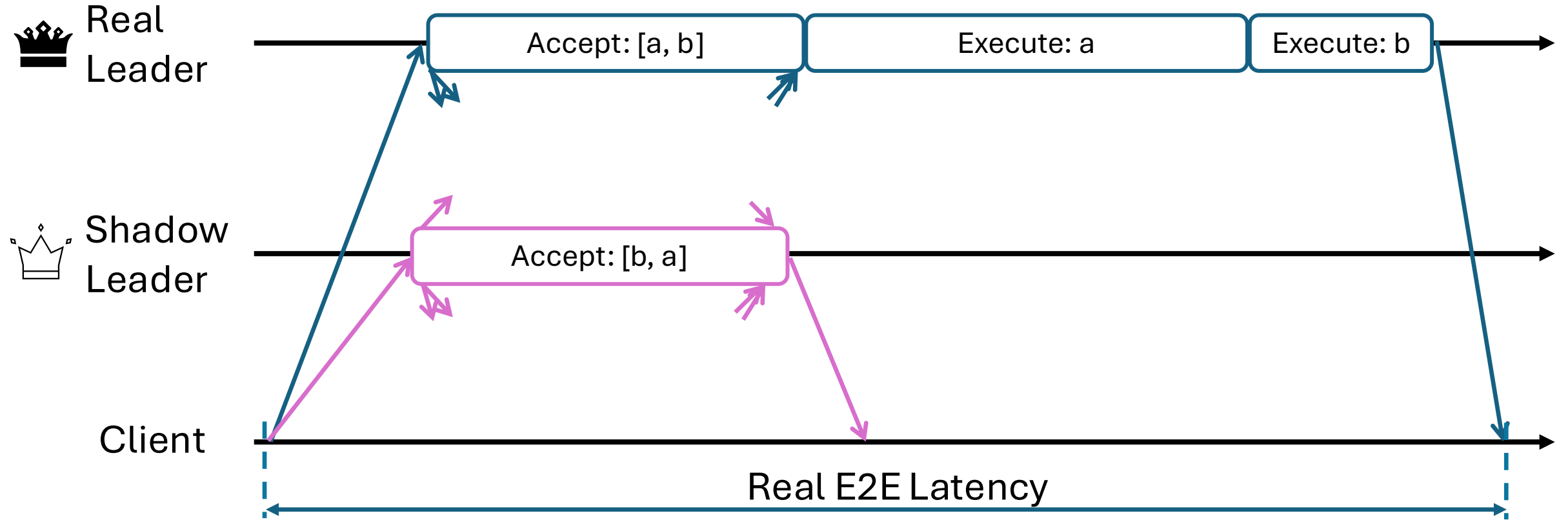
Components of Real E2E Latency



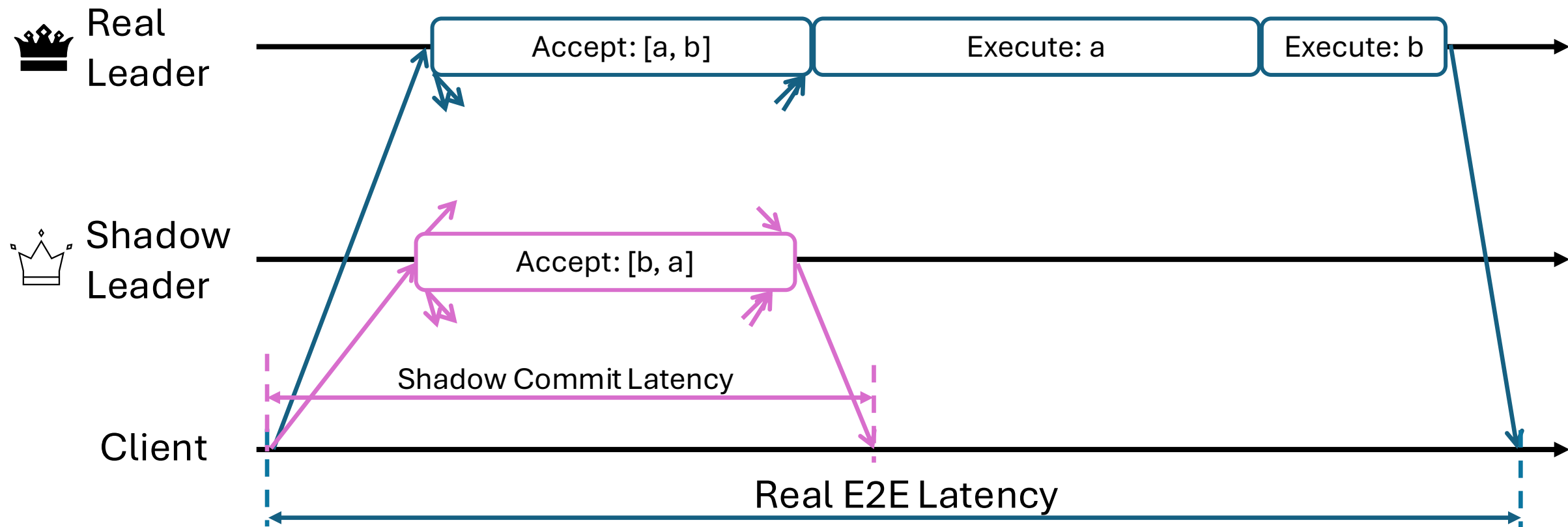
Components of Real E2E Latency



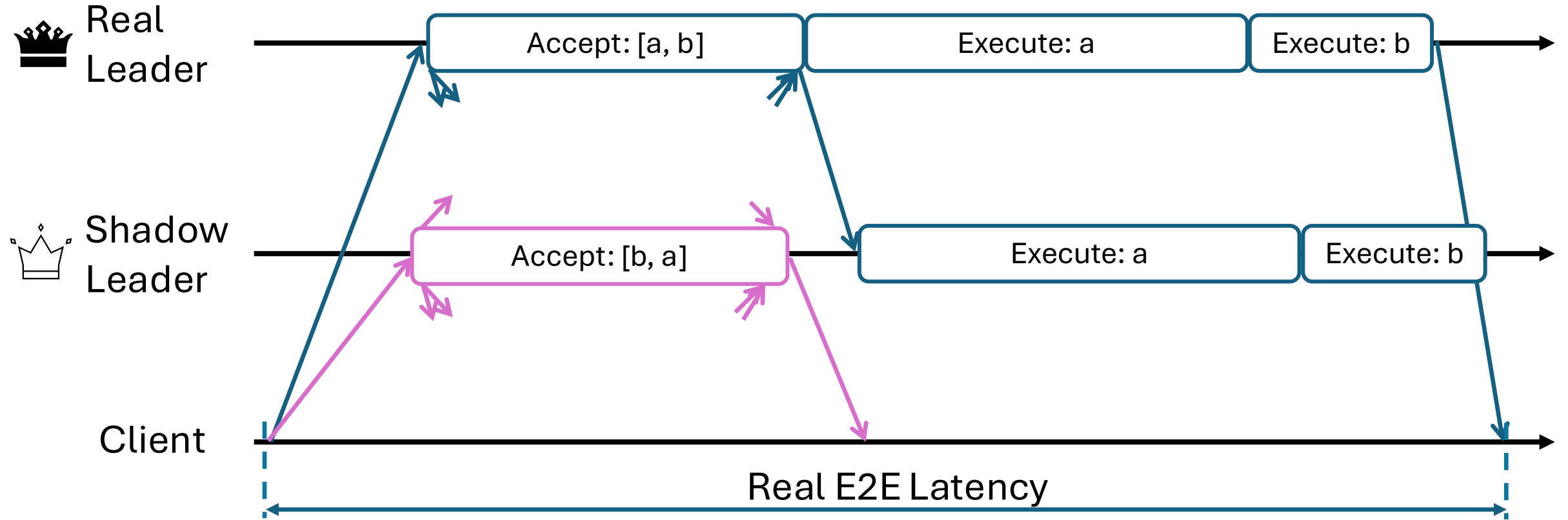
Reconstruct Shadow E2E Latency



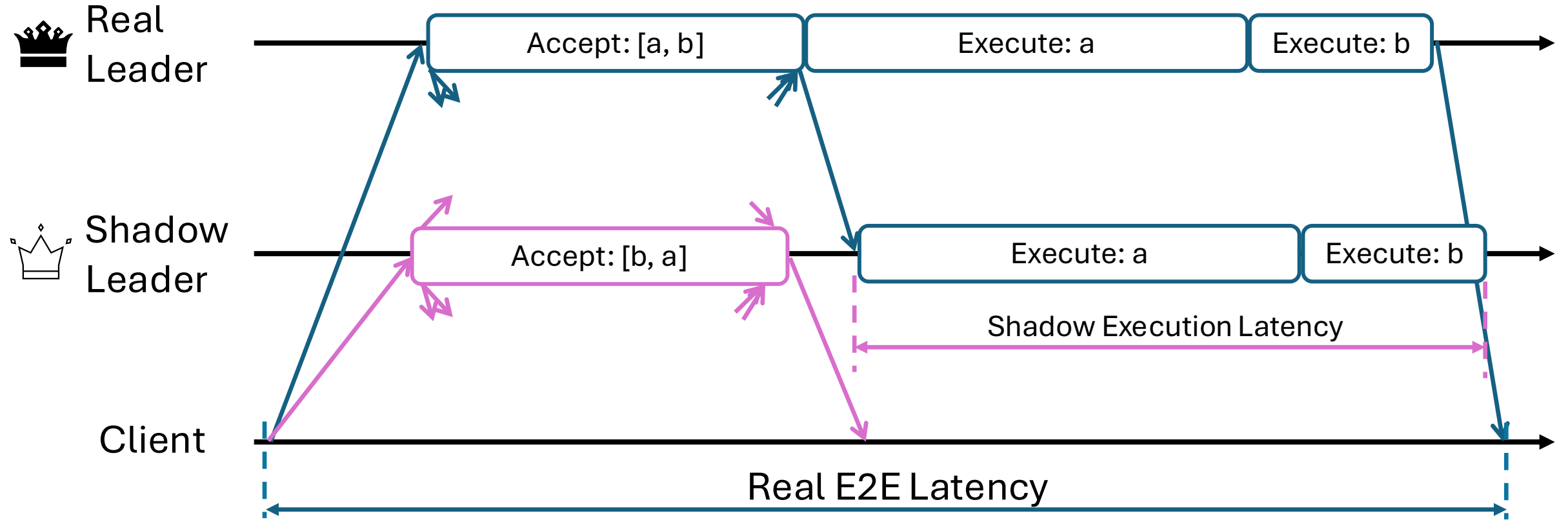
Reconstruct Shadow E2E Latency



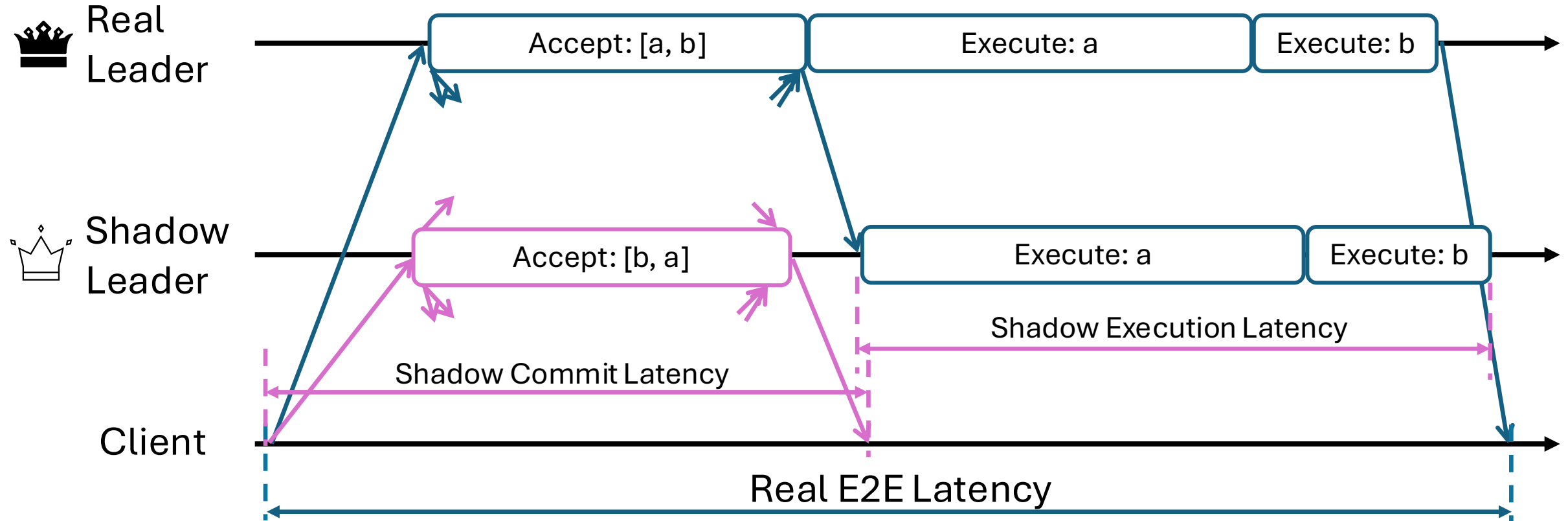
Reconstruct Shadow E2E Latency



Reconstruct Shadow E2E Latency



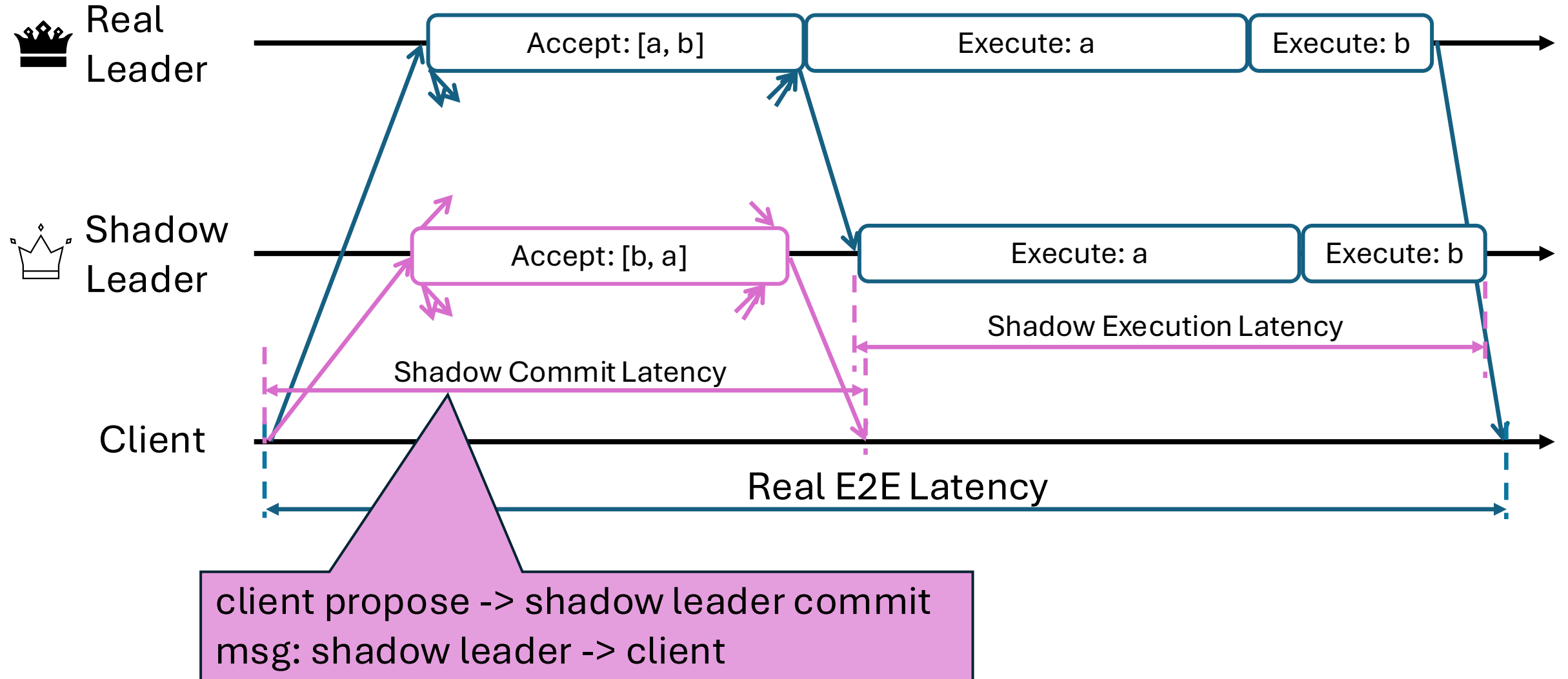
Reconstruct Shadow E2E Latency



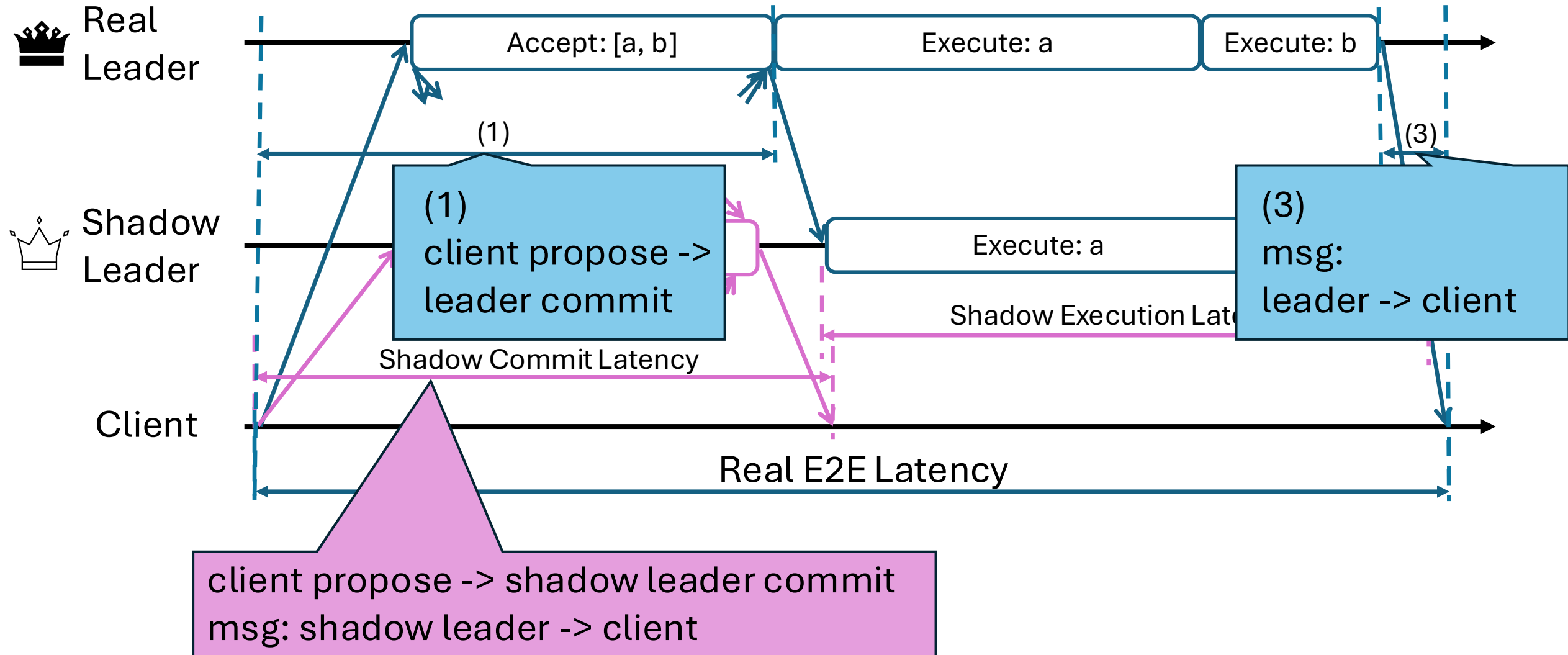
Counterfactual Shadow E2E Latency = Shadow Commit Latency + Shadow Execution Latency

Reconstruction is Faithful

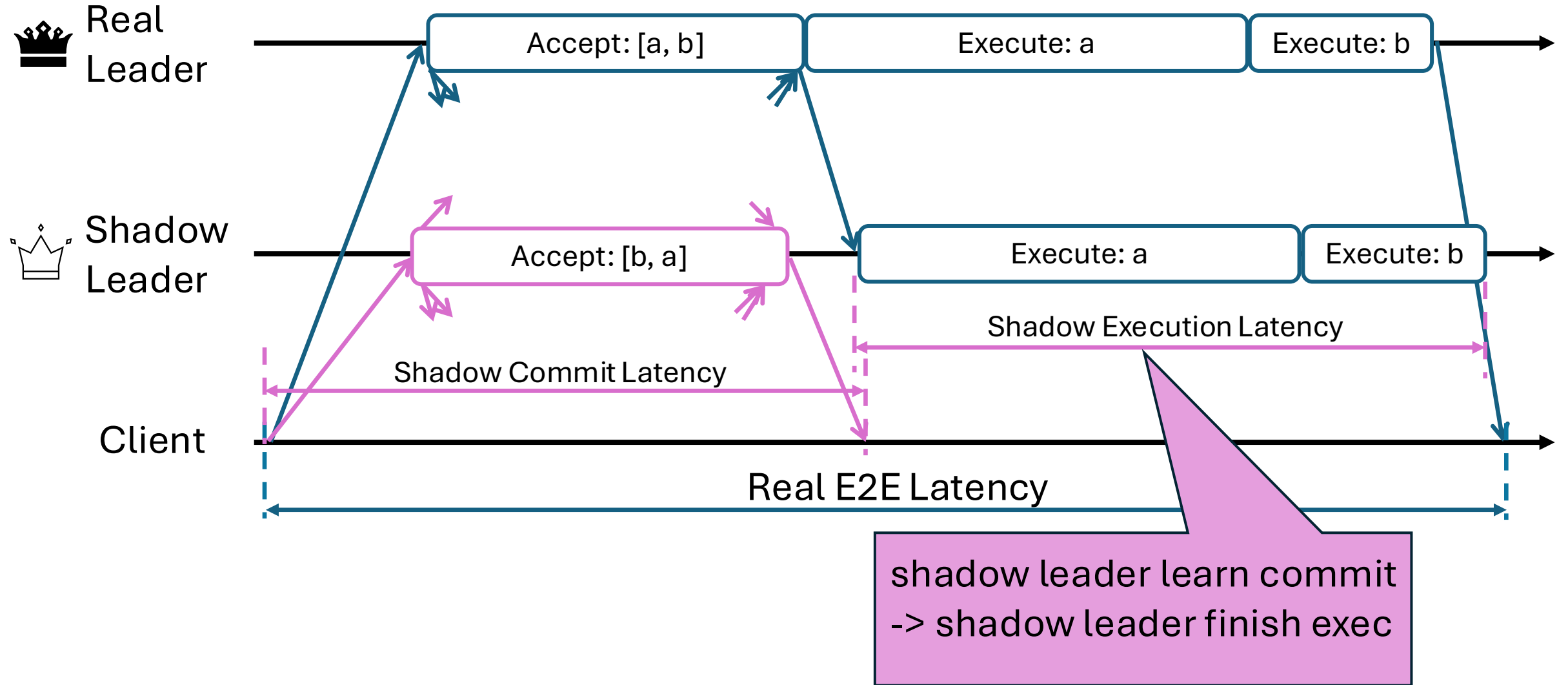
Reconstruction is Faithful



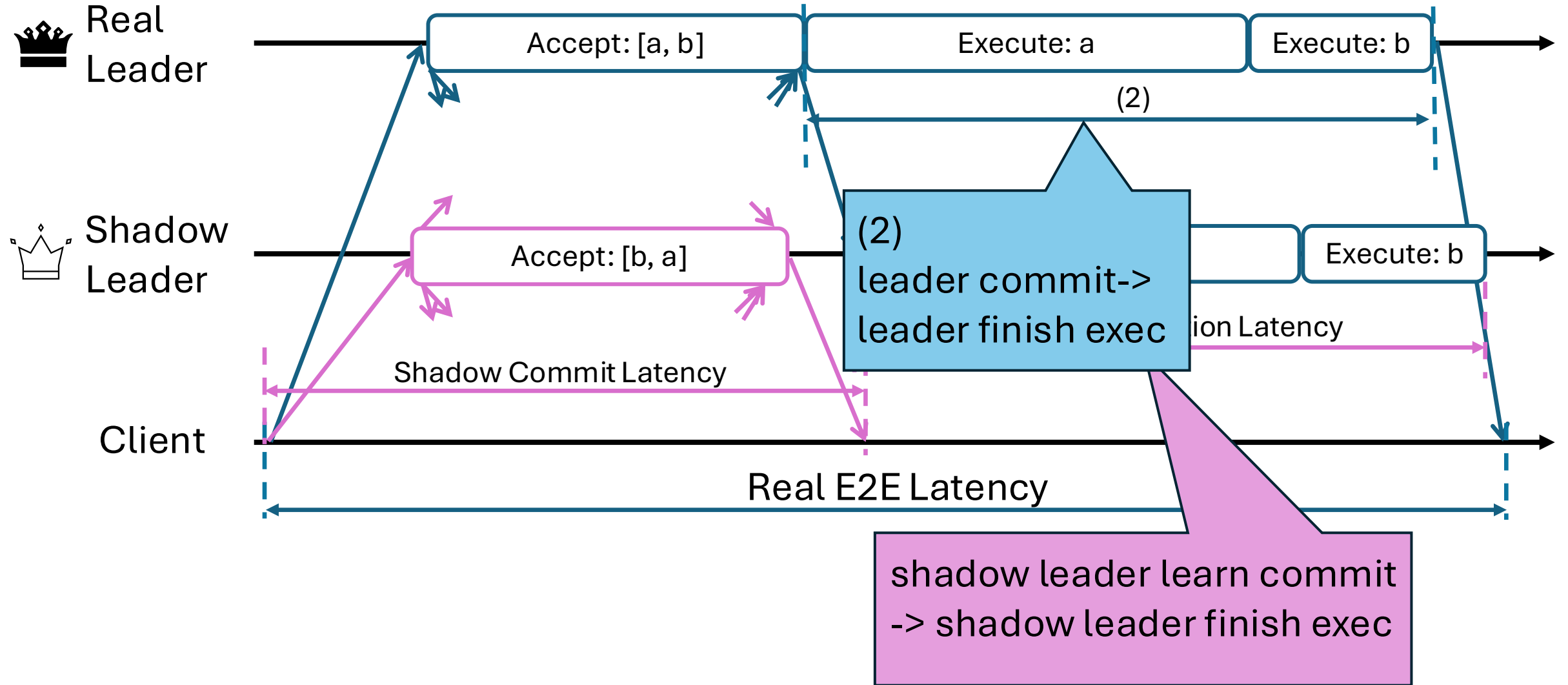
Reconstruction is Faithful



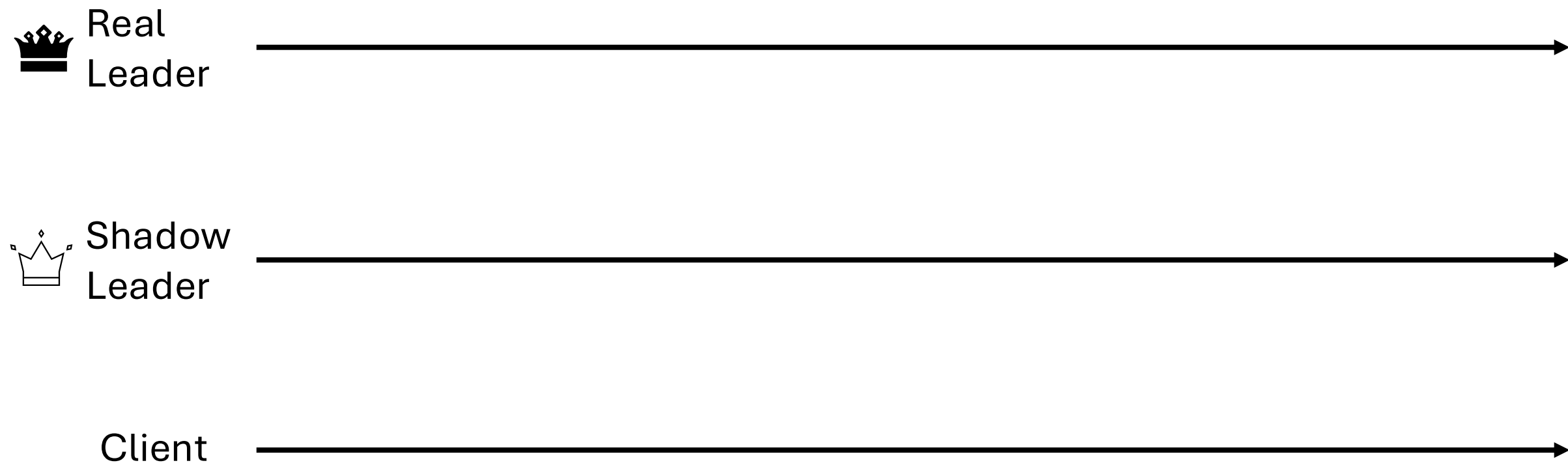
Reconstruction is Faithful



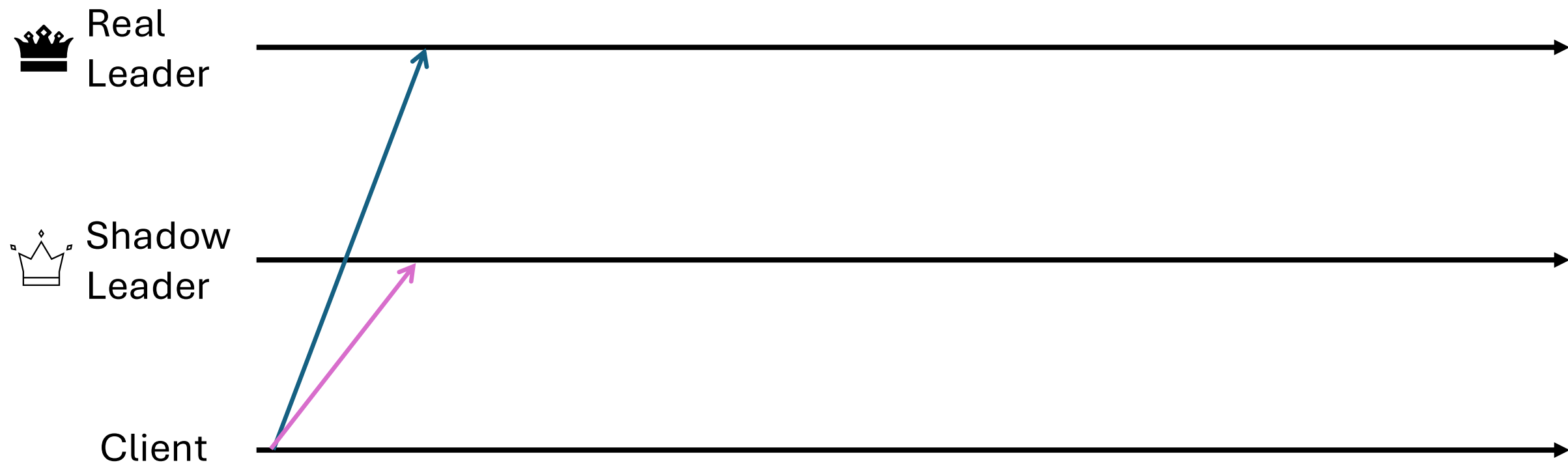
Reconstruction is Faithful



Fail-Slow Detection Workflow

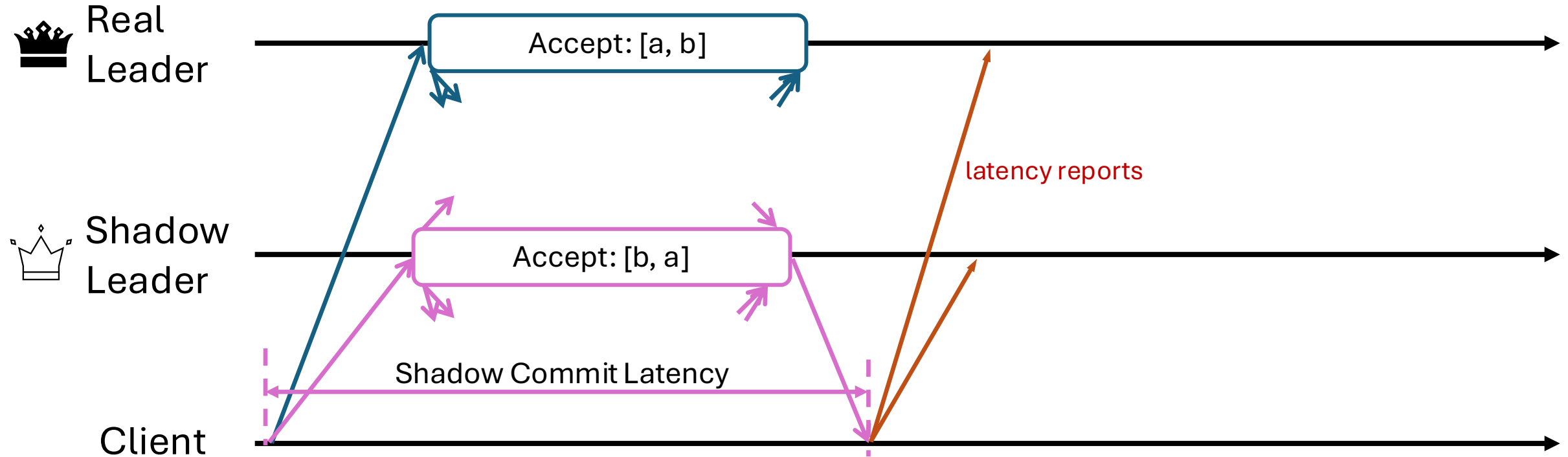


Fail-Slow Detection Workflow



Step 1: Client sends request to both real and shadow leaders

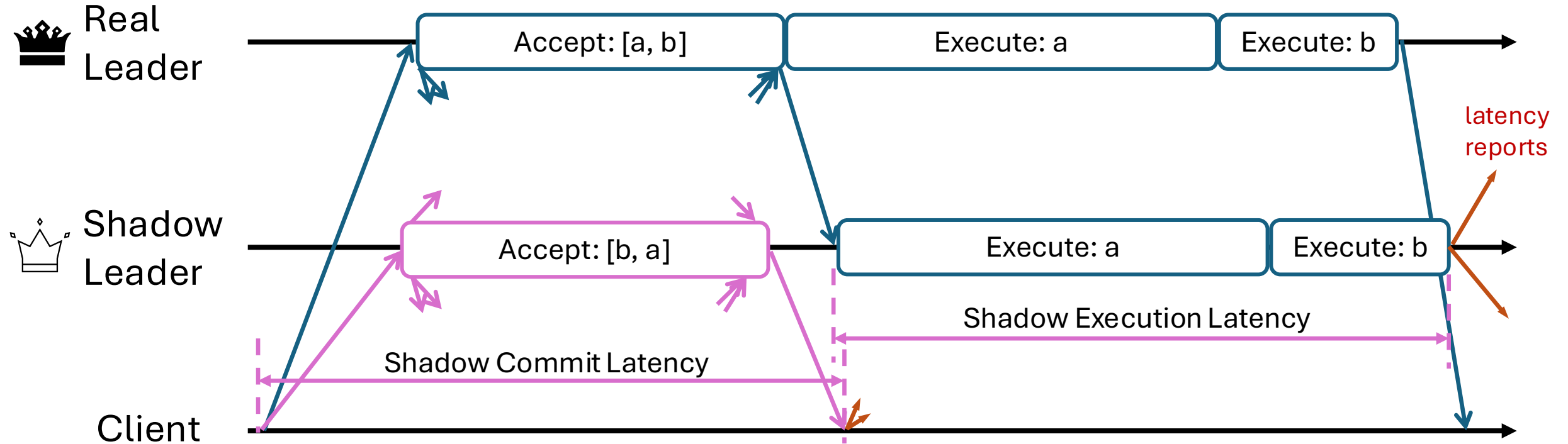
Fail-Slow Detection Workflow



Step 1: Client sends request to both real and shadow leaders

Step 2: Client measures shadow commit latency, reports to every replica

Fail-Slow Detection Workflow

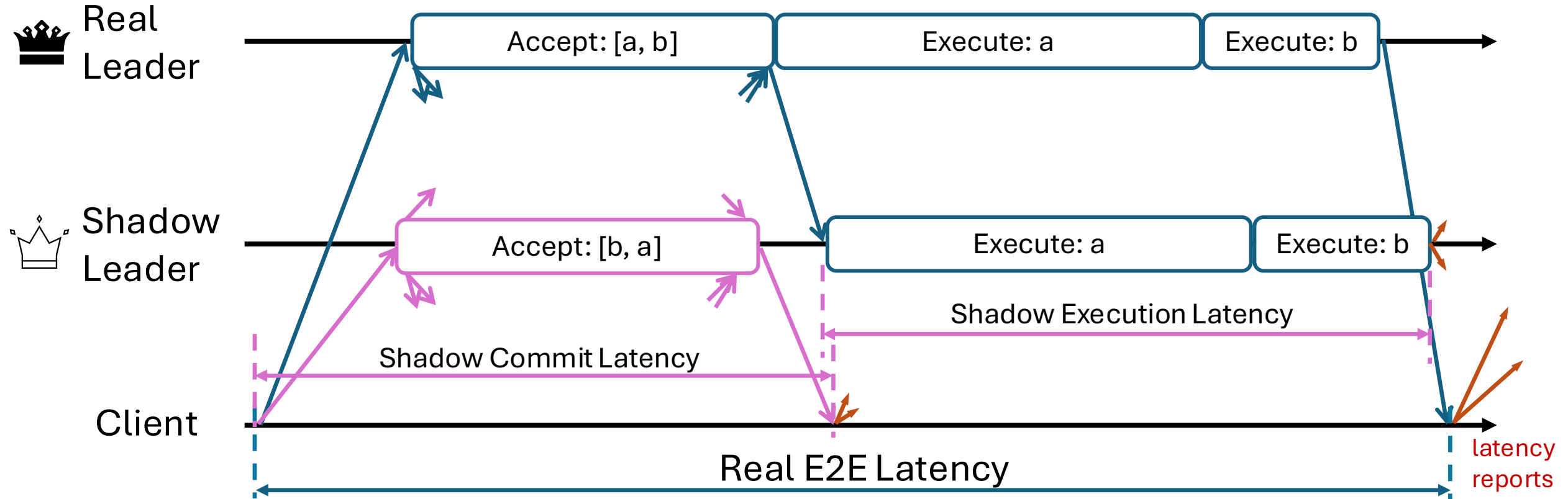


Step 1: Client sends request to both real and shadow leaders

Step 2: Client measures shadow commit latency, reports to every replica

Step 3: Shadow leader measures shadow execution latency, reports to every replica

Fail-Slow Detection Workflow



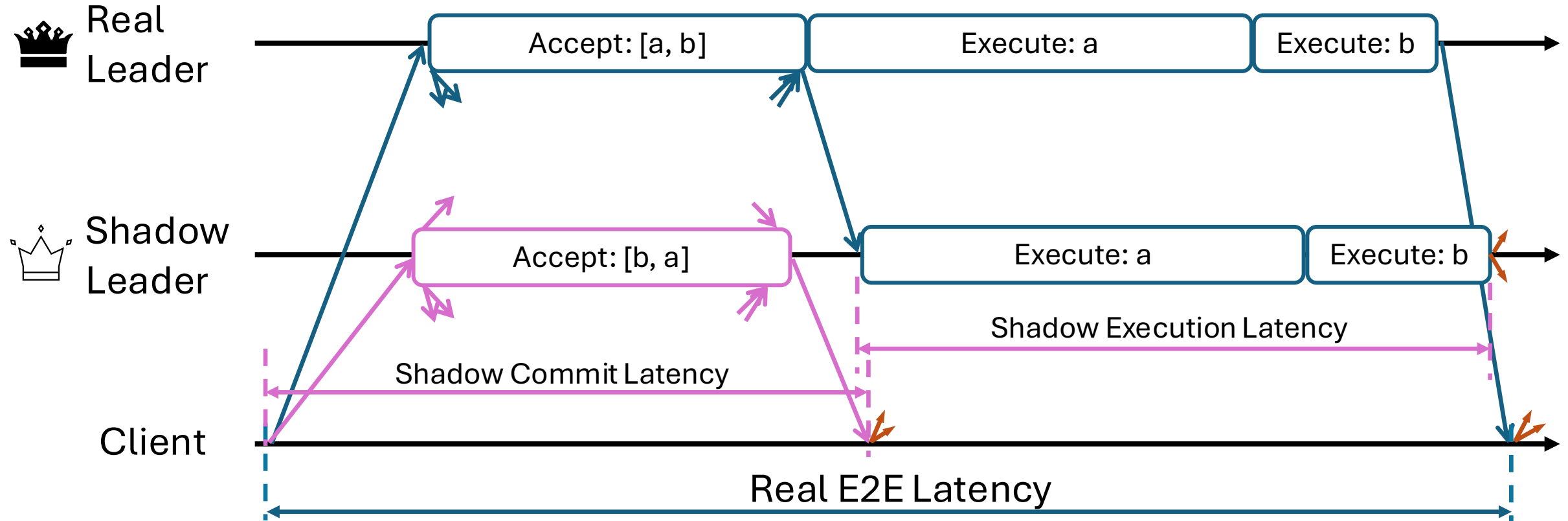
Step 1: Client sends request to both real and shadow leaders

Step 2: Client measures shadow commit latency, reports to every replica

Step 3: Shadow leader measures shadow execution latency, reports to every replica

Step 4: Client measures real e2e latency, reports to every replica

Fail-Slow Detection Workflow



Step 1: Client sends request to both real and shadow leaders

Step 2: Client measures shadow commit latency, reports to every replica

Step 3: Shadow leader measures shadow execution latency, reports to every replica

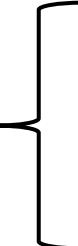
Step 4: Client measures real e2e latency, reports to every replica

Step 5: Each replica reconstructs shadow e2e latency

Fail-Slow Detection

- Shadow processing simulates shadow latencies
- Real e2e latency is directly measured
- Counterfactual shadow e2e latency is reconstructed
- Compare real and shadow e2e latency to detect leader fail-slow

Outlines

- Replicated State Machines
- Fail-slow Faults in Distributed Systems
- Overview of Avicenna
- **Detailed design of Avicenna** 
 - Normal-Case Operation
 - Fail-Slow Detection
 - Fail-Slow Reaction**
- Evaluations
- Discussion and Conclusion

Fail-Slow Reaction

When the real leader is slow,
rotate leadership to the shadow leader!

Fail-Slow Reaction Challenges

Fail-Slow Reaction Challenges

How to rotate the leadership?

The way Multi-Paxos / Raft do?

One round of messages; contact geographically distant replicas.

Fail-Slow Reaction Challenges

How to rotate the leadership?

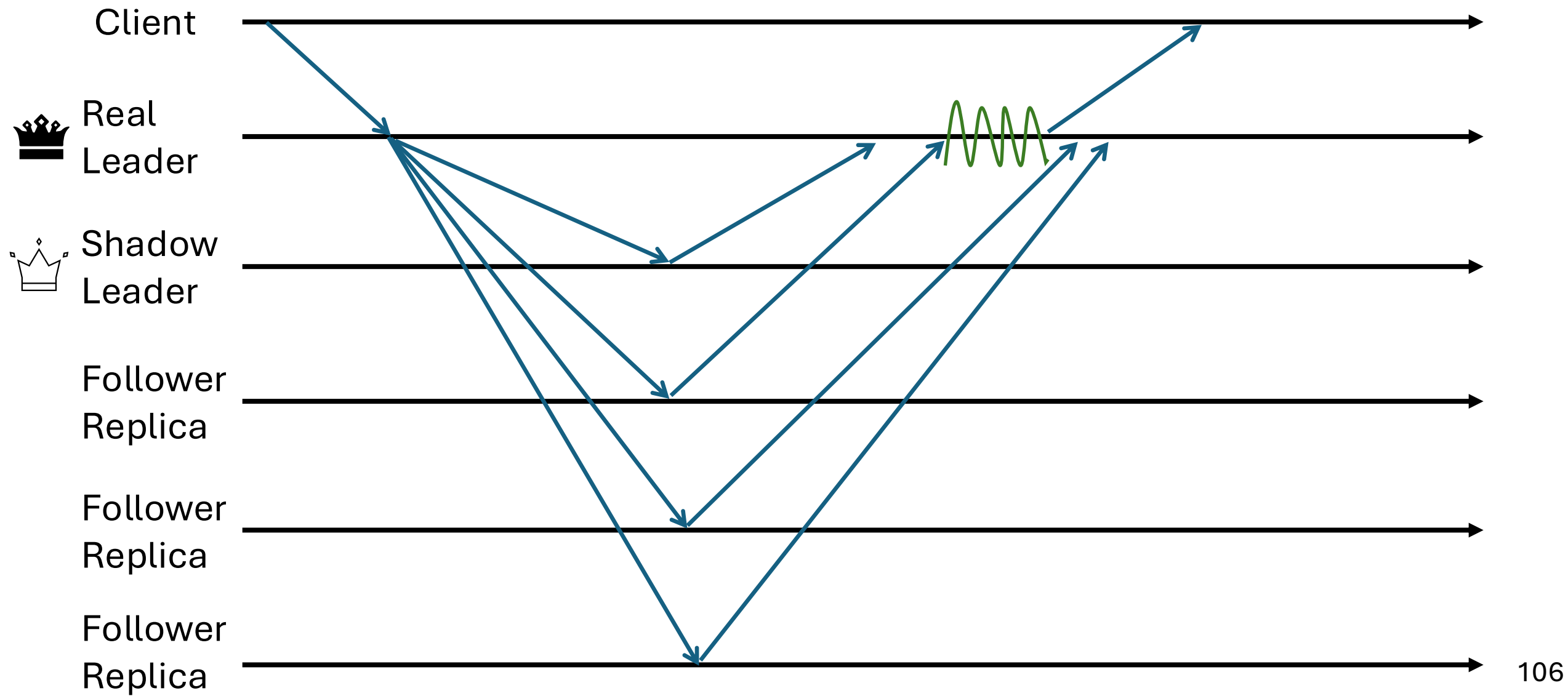
The way Multi-Paxos / Raft do?

One round of messages; contact geographically distant replicas.

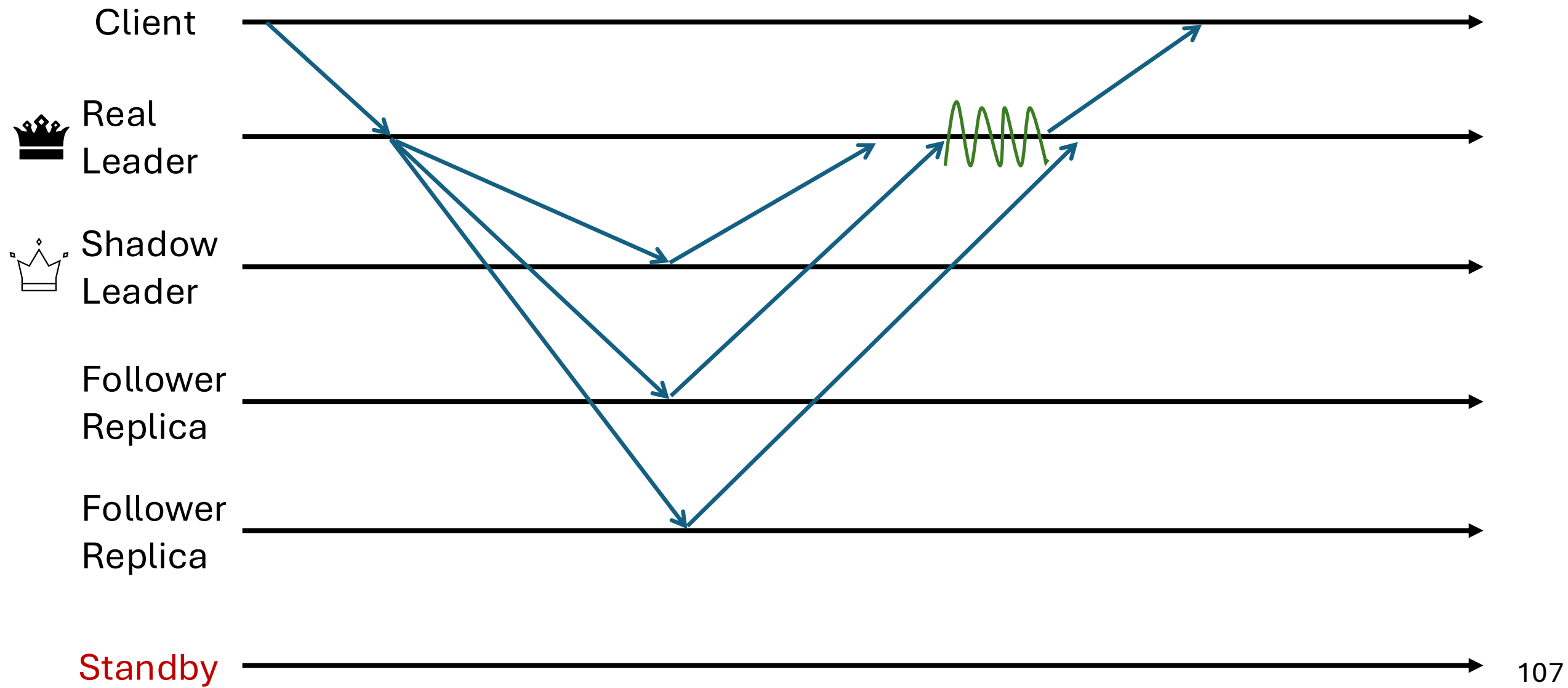
Proposed fast rotation protocol:

- Fast: one message, closest replica
- Safety: refine normal-case operation to constrain commit quorums

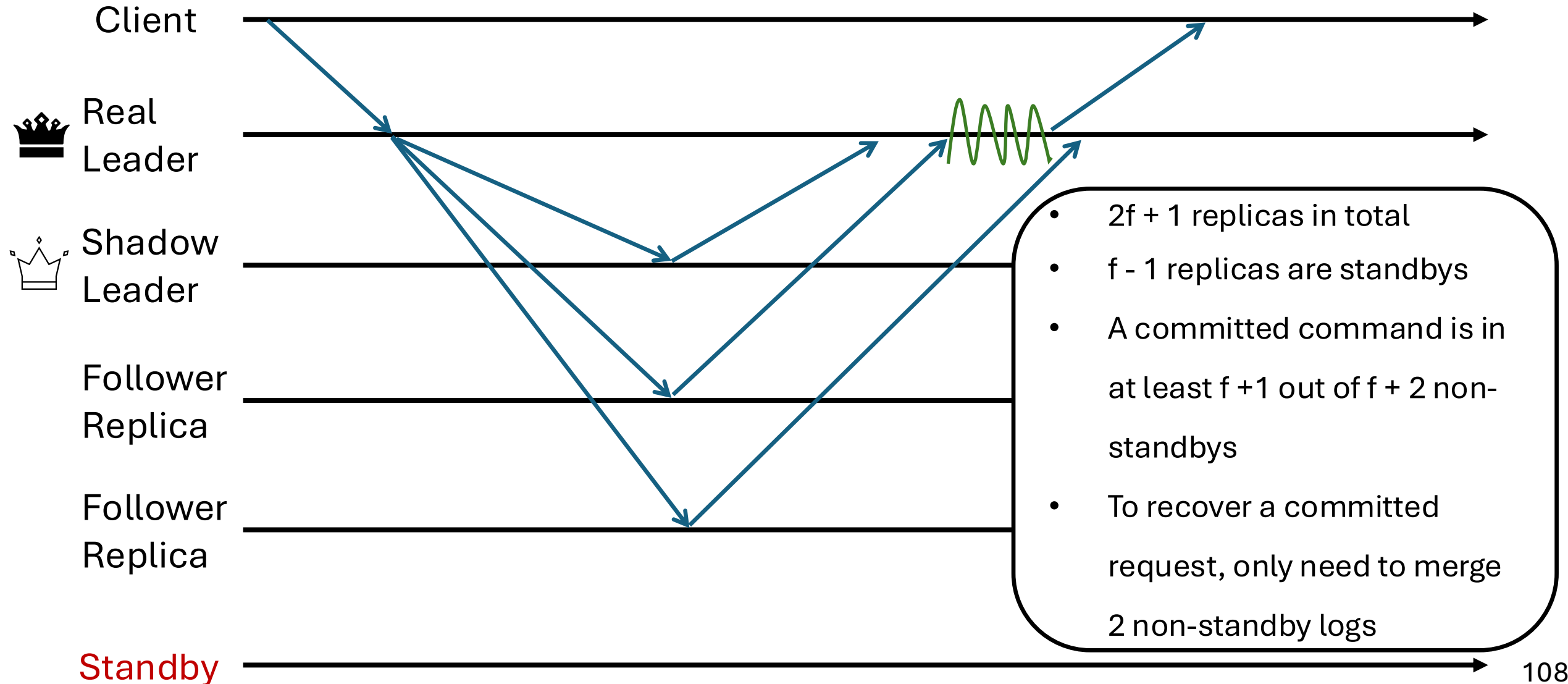
Avicenna Normal-Case Operation Refined



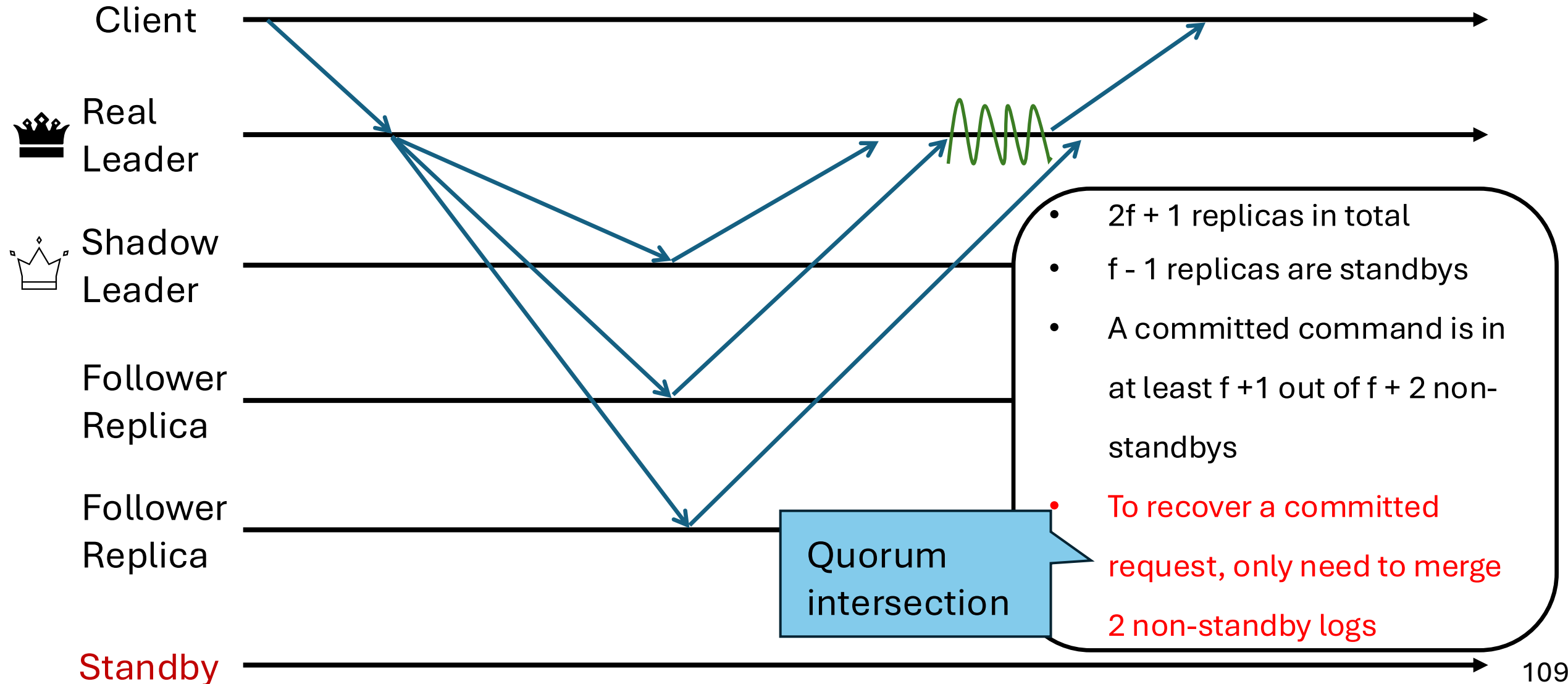
Avicenna Normal-Case Operation Refined



Avicenna Normal-Case Operation Refined



Avicenna Normal-Case Operation Refined



Discover Committed Request by Quorum Intersection

Discover Committed Request by Quorum Intersection

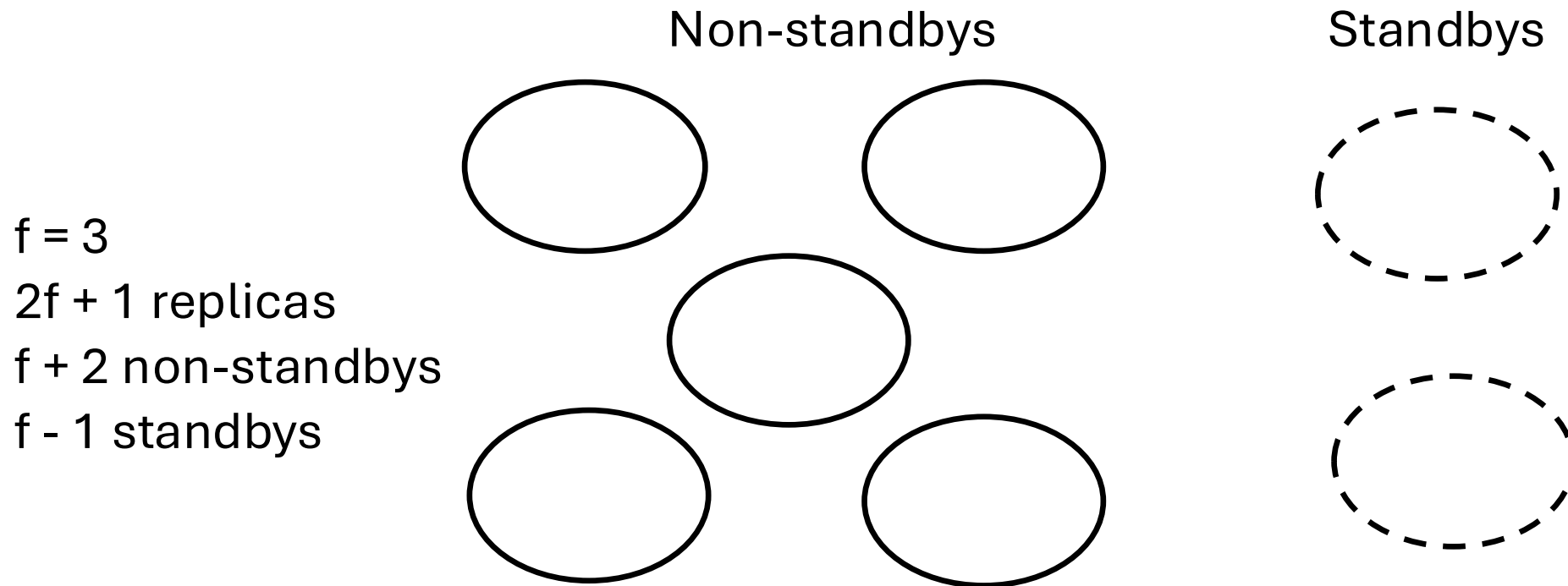
Guarantee linearizability

Core: preserve all committed log entries across rotation

Discover Committed Request by Quorum Intersection

Guarantee linearizability

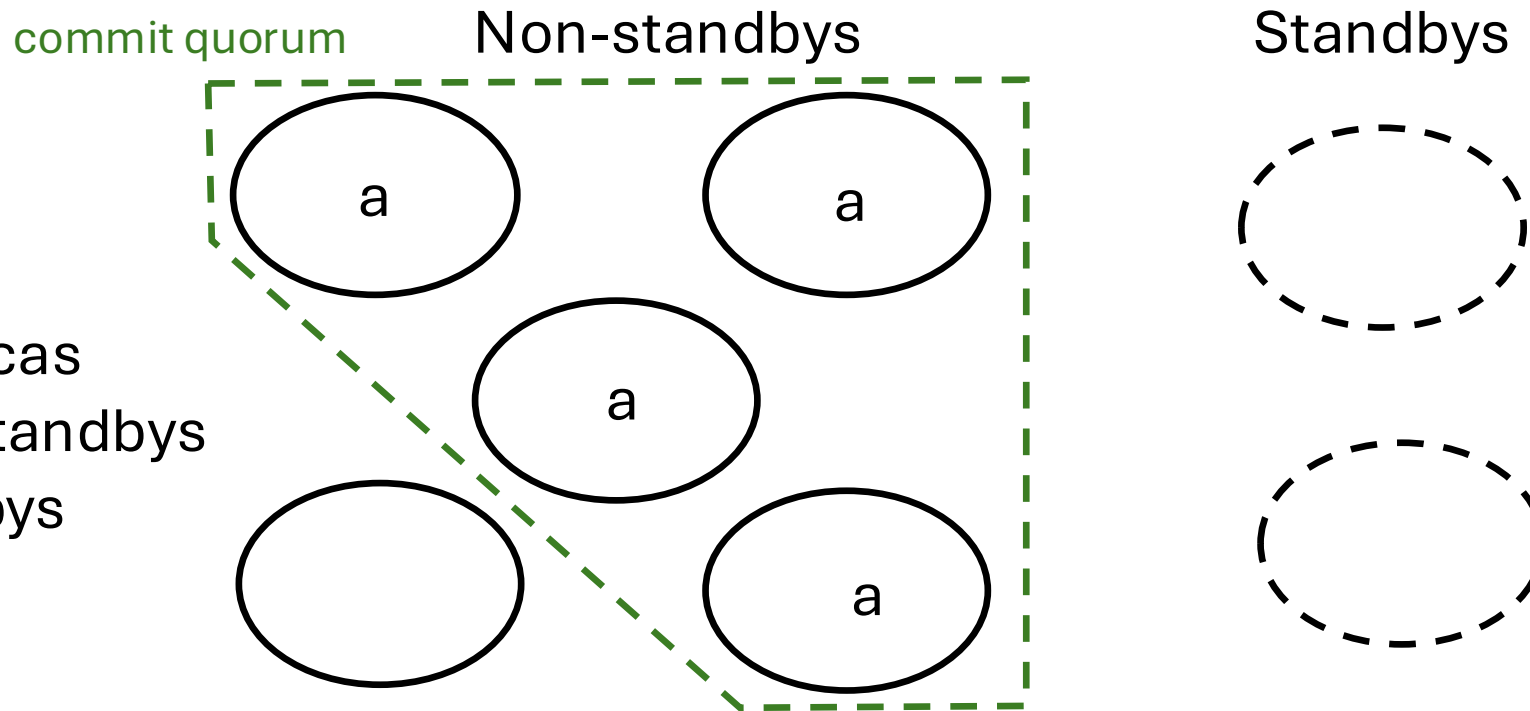
Core: preserve all committed log entries across rotation



Discover Committed Request by Quorum Intersection

Guarantee linearizability

Core: preserve all committed log entries across rotation



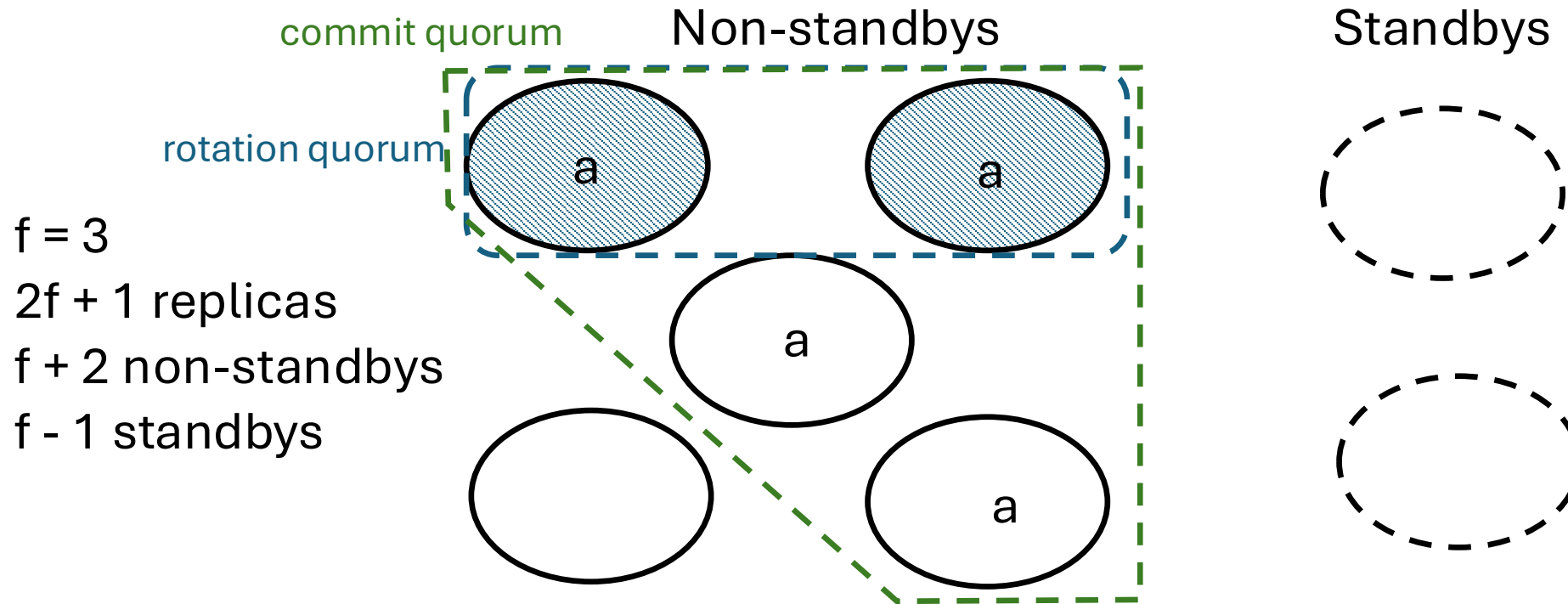
$f = 3$
 $2f + 1$ replicas
 $f + 2$ non-standbys
 $f - 1$ standbys

If command a is committed: at least $f+1$ in $f+2$ non-standbys have accepted it
Merging logs from any 2 non-standbys preserves command a

Discover Committed Request by Quorum Intersection

Guarantee linearizability

Core: preserve all committed log entries across rotation

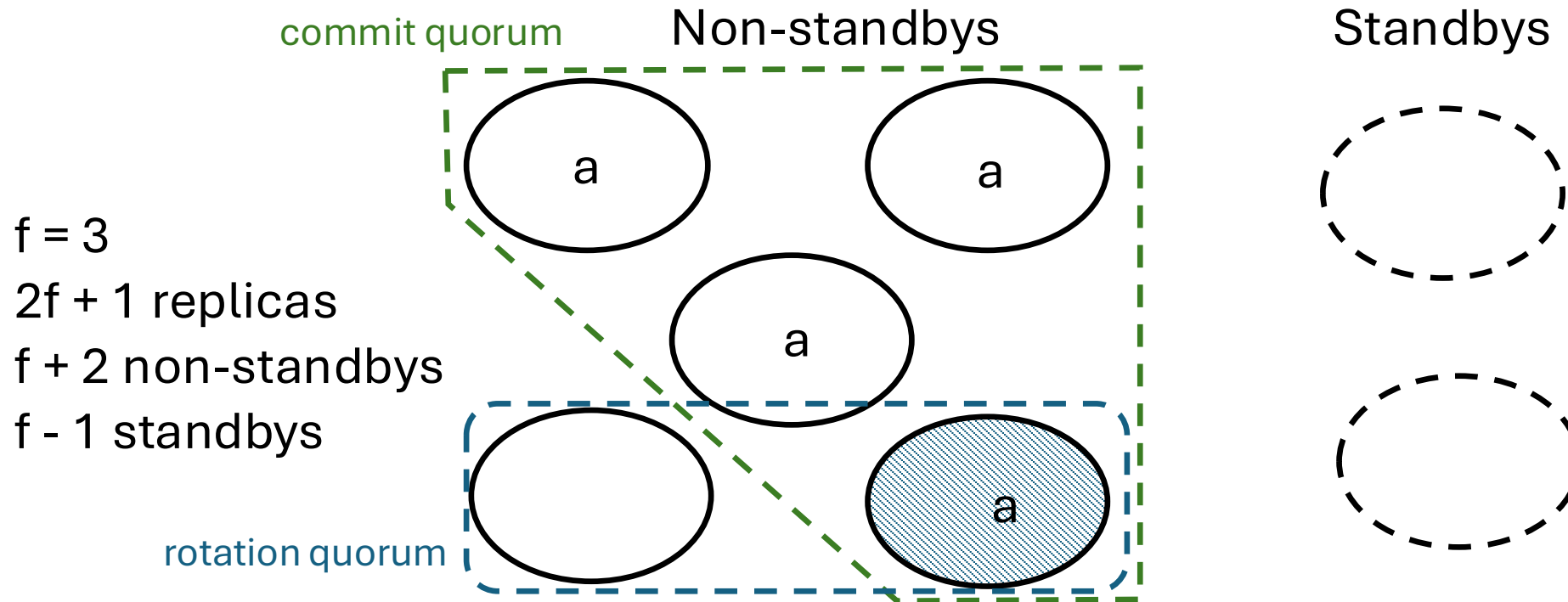


If command *a* is committed: at least $f+1$ in $f+2$ non-standbys have accepted it
Merging logs from any 2 non-standbys preserves command *a*

Discover Committed Request by Quorum Intersection

Guarantee linearizability

Core: preserve all committed log entries across rotation



If command a is committed: at least $f+1$ in $f+2$ non-standbys have accepted it
Merging logs from any 2 non-standbys preserves command a

Log Merging Protocol

Log Merging Protocol

Merge logs from two non-standbys

Log entry state: Received, Accepted, Committed

Log Merging Protocol

Merge logs from two non-standbys

Log entry state: Received, Accepted, Committed

Intuition: 1. if an entry is Committed in one log, definitely preserve
2. if an entry is Accepted in one log, it may have been committed,
we must preserve this entry

Log Merging Protocol

Merge logs from two non-standbys

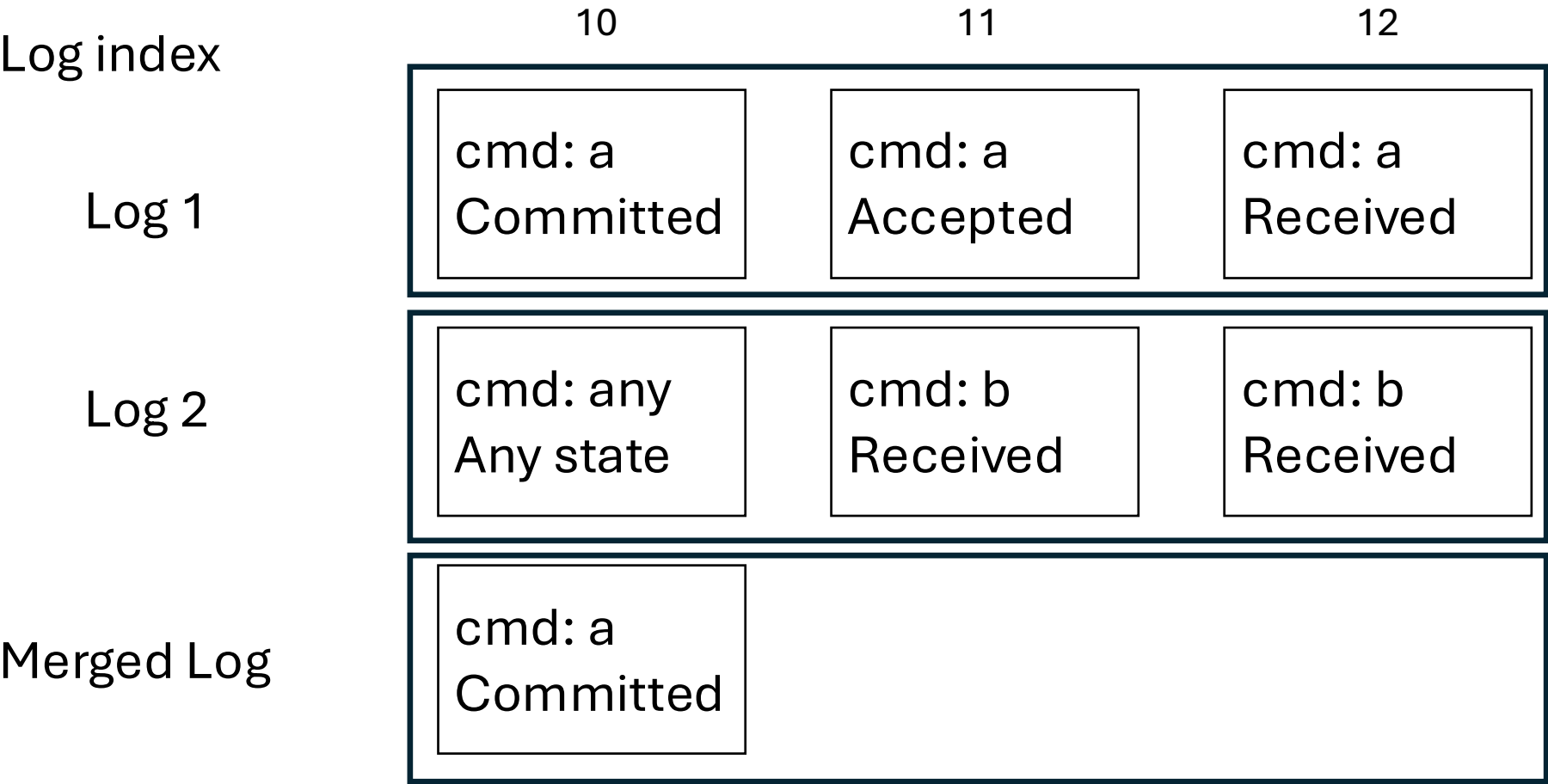
Log entry state: Received, Accepted, Committed

Log index	10	11	12
Log 1	cmd: a Committed	cmd: a Accepted	cmd: a Received
Log 2	cmd: any Any state	cmd: b Received	cmd: b Received
Merged Log			

Log Merging Protocol

Merge logs from two non-standbys

Log entry state: Received, Accepted, Committed



Log Merging Protocol

Merge logs from two non-standbys

Log entry state: Received, Accepted, Committed

Log index	10	11	12
Log 1	cmd: a Committed	cmd: a Accepted	cmd: a Received
Log 2	cmd: any Any state	cmd: b Received	cmd: b Received
Merged Log	cmd: a Committed	cmd: a Accepted	

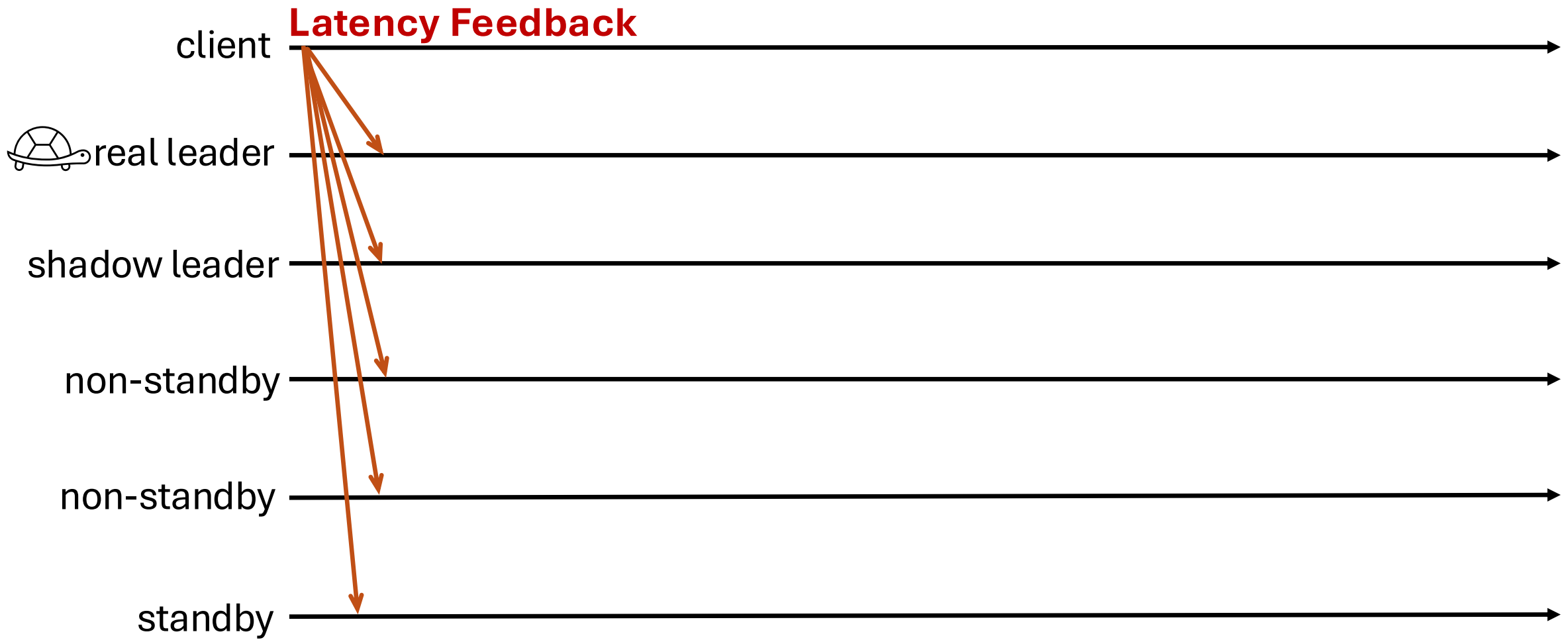
Log Merging Protocol

Merge logs from two non-standbys

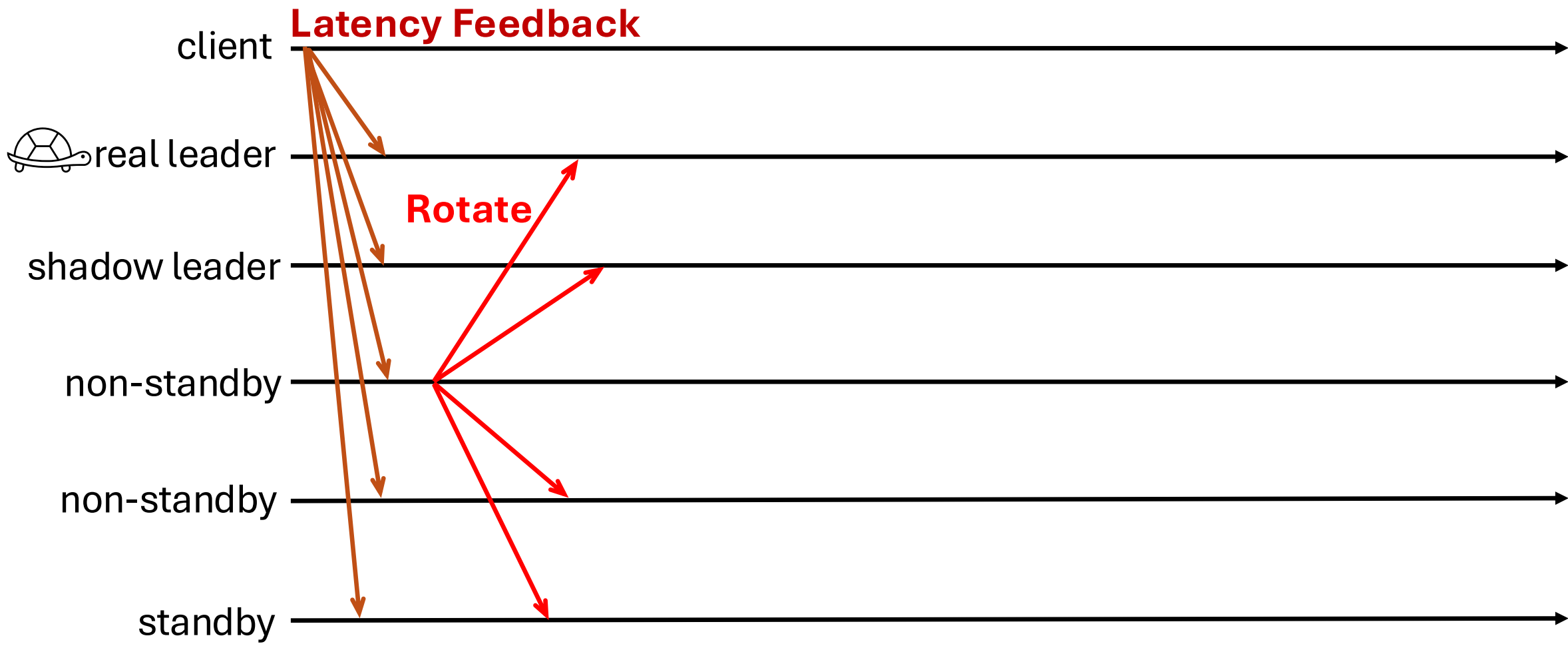
Log entry state: Received, Accepted, Committed

Log index	10	11	12
Log 1	cmd: a Committed	cmd: a Accepted	cmd: a Received
Log 2	cmd: any Any state	cmd: b Received	cmd: b Received
Merged Log	cmd: a Committed	cmd: a Accepted	cmd: a or b Received

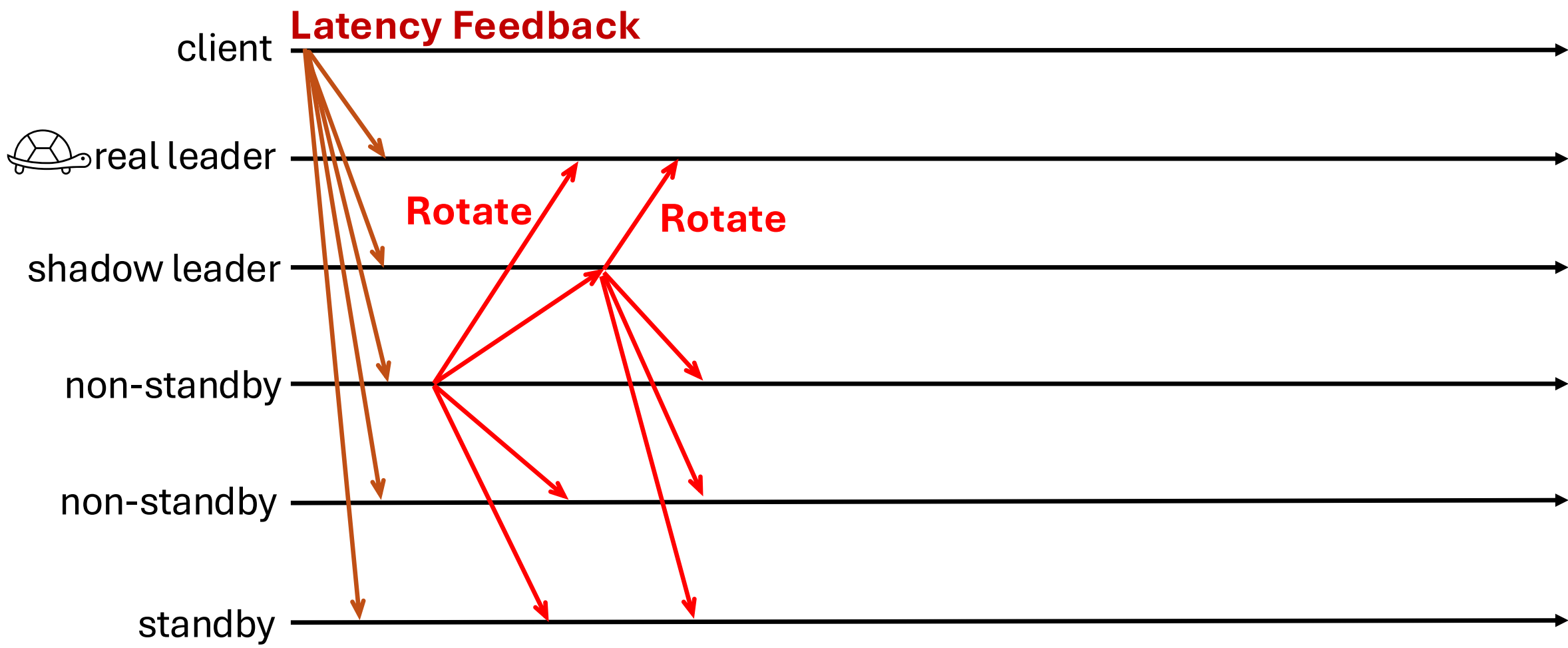
Rotation Workflow



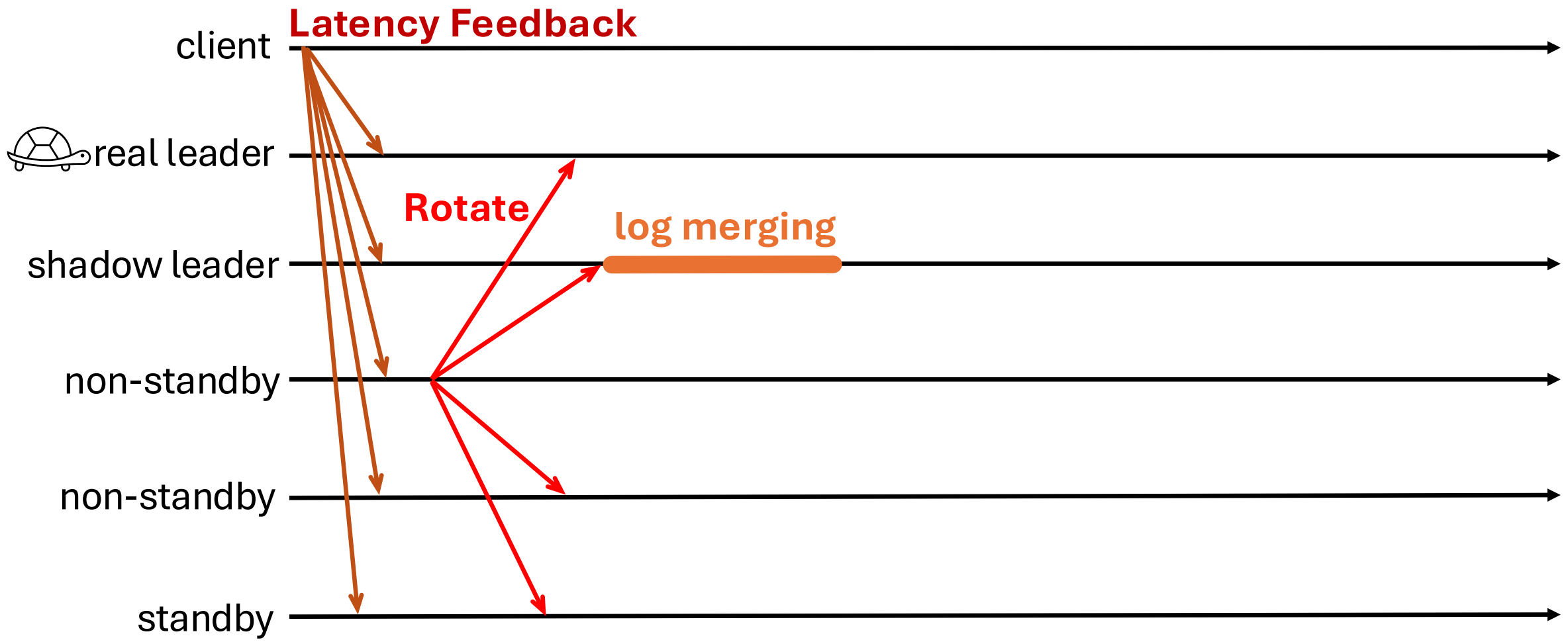
Rotation Workflow



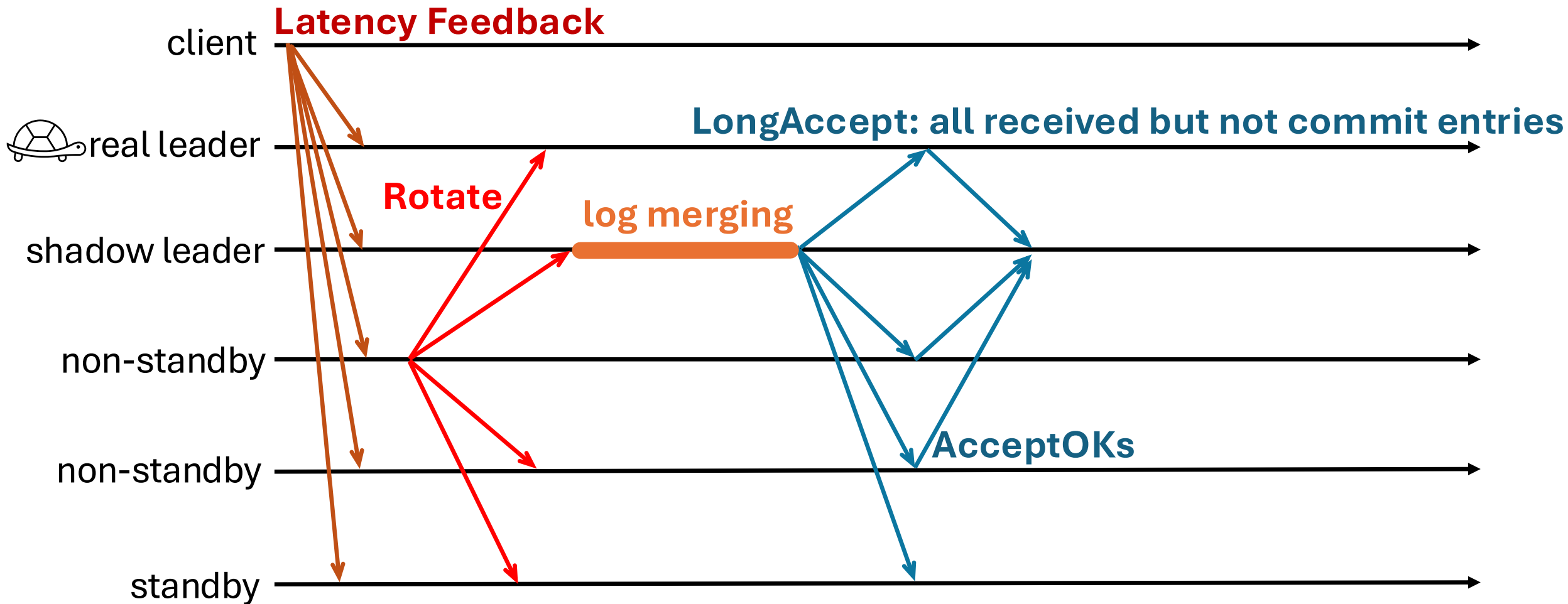
Rotation Workflow



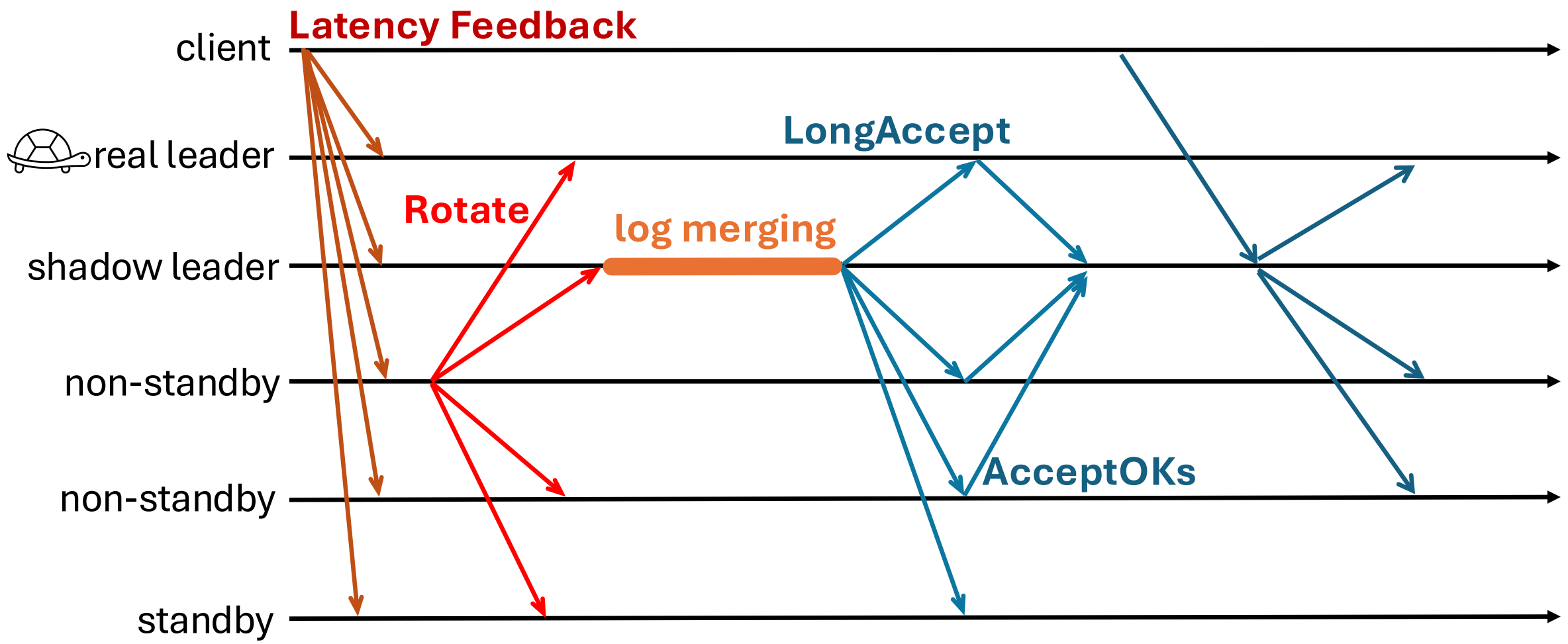
Rotation Workflow



Rotation Workflow



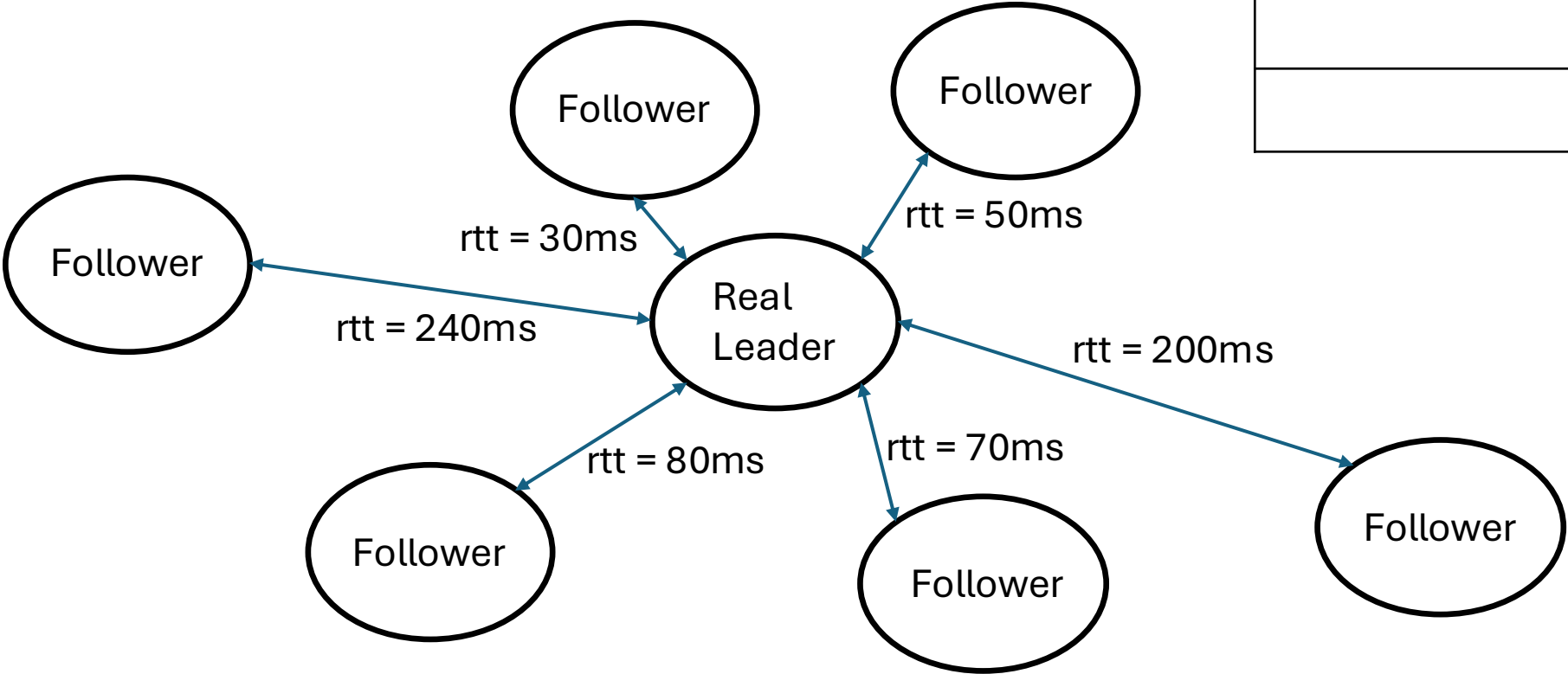
Rotation Workflow



How to Choose Standbys

How to Choose Standbys

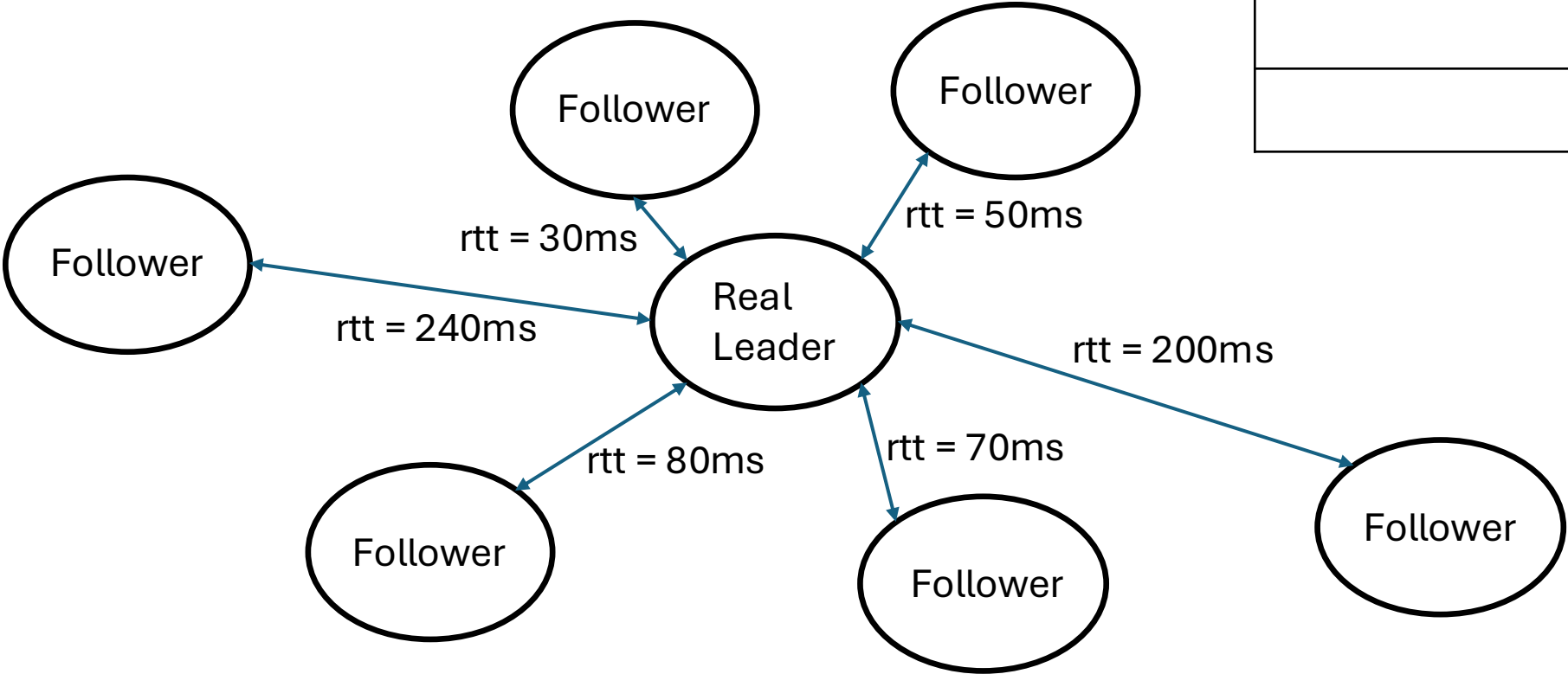
Standby choice	Commit latency



How to Choose Standbys

No standby

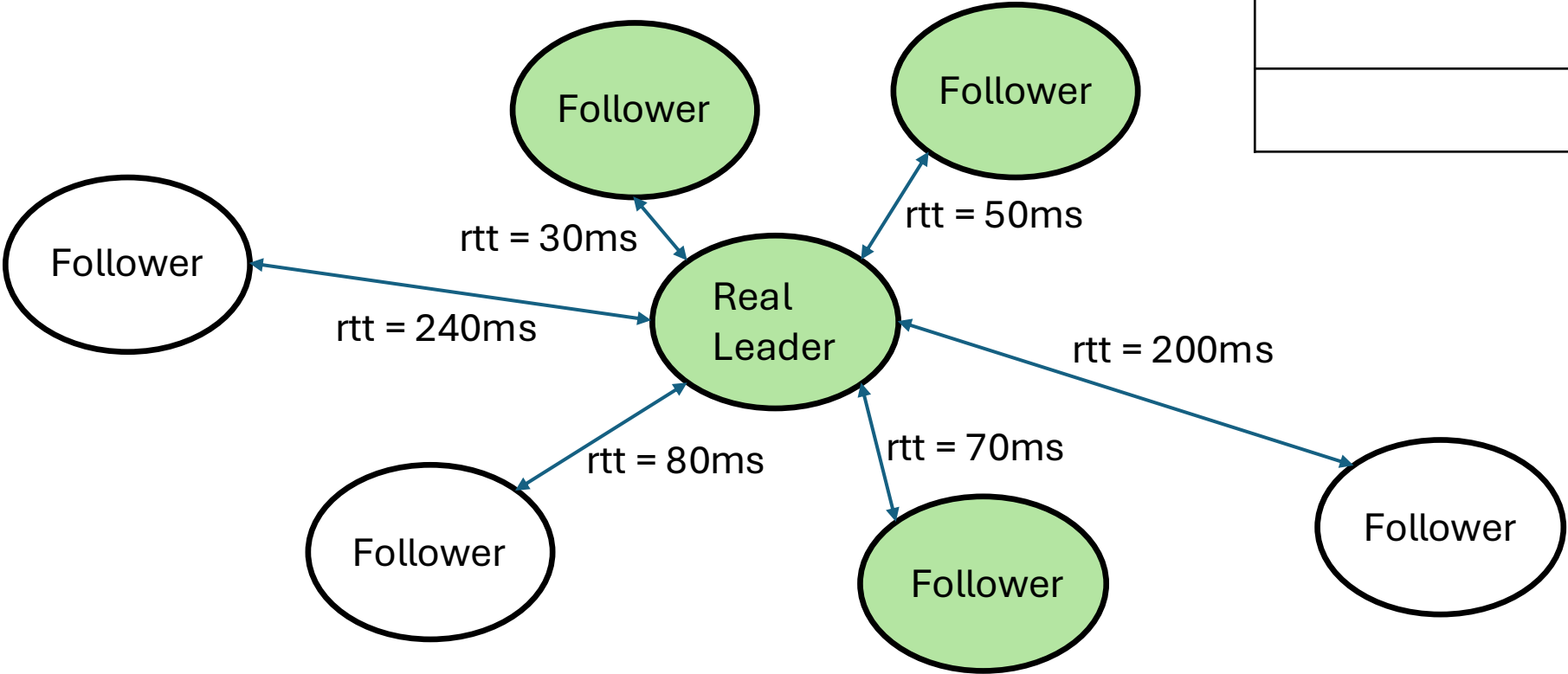
Standby choice	Commit latency
No standby	



How to Choose Standbys

No standby

Standby choice	Commit latency
No standby	70ms

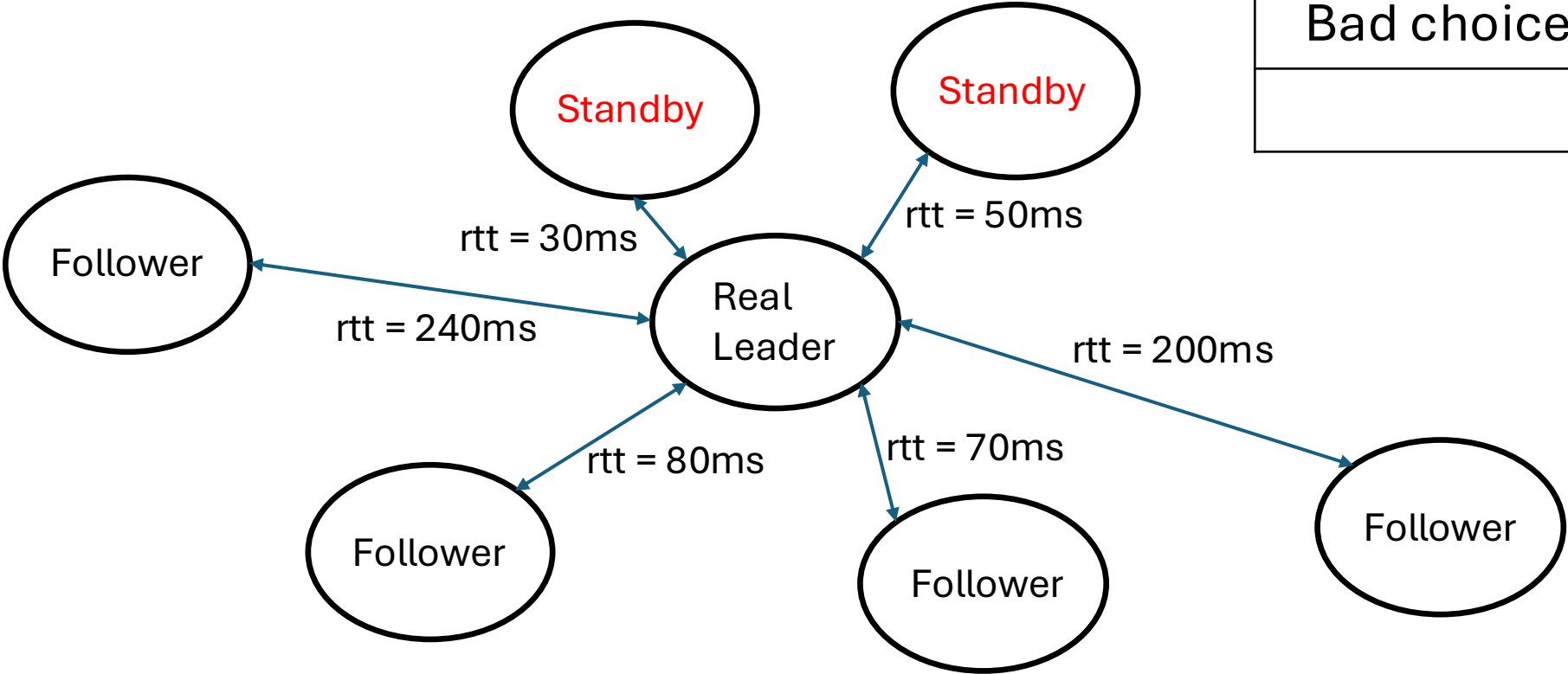


Commit when reaching a majority
Commit latency = 70ms, optimal

How to Choose Standbys

A bad choice of standbys

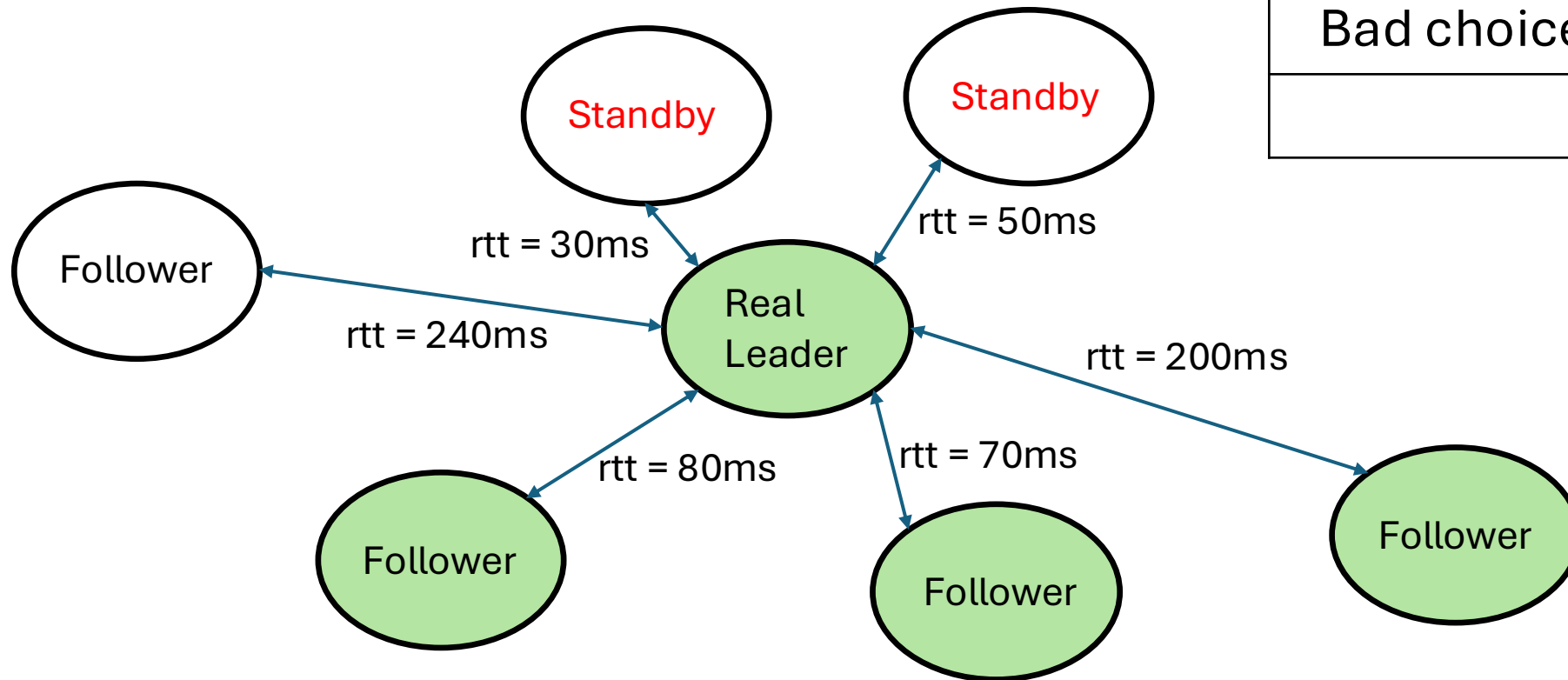
Standby choice	Commit latency
No standby	70ms
Bad choice	



How to Choose Standbys

A bad choice of standbys

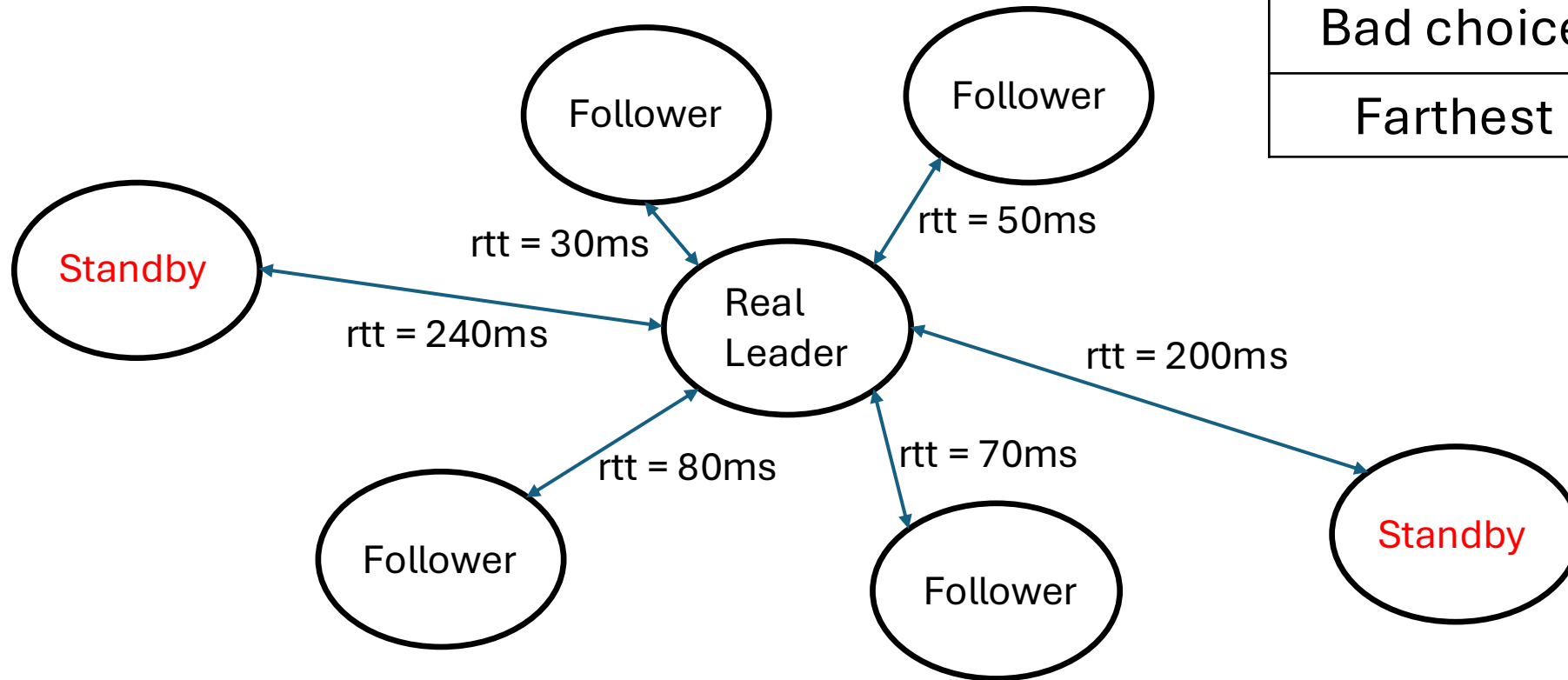
Standby choice	Commit latency
No standby	70ms
Bad choice	200ms



Commit when reaching a majority, only **non-standbys**
Commit latency = 200ms, much higher than no-standby case

How to Choose Standbys

Exploiting geographic locality!

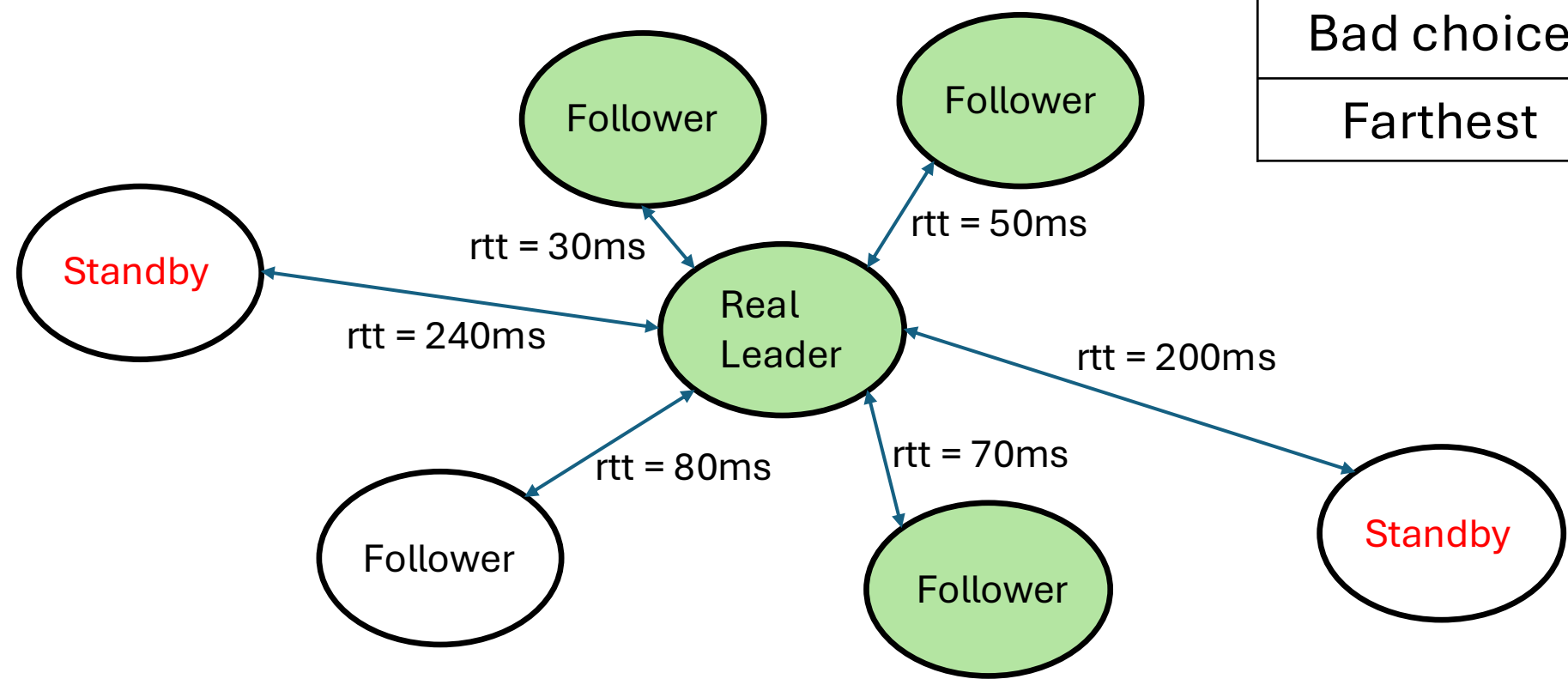


Standby choice	Commit latency
No standby	70ms
Bad choice	200ms
Farthest	

How to Choose Standbys

Exploiting geographic locality!

Standby choice	Commit latency
No standby	70ms
Bad choice	200ms
Farthest	70ms

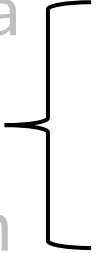


Commit when reaching a majority, only **non-standbys**
Commit latency = 70ms, same as no-standby case, optimal

Fail-Slow Reaction

- In normal case, $f - 1$ farthest replicas are standbys that don't participate in commit quorums
- To initiate a rotation, replicas broadcast Rotate messages, containing their logs
- Every replica merges logs from at least 2 non-standbys
- The shadow leader takes over as the new real leader

Outlines

- Replicated State Machines
 - Fail-slow Faults in Distributed Systems
 - Overview of Avicenna
 - Detailed design of Avicenna
 - **Evaluations**
 - Discussion and Conclusion
- 
- Normal-Case Latency
 - Fail-Slow Tolerance
 - Throughput Latency

Experimental Setup

Replica

Replica

Replica

Replica

Replica

Replica

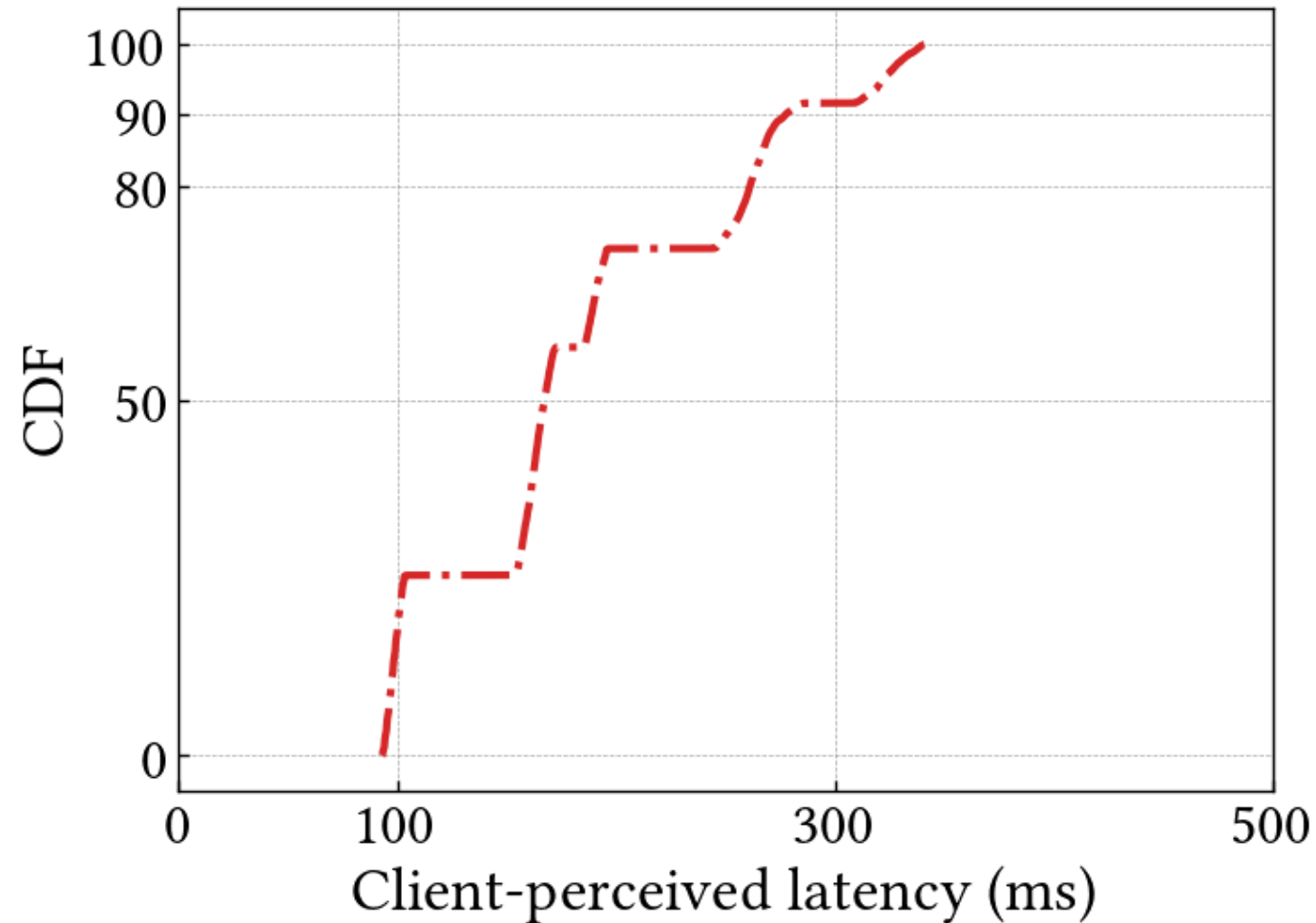


Replica

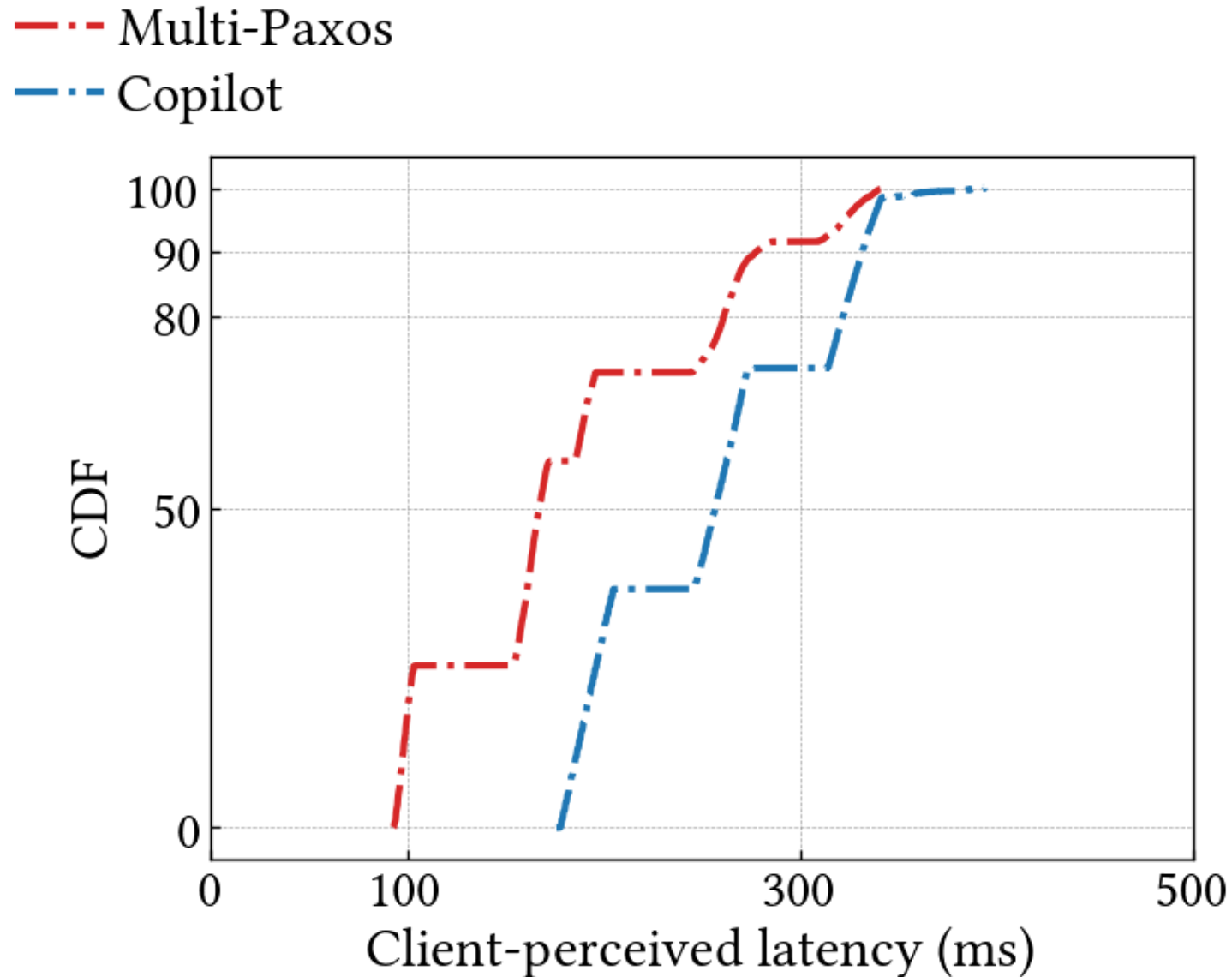
AWS ec2 instances
7 geo-distributed replicas
20 closed-loop clients each site

Evaluation – Normal-Case Latency

— Multi-Paxos

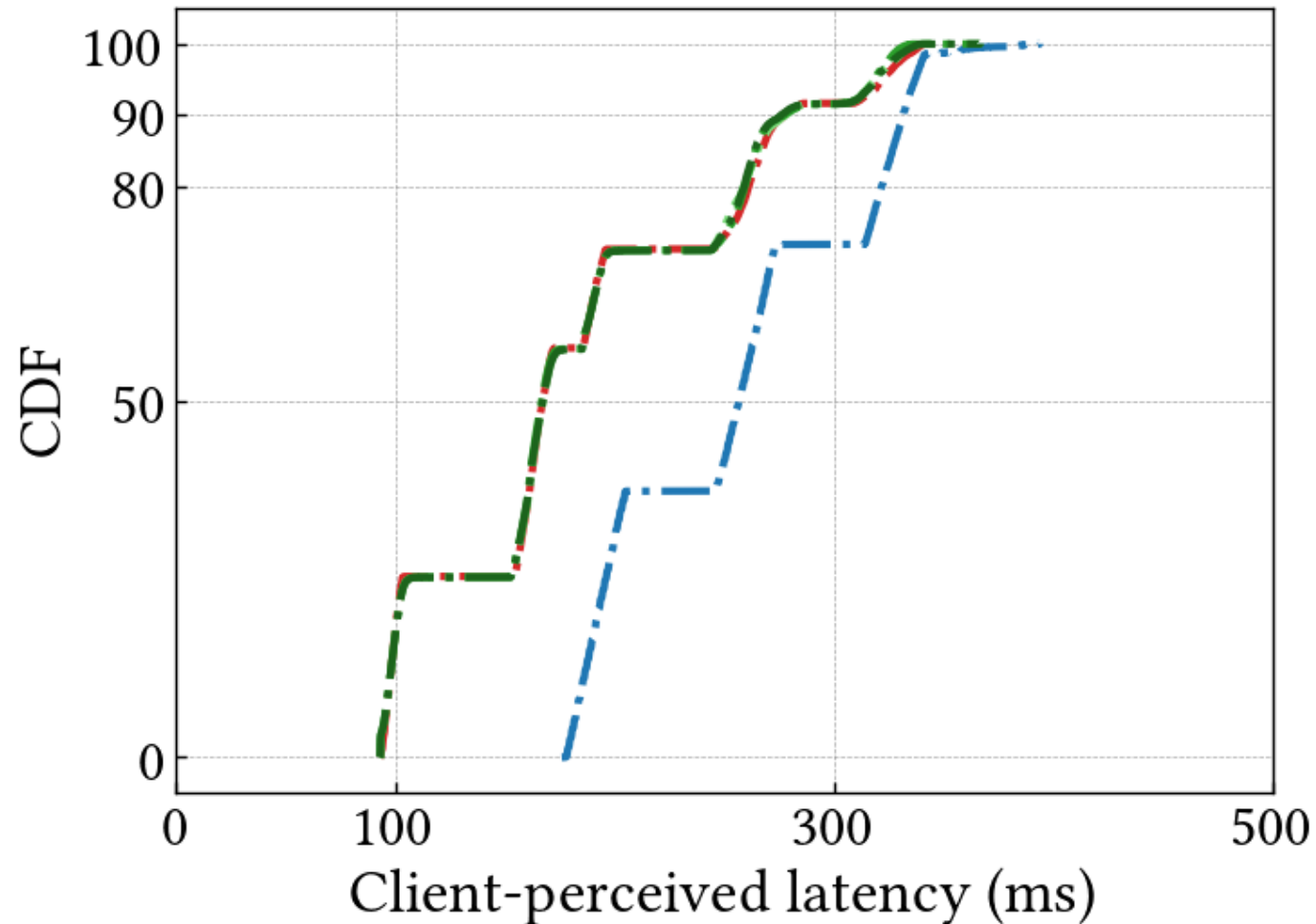


Evaluation – Normal-Case Latency



Evaluation – Normal-Case Latency

Multi-Paxos Avicenna-0% Avicenna-25%
Copilot Avicenna-5%

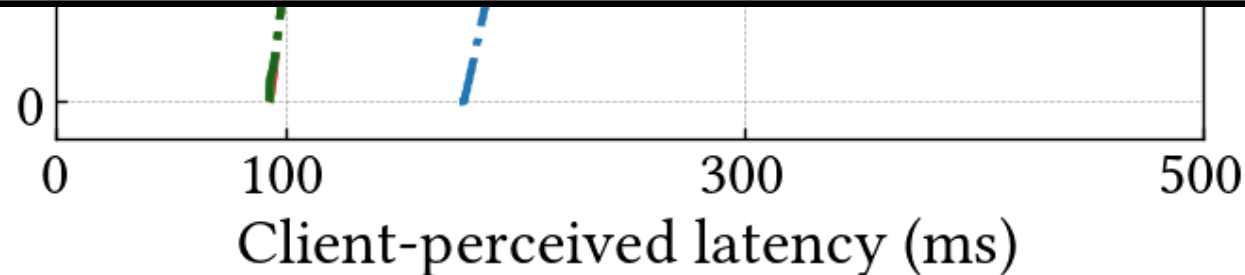


Evaluation – Normal-Case Latency

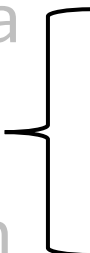
— Multi-Paxos — Avicenna-0% — Avicenna-25%
— Copilot — Avicenna-5%

Avicenna achieves

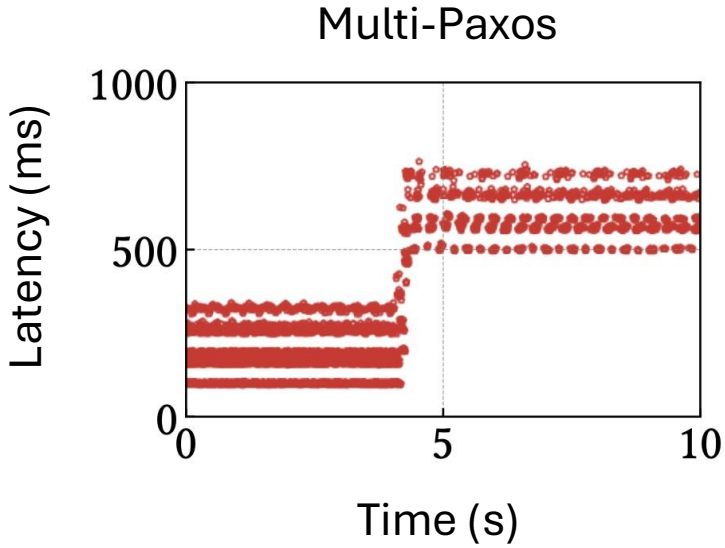
1. the same normal-case latency as Multi-Paxos
2. much lower normal-case latency than Copilot



Outlines

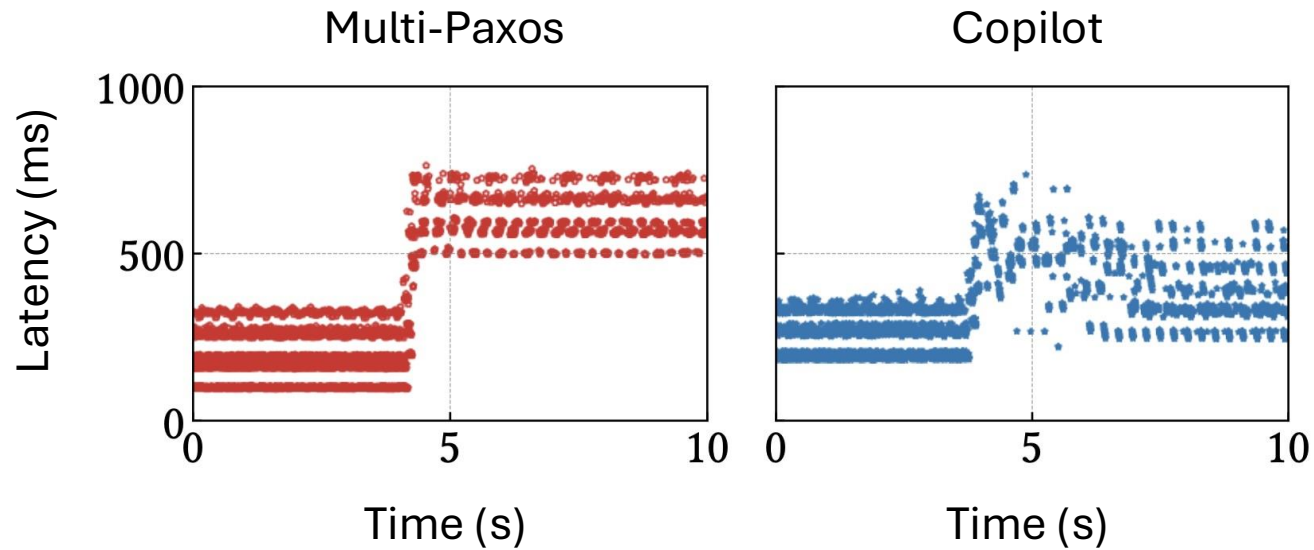
- Fail-slow Faults in Distributed Systems
 - Current RSMs Are Not Fail-Slow Fault-Tolerant
 - Basic Intuition of Avicenna
 - Detailed design of Avicenna
 - **Evaluations**
 - Discussion and Conclusion
- 
- Normal-Case Latency
 - Fail-Slow Tolerance**
 - Throughput Latency

Evaluation – Fail-Slow Tolerance



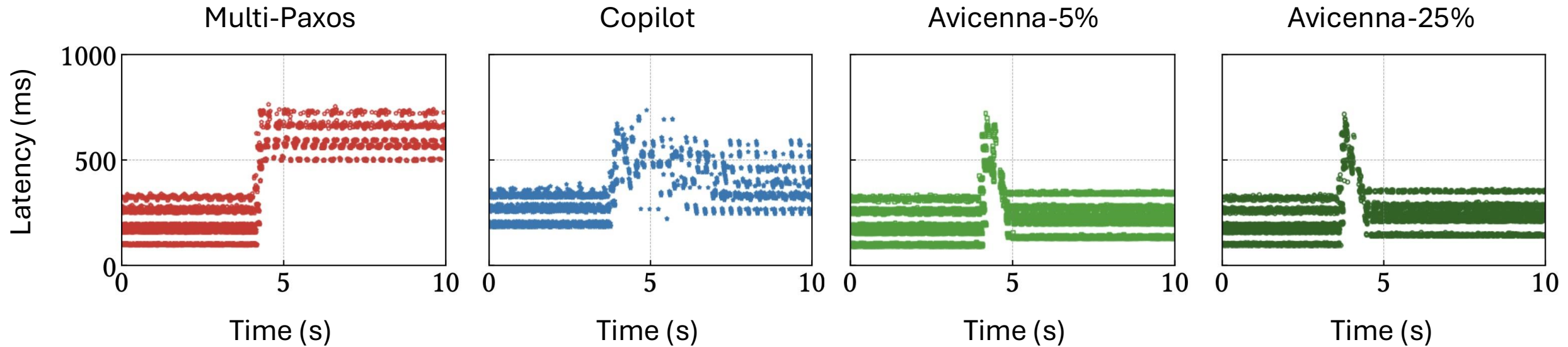
100ms message delay on the leader

Evaluation – Fail-Slow Tolerance



100ms message delay on the leader

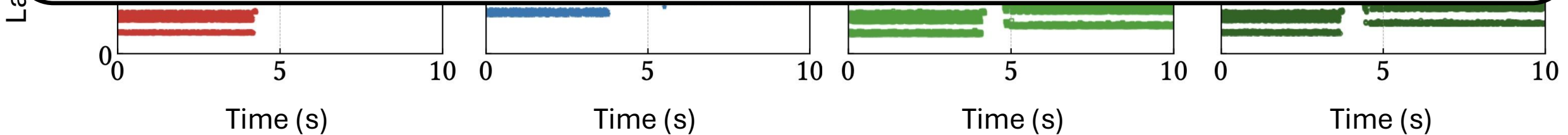
Evaluation – Fail-Slow Tolerance



100ms message delay on the leader

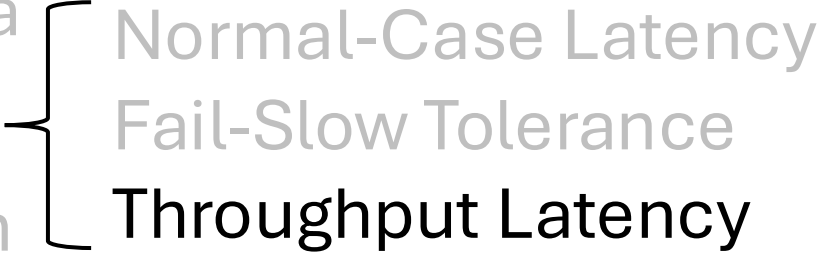
Evaluation – Fail-Slow Tolerance

When the leader is slow, Avicenna detects and rotates the leadership to the shadow leader, achieving fail-slow tolerance



100ms message delay on the leader

Outlines

- Fail-slow Faults in Distributed Systems
 - Current RSMs Are Not Fail-Slow Fault-Tolerant
 - Basic Intuition of Avicenna
 - Detailed design of Avicenna
 - **Evaluations**
 - Discussion and Conclusion
- 
- Normal-Case Latency
 - Fail-Slow Tolerance
 - Throughput Latency**

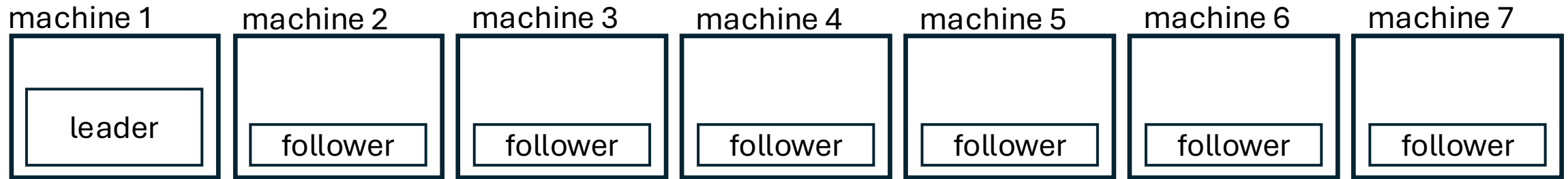
Evaluation Setup

In cloudlab, all replicas and clients are in the same data center
7 replicas, variable clients

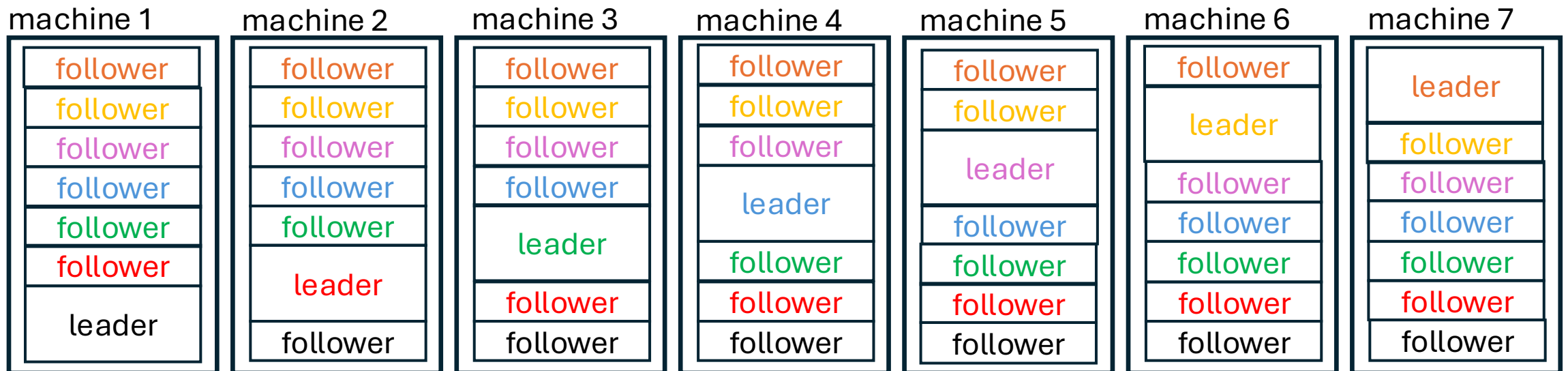
Evaluation Setup

In cloudlab, all replicas and clients are in the same data center

7 replicas, variable clients



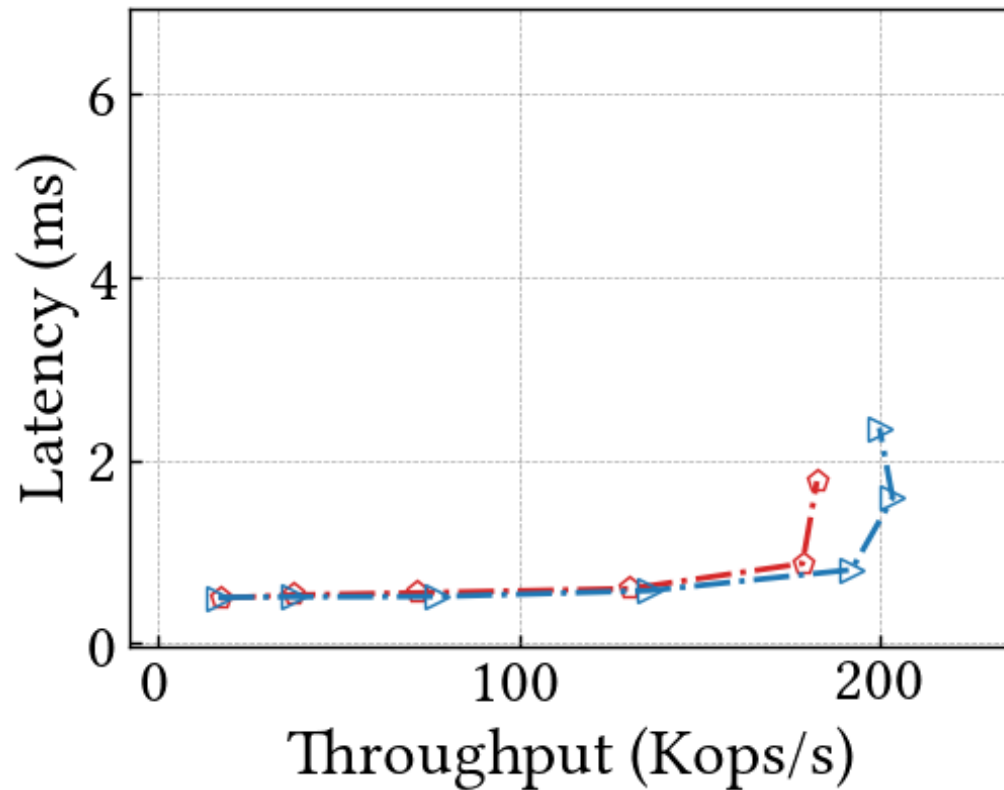
1-shard setup



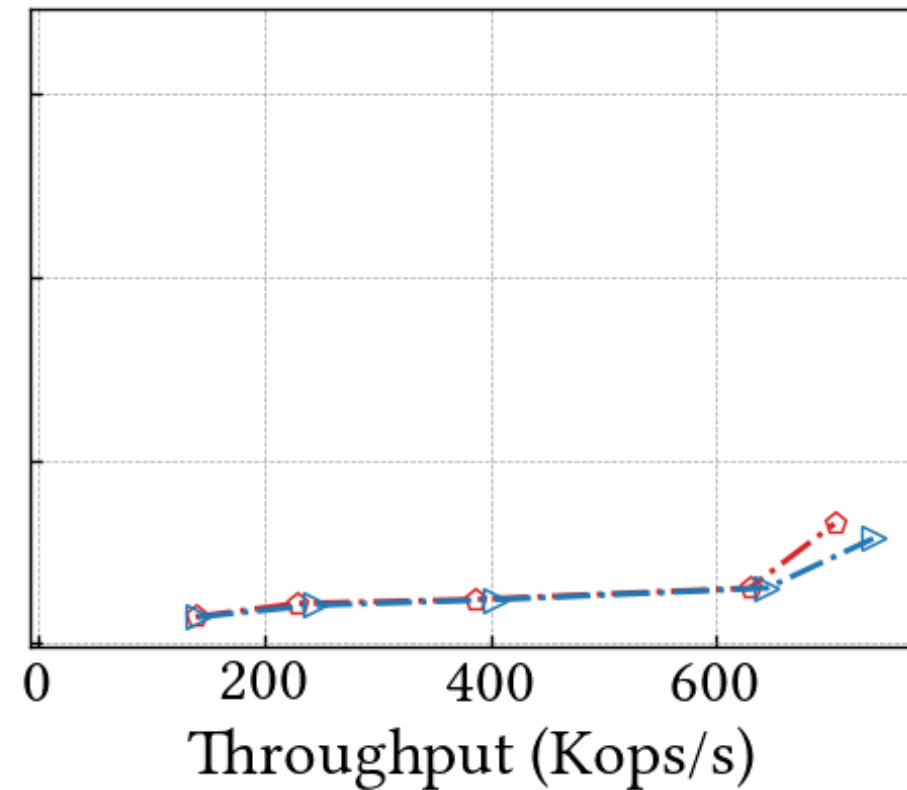
7-shard setup

Evaluation – Throughput Latency

- ◇— Multi-Paxos FVC
- ▷— Multi-Paxos FVC (-)



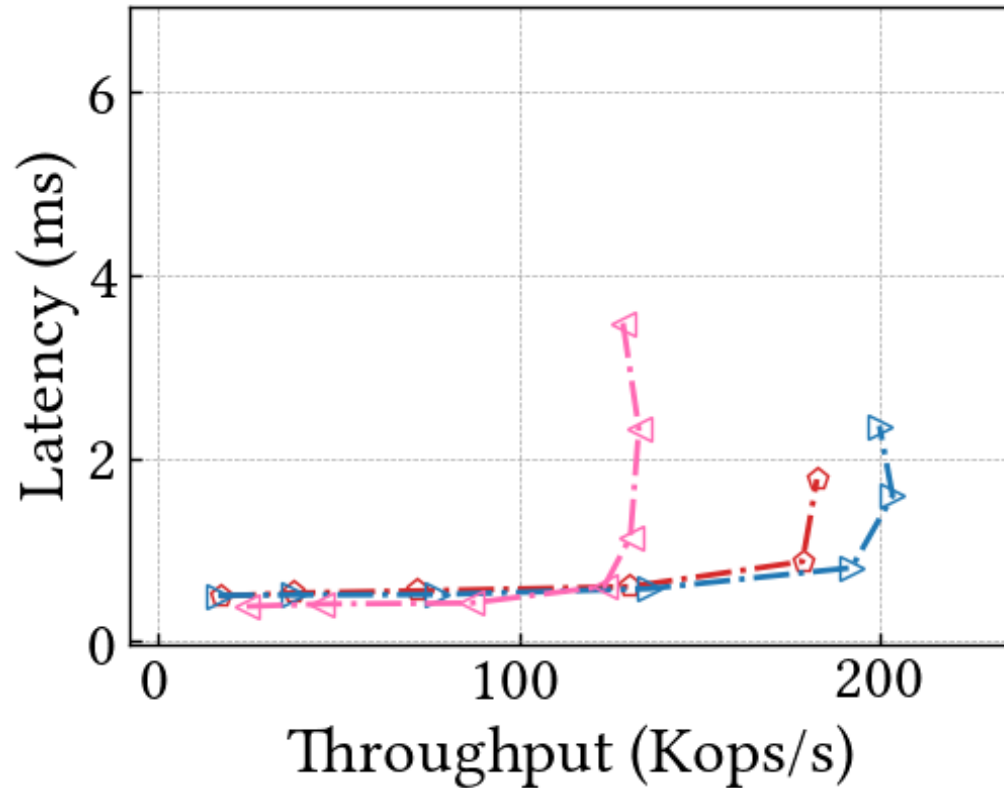
(a) 1 shard



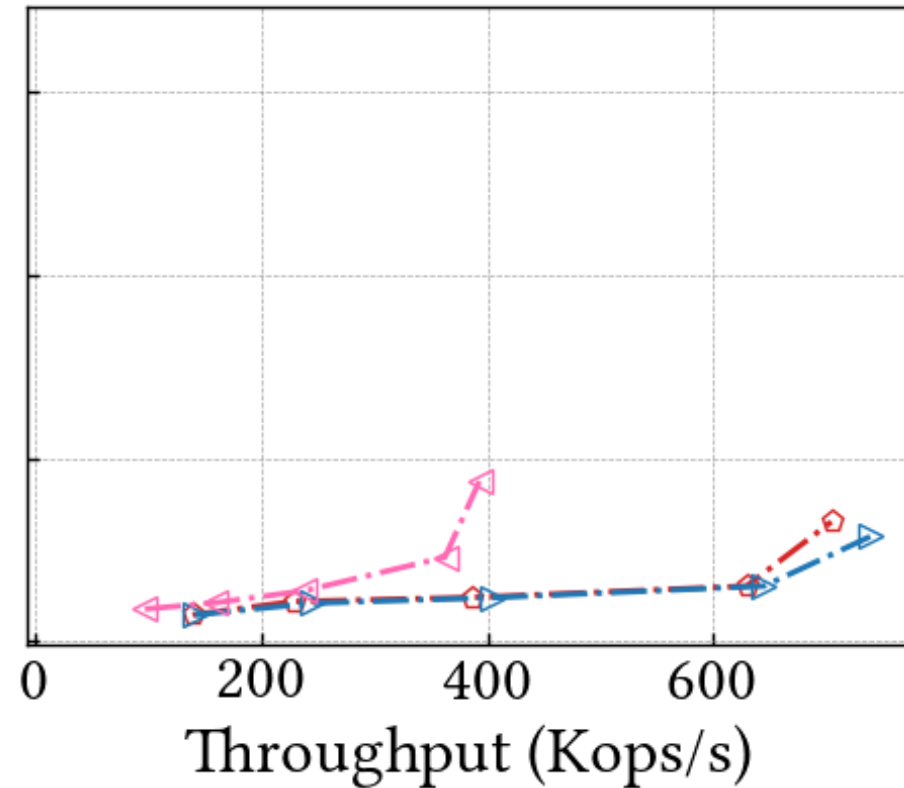
(b) 7 shards

Evaluation – Throughput Latency

- Multi-Paxos FVC
- Multi-Paxos FVC (-)
- Copilot



(a) 1 shard

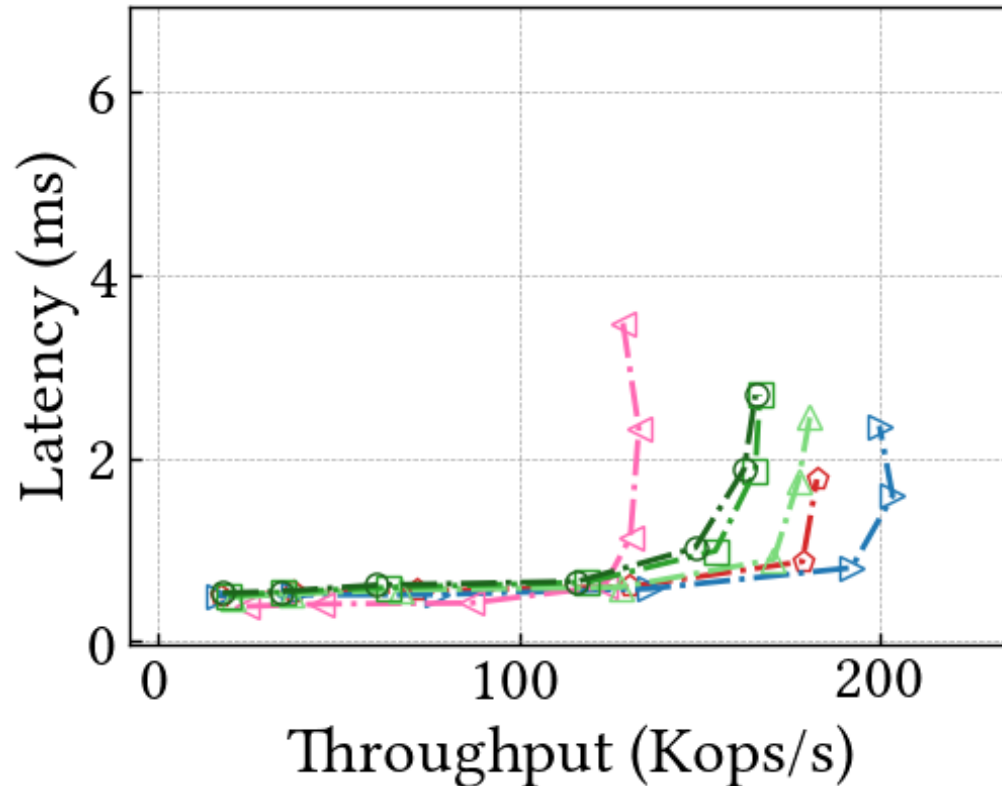


(b) 7 shards

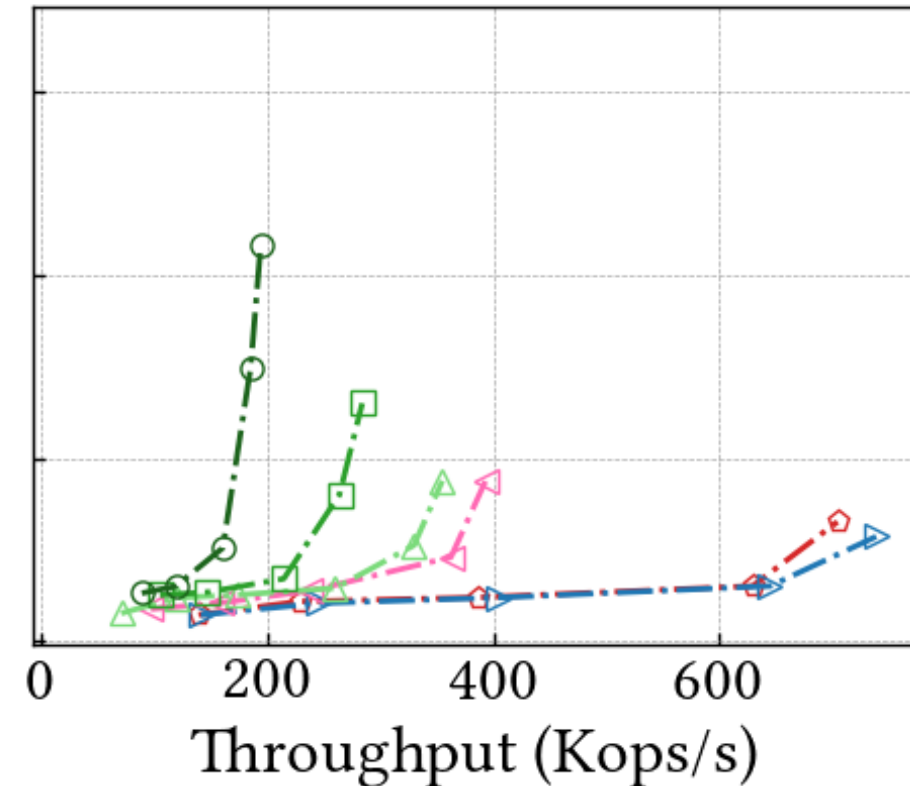
Evaluation – Throughput Latency

- Multi-Paxos FVC
- Multi-Paxos FVC (-)
- Copilot

- Avicenna-0%
- Avicenna-5%
- Avicenna-25%



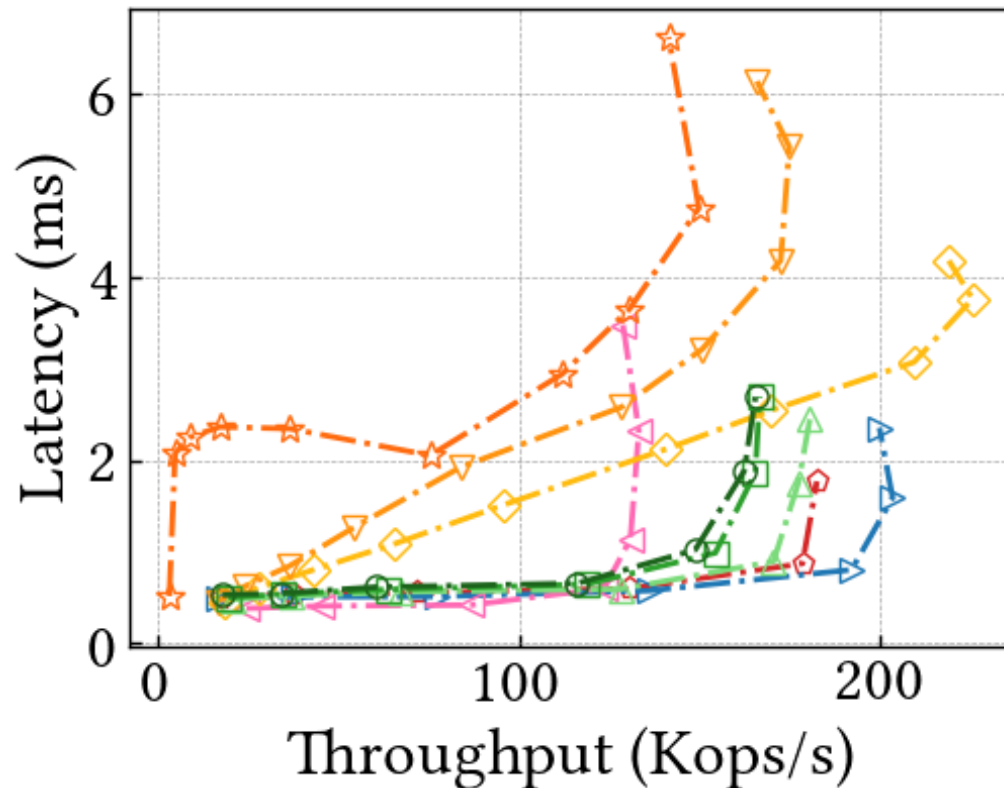
(a) 1 shard



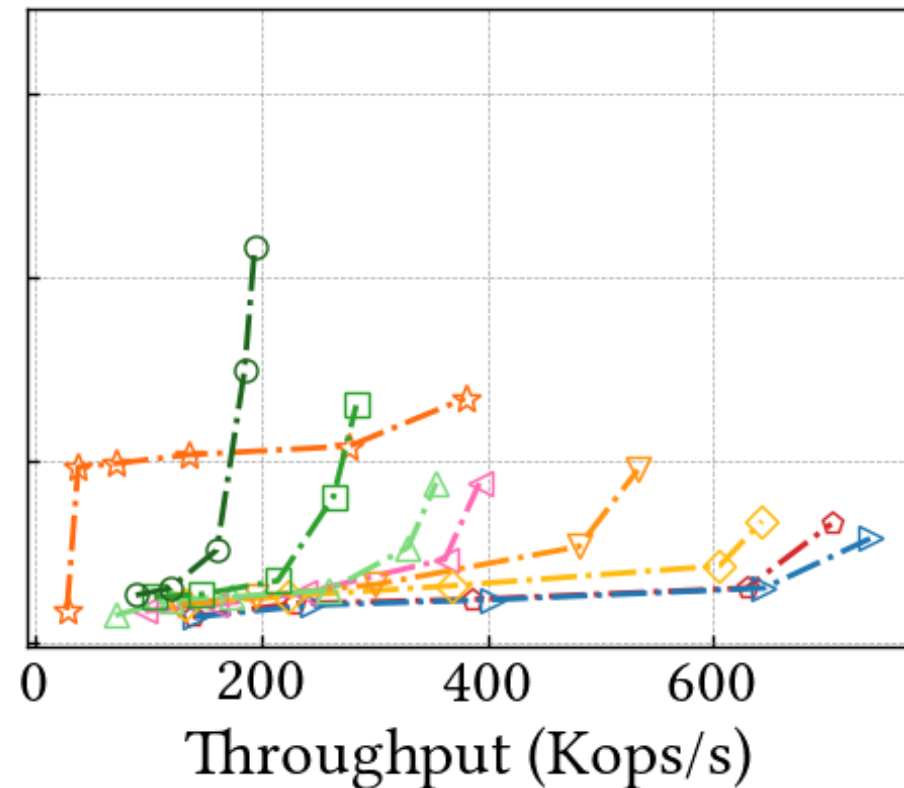
(b) 7 shards

Evaluation – Throughput Latency

- Multi-Paxos FVC
- Multi-Paxos FVC (-)
- Copilot
- EPaxos (0.01 skewness)
- EPaxos (0.99 skewness)
- EPaxos (10.0 skewness)
- Avicenna-0%
- Avicenna-5%
- Avicenna-25%

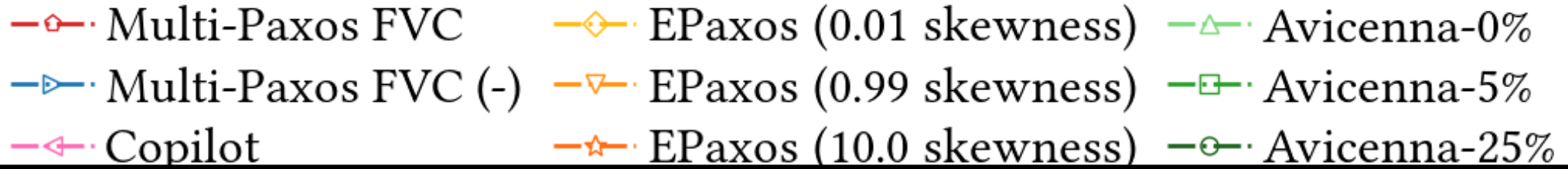


(a) 1 shard



(b) 7 shards

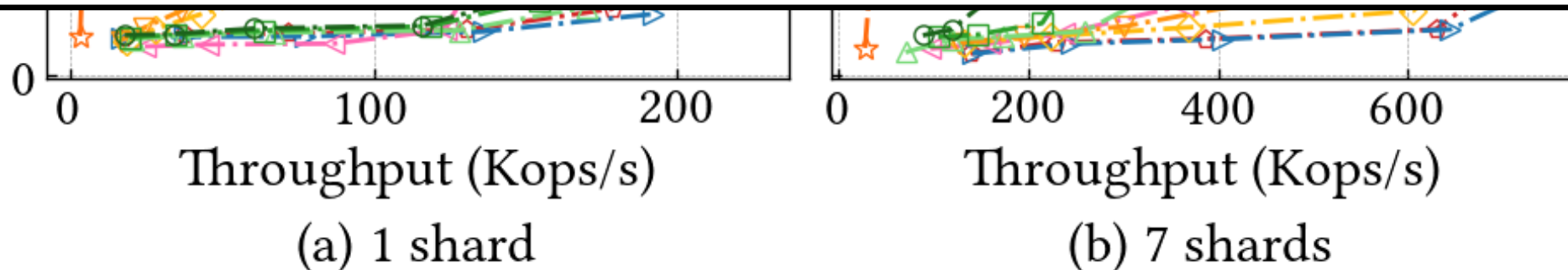
Evaluation – Throughput Latency



When running 1 shard, Avicenna shows 75-90% throughput as Multi-Paxos

When running 7 shards, Avicenna only achieves about 1/3 throughput as Multi-Paxos

Throughput is the design tradeoff of Avicenna.



Outlines

- Replicated State Machines
- Fail-slow Faults in Distributed Systems
- Overview of Avicenna
- Detailed design of Avicenna
- Evaluations
- Discussion and Conclusion

Discussion

- Discussion:
- Only tolerates one fail-slow replica
 - Counterfactual evaluation can be used in other purposes
example: dynamically choosing a better leader

Conclusion

Avicenna: fail-slow fault-tolerant RSM without sacrificing normal-case latency

- Shadow processing:
simulates latency under shadow leader
- Independent real and shadow processing:
no conflict proposals, thus low normal-case latency
- Fast Rotation protocol:
promotes the shadow leader when real leader is slow

Thank you!